

MIT RES.9-009: Introduction to Computational Neuroscience with Neuroblox (January IAP 2025)

教材级中文讲义

基于 MIT OpenCourseWare、Neuroblox 官方课程页与 MIT Video Productions 公开录播重建

2026 年 4 月 16 日

目录

课程说明	2
前言	2
如何使用本书	2
章节目录	3

课程说明

本书由 ‘lectures/’ 下通过 evaluator 与 validator 的章节工作区合并而成；视频侧的公开播放入口以 MIT Video Productions 的 ‘IAP 2025’ playlist 为准，章节边界则以官方 Neuroblox 课程页单元结构为准。

前言

本书是对 MIT RES.9-009: Introduction to Computational Neuroscience with Neuroblox (January IAP 2025) 的一次 coverage-first 中文重建。全书以 MIT OpenCourseWare 课程页、Neuroblox 官方课程页、MIT Video Productions 的公开录播，以及各单元页面中的官方图、代码与 references 为证据基础，目标不是做一份简短摘要，而是提供一部可自学、可引用、可复习的教材式讲义。

与常见的课程笔记不同，本书把“来源完整性”放在优先级最高的位置。凡是官方页面、字幕、代码块、图示或 reading 中有独立教学价值的内容，都尽量在正文中获得明确落点；无法直接取得的 slide PDF、页面缺图或非教学性片段，则会在工作区的 omission_log.jsonl 中留下可审计记录，而不是被静默跳过。

本次课程视频的公开 YouTube 入口，最终以 MIT Video Productions 的 IAP 2025 playlist 为准，其中包含本课程 11 条编号录播。与此同时，章节顺序与内容边界仍以官方 Neuroblox 课程页的单元结构为主，因为该课程页同时提供了文字讲解、代码与图示，是比纯视频列表更完整的教学证据层。

就“本书是否依照课程视频而来”这一点，可以明确回答：**是的，但不是机械地只抄视频**。其中 11 章属于明确的 video-backed 重建，正文直接受官方 YouTube 字幕与页面内容共同约束；另外 3 章 (08、09、12) 没有官方嵌入视频，因此采用 page-backed 的方式，以官方课程页、图、代码和 reading 为主证据来补足。也就是说，本书整体遵循课程视频与课程页的联合证据，而不是脱离课程材料的自由重写。

从使用角度看，你可以把这本书当成三种东西的结合：

- 一部按课程顺序重建的中文教材；
- 一套带证据侧车的可审计课程档案；
- 一个可以继续增补、修订、重新导出的研究型工作区。

如果你需要正式引用本书中的某一部分，建议同时引用对应章节、课程原始单元页以及相关视频或 reading。正文是中文重建与教学化重写，而事实根据与图文出处则应回溯到各章工作区中的 source_manifest.json、figure_manifest.json 和 eval_reports/pass_##.json。

如何使用本书

建议先按章节阅读主文，再回看对应 lecture workspace 里的 lecture_plan.json、coverage_units.jsonl、figure_manifest.json 和 eval_reports/pass_##.json。如果你只是把这本书当教材来读，可以直接沿章节顺序通读；如果你把它当研究资料来用，则应同时查看每章的 source 与 coverage 侧车。

如果某章尚未进入 book/，优先看该章的 omission log 与 repair log，再决定是补源、修复，还是接受缺口。对于已经进入本书的章节，也仍建议把 omission log 当作正文的附属阅读，因为它会告诉你哪些地方是“证据不足但已显式处理”的，而不是“完全不存在问题”。

为避免逐讲原始版面中的局部页码与合并后的全书页码混淆，all-in-one PDF 会在每一页右上角额外标出统一连续页码；全书引用与翻页时请以该统一页码为准。

若要引用本书中的具体论断、图或代码说明，推荐采用三层引用方式：

- 第一层：引用本书的章节标题与统一页码；
- 第二层：引用该章对应的官方课程页或官方视频；

- 第三层：必要时回到工作区中的 `figure_manifest.json`、`source_manifest.json` 与 `references` 条目，给出更精确的原始出处。

简言之：本书适合连续学习，也适合带着审计意识来读。正文负责教学组织，侧车负责来源与边界条件。

章节目录

章号	章节标题	状态
01	Intro to Neuroblox	pass
02	The Neuroblox GUI	pass
03	Course Structure and Introduction to Julia	pass
04	Differential Equations and Plotting in Julia	pass
05	Blox and Connections in Neuroblox	pass
06	Neurons, Neural Masses and Sources	pass
07	Circuit Models in Neuroblox	pass
08	Biomimetic Model of Corticostriatal Micro-assemblies	pass
09	Pyramidal-Interneuron Gamma Network	pass
10	Decision Making in a Circuit Model	pass
11	Synaptic Plasticity and Reinforcement Learning	pass
12	Parameter Fitting using Optimization	pass
13	Parameter Fitting using Spectral Dynamic Causal Modeling	pass
14	Experimental Design	pass

课程笔记

MIT RES.9-009 课程讲义第 01 讲: Intro to Neuroblox

MIT RES.9-009 课程整理 & Codex

2026 年 4 月 15 日



视频作者/频道: MIT Video Productions

发布日期: 2025-01-13

视频时长: 57:16

视频链接: <https://www.youtube.com/watch?v=2dbAePEmbhQ>

目录

1 课程开场、工作方式与整体路线 2

1.1 本章小结 2

2 什么是计算建模：模型要回答什么问题 2

2.1 本章小结 3

3 Neuroblox 的多尺度科学问题 3

3.1 本章小结 4

4 从相关性到动力系统：为什么要模拟 4

4.1 本章小结 4

5 GUI、平坦大脑与模块化 workflow 5

5.1 本章小结 6

6 为什么是 Julia：性能、可组合性与平台基础 6

6.1 本章小结 7

7 总结与延伸 7

7.1 拓展阅读 7

7.2 本章小结 7

1 课程开场、工作方式与整体路线

这门课一开始就把两个事实说得很清楚。第一，课程本身是一个 IAP 里的短时密集工作坊，讲者希望把大量时间留给你在 notebook 里动手。第二，Neuroblox 还是一个正在快速迭代的系统，视频里的讲者甚至明确提醒你们是第一批非开发者用户之一，因此遇到 bug 不要惊讶，而要把它当作课程流程的一部分。

从官方课程页和 MIT OCW 页面看，这门课的定位也完全一致：它强调 hands-on model building in Neuroblox and Julia，强调从 literature 中的模型出发，最后把模型拟合到神经数据，并进一步探索 drugs 和 stimulation probes 这类干预。换句话说，视频开场讲的“先给背景，再做练习”，并不是随口一说，而是官方课程结构本身的一部分。

为什么一开始就要报告 bug

讲者不是在推卸责任，而是在定义这门课的协作方式。Neuroblox 仍处于 beta 阶段，很多问题会出现在真实使用场景里，而这些场景恰好是开发者事先很难完全预料到的。换句话说，学生的“卡住”本身就是产品成熟化的重要反馈信号。

从教学组织上看，这门课明显不是“先讲完理论，再做一次大作业”的传统结构，而是“每次先给一个 10 到 15 分钟的概念框架，然后把剩余时间留给 notebook 和练习”。这一点非常重要，因为 Neuroblox 的目标并不是只让你理解术语，而是让你学会在同一套生态里把模型、参数、连接规则、仿真和分析串起来。

这门课的路线也很清楚：先完成安装和环境准备，再进入 Julia，接着学习微分方程与绘图，然后进入 Neuroblox 的 blocks、连接、神经元、神经质量、回路、决策、突触可塑性、参数拟合、谱 DCM 和实验设计。你可以把它理解为一条从“能跑起来”到“能搭系统”再到“能做推断和设计”的路径。

整门课的组织逻辑

课程不是按“软件功能列表”展开，而是按“科学建模能力”展开。最前面解决环境和语言问题，中间解决单个模块和单个回路问题，后面再解决模型比较、拟合和实验设计问题。这个顺序本身就体现了 Neuroblox 的目标：它不是单纯的画图工具，而是一套面向计算神经科学研究的建模语言与工作流。

1.1 本章小结

本讲的开场重点不在技术细节，而在工作方式：Neuroblox 还是 beta 软件，课程鼓励真实使用、真实报错和真实修复；同时，后续所有内容都围绕 notebook 实操展开，而不是只停留在概念演示。

2 什么是计算建模：模型要回答什么问题

讲者用一个很重要的观点切入：模型的好坏，不在于它是不是“把所有细节都装进去”，而在于它是否与研究问题匹配。这个意思并不抽象。若研究目标是解释所有可能的现象，那么模型就必须接近系统本身；但现实研究里，我们几乎总是要对分辨率做战略选择。

这也是为什么讲者借“Lemma the cat”的玩笑来说明模型哲学。一个模型如果能把对象的每个特征都一一复刻，就会迅速滑向过拟合，最后反而失去可解释性。更好的做法是先问：我要回答的是哪类问题，哪些变量是必要的，哪些细节可以暂时抽象掉。

模型分辨率是设计选择，不是默认值

如果你的问题是“为什么这个病程会沿着某条轨迹演化”，那你需要时间和动力系统。若你的问题是“这个系统在这个尺度上由哪些部件组成”，那你需要结构化的组件和连接。Neuroblox 选择的不是某一种“唯一正确”的分辨率，而是允许你在不同尺度上构造模型，并让这些尺度彼此对接。

这直接引出 Neuroblox 的定位。讲者明确把它描述成一个面向生物医学工程的工具，关注的是“机制整合”和“治疗相关靶点”。它并不只是为了复现某个漂亮的神经科学图，而是要让你能把病理机制、药物作用、刺激装置和行为输出放在同一个建模框架里讨论。

Neuroblox 关注的问题也很具体。比如痴呆、抑郁、成瘾、双相障碍、精神病性障碍都不是简单的二元标签，而是有轨迹、有阶段、有退化、有起病时间差异的动态过程。讲者反复强调，临床表型的差异并不意味着底层回路一定不同，很多时候只是同一回路以不同方式失调。

- 痴呆的问题不是“有或没有”，而是“如何进展、谁有风险、何时加速”。
- 双相障碍更像状态切换问题，而不是静态分类问题。
- 成瘾与抑郁也常呈现长期轨迹，而不是单次测量即可解释的离散标签。

2.1 本章小结

好的模型不是“越细越好”，而是“分辨率与问题匹配”。Neuroblox 的设计哲学正是围绕这个原则展开的：让模型既能跨尺度，又能保持研究问题所需的可解释性。

3 Neuroblox 的多尺度科学问题

从课程和讲者的描述看，Neuroblox 要处理的是一个跨度很大的尺度链条。最低层可以是离子通道和受体，中间层可以是神经元、神经质量和脑区，更高层可以是回路、脑区间交互、神经影像信号，最终输出则可能是任务表现、情绪状态或者临床症状。

尺度层级	典型对象	典型问题
分子/受体层	离子通道、受体、代谢	药物怎样改变底层动力学
细胞层	神经元、放电模式	参数变化如何改变尖峰样式
中尺度	神经质量、脑区	振荡与平均活动如何演化
回路层	多脑区网络	DBS 或 TMS 会怎样影响回路
行为层	任务、情绪、症状	模型能否再现临床轨迹

讲者给出的例子非常一致。对于药物，问题不仅是它影响神经递质水平，还要看受体动力学、连接规则和更高层回路如何一起变化。对于代谢，问题不仅是“能量代谢异常是否存在”，而是“代谢异常通过什么路径影响脑功能”。对于刺激，问题不仅是“刺激是否有效”，而是“刺激如何沿着回路反馈回来，并在行为上留下轨迹”。

在这里，Neuroblox 的独特之处不是“能不能模拟一个单独模块”，而是“能不能把模块拼成一个可解释的多尺度系统”。讲者提到的 neurodiagnostic、pharma module、metabolic module 和 neuromodulation module，都是围绕这一目标展开的扩展方向。

多尺度建模的核心不是堆叠，而是贯通

如果一个回路模型只能给出静态连接图，那它很难回答“为什么这个患者在两周后恶化”这样的问题。Neuroblox 要做的是让代谢、刺激、连接、受体和行为都进入同一条时间轴上，然后观察系统如何在反馈中演化。

3.1 本章小结

Neuroblox 要解决的是多尺度、动态、可干预的问题。它把分子、细胞、脑区、回路和行为放到同一个框架里，目标不是分类而是模拟，不是静态描述而是轨迹解释。

4 从相关性到动力系统：为什么要模拟

讲者在这一讲里花了很多时间区分两类“看起来像模型”的东西。第一类是 neuroimaging 里常见的 connectivity、GLM、ICA、meta-analysis 这类统计方法，它们可以帮助你找出相关模式、提高可靠性、做 feature extraction 或 decoding。第二类才是 Neuroblox 关注的 computational neuroscience：一个能够被真正仿真的动力系统。

这两类方法各有用处，但任务不同。统计方法适合问“哪些区域共同激活”“哪些特征能区分状态”“某个模式是否可重复出现”；动力系统则适合问“这个系统在时间上如何演化”“一个药物或刺激会怎样改变轨迹”“多级反馈会不会导致状态翻转或崩溃”。

讲者特别强调，相关性方法在有反馈的系统里会非常脆弱。因为当你把多个 pairwise correlation 一层层串起来时，误差会乘法式地放大。即使单条边的相关系数看起来不错，一旦进入多边网络，整体解释力也会迅速下降。换句话说，“谁和谁相关”并不能自动还原“系统如何工作”。

这也是为什么讲者用 glucose-insulin regulation 作为一个很好的 ODE 类比。你吃一个糖块，系统先响应 glucose，再响应 insulin，再把 glucose 拉回去；如果调节失衡，你会看到不同的长期轨迹，甚至是 overshoot and collapse。这个例子非常适合说明：临床问题往往是时间尺度很长的动力学问题，而不是一次性测量可以解决的静态分类问题。

可以把讲者在这里的意思抽象成一个最一般的状态方程：

$$\dot{x}(t) = f(x(t), u(t); \theta)$$

其中， $x(t)$ 表示系统在时刻 t 的状态， $u(t)$ 表示外部输入或干预， θ 表示参数， f 表示决定系统如何随时间演化的动力学规则。这里的写法不是在替代讲者的具体例子，而是把“输入、状态、参数和时间演化”这几个核心关系抽象出来，方便你把 glucose-insulin regulation、药物作用和刺激响应放进同一类动力系统里理解。

为什么仿真比相关性更适合回答临床问题

临床和实验室的观察窗口很短，但疾病演化可能持续数月、数年甚至数十年。只有把系统写成能时间推进的动力系统，你才可能讨论慢变量、长期轨迹、状态翻转和治疗后的反馈效应。

这一讲里还有一个重要的观点：Neuroblox 不是要完全替代统计方法，而是要把问题推进到“机制可模拟”的层面。对于神经影像语境中的网络、回路和 computational neuroscience，讲者提醒你要区分两件事：一件是用统计工具提取模式，另一件是构造可解释、可干预的动态模型。前者回答“像什么”，后者回答“为什么会这样、如果我干预会怎样”。

4.1 本章小结

相关性分析擅长描述模式，动力系统擅长解释轨迹。Neuroblox 站在后者一边，因为它要处理药物、刺激、代谢和行为的时间演化，而不是只给出静态标签。

5 GUI、平坦大脑与模块化 workflow

讲者把 Neuroblox 的核心界面讲得很具体：它有一个 flat brain，也就是把整张脑的结构展开成平面地图，让你把 block 放到对应的 brain region 上；它有节点和边、参数和 simulation 等侧栏；它还允许你通过 Alt 或 Option 方式建立连接、通过拖拽改变边的几何形状，并在同一界面里运行仿真、观察结果和修改参数。

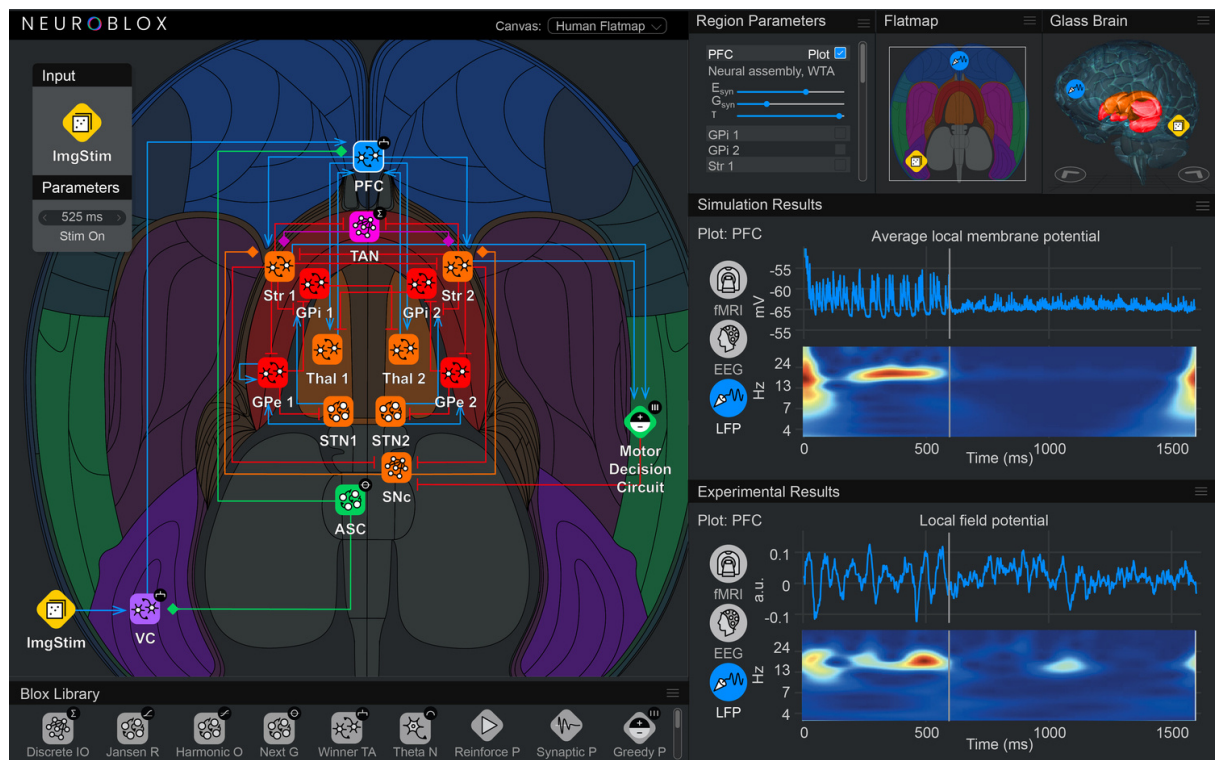


图 1: Neuroblox GUI 截图。画面中同时展示了 flat brain 视图、节点布局、仿真结果和实验结果面板，能直观体现 Neuroblox 的“回路 + 地理 + 输出” workflow。¹

这张图的价值在于，它把抽象的模块化思想具体化了。Neuroblox 不是只让你写一串方程，而是让你把方程封装成 block，再把 block 放到脑区上，最后把整个系统当成一个图来编辑。讲者甚至强调，系统的层次关系是“blocks of blocks of blocks”。这不是口号，而是整个平台的设计原则。

这种设计的关键优点是可复用和可比较。若不同团队都用同一套模块定义方式，那么模型之间就更容易对比、整合和扩展，而不是每个组都用自己的术语、自己的接口、自己的连接规则。讲者的意思可以归纳为：Neuroblox 不是单个模型，而是一个方便把模型统一管理、统一比较的建模环境。

更重要的是，Neuroblox 的模块不是空壳。讲者举了几个非常具体的例子：lateral inhibition、winner-takes-all circuits、cortical region、pharma module、metabolic module、neuromodulation module。每个模块都对应一种真实的神经科学功能假设，而不是为了视觉上好看而做的装饰。

¹ 视频画面时间区间：00:33:45–00:36:10。

- ‘nodes and edges’ 用来编辑局部结构，节点对应 block，边对应连接规则。
- ‘parameters’ 面板允许你把许多连接统一到一个共享参数上，便于整体调参。
- ‘simulation’ 面板允许你设定时长、容差、求解器和时间步进策略。
- 右侧输出面板让你把模拟结果和实验结果并排比较，便于直接检视模型是否抓住了数据里最重要的动态特征。

讲者还强调了一个很现实的问题：GUI 是给更多人用的，而不是只给“住在洞穴里的硬核计算神经科学家”用的。实验人员、理论人员、临床方向的人都应该能够理解和使用它。所以 GUI 的设计既要保留神经科学直觉，也要支持复杂的模拟和数据分析。

平坦大脑为什么有意义

讲者采用 Mercator projection 的类比，说明为什么要把脑展开成 flat map。目的不是做一个好看的背景，而是让你在一个视图里同时看到 cortical、subcortical、front/back 等不同区域，并把机制图层、输入、输出和区域地理叠加起来。

讲者最后还提醒，Julia 是编译型语言，所以第一次运行时往往会先付出编译开销；但一旦编译完成，后续运行就会快得多。长期看，JuliaHub 这类云平台会成为重要的托管方式。

5.1 本章小结

Neuroblox 的 GUI 不是附属品，而是平台的一部分。它把模块、连接、地理位置、参数和输出统一到 flat brain 上，使研究者既能沿着神经科学直觉建模，又能沿着软件工程原则复用和比较模型。

6 为什么是 Julia：性能、可组合性与平台基础

讲座后半段给出了选择 Julia 的核心原因。第一是所谓 two-language problem。高性能科学计算通常会把“开发快”和“运行快”分成两套语言来做，例如用 Python 组织逻辑、再把性能关键部分交给 C 或 C++。讲者认为 Julia 的价值就在于把这两件事统一起来：它像 Python 一样易写，又能像 C 一样跑得快。

这种性能并不是空口说白话，而是和 Julia 的编译模型有关。讲者强调 Julia 是 compiled language，并提醒第一次运行会因为编译而较慢，但后续会快很多。更关键的是，Julia 的 solver 生态非常成熟，尤其在 ODE、SDE、DAE 以及延迟微分方程等场景下都很强。这对 Neuroblox 这种大量依赖动力系统仿真的平台非常重要。

Julia 的第二个优势是自动微分 (automatic differentiation)。讲者特别强调，它之所以关键，是因为 Neuroblox 需要 parameter fitting、optimization 和 Bayesian posterior estimation。换句话说，只有模型可微，参数估计、模型比较和逆问题求解才真正可做。

为什么可微性会影响研究范式

可微不只是“能不能算梯度”的技术细节，而是决定你能否从“手工调参”走向“自动拟合”。对于 Neuroblox 来说，这意味着模型不仅能跑，还能拿真实数据来拟合、比较和排序。

Julia 的第三个优势是 composability，也就是可组合性。讲者用一类方程系统的连接过程来说明：你可以把一个系统写成方程，再把两个 block 的输出和输入连起来，最后把整个连接关系整理成可以求解的 ODE system。这个过程和 Neuroblox 的 block 设计几乎是同构的。

更进一步，讲者解释了为什么大型系统会推动 Graph Dynamics 这类库的发展。因为当系统非常大时，直接做符号化简会很慢，编译时间甚至会超过运行时间。Graph Dynamics 把系统视为 graph，用节点和边来表达连接规则，从而大幅提高大型图系统的编译效率。这一点对 Neuroblox 尤其重要，因为它面对的就是大规模、层级化、可复用的神经系统图。

最后，讲者还提到了 Julia 社区、文档和学习资源。对一个快速演化的平台来说，生态并不是附属品，而是生产力的一部分。Neuroblox 之所以能把课程、GUI、动力系统、拟合和优化连接在一起，背后依赖的正是 Julia 的语言设计、相关数值工具链和可组合建模方式。

6.1 本章小结

Julia 不是仅仅“因为快”才被选中，而是因为它同时满足三件事：交互式开发、数值性能和可组合建模。对 Neuroblox 来说，这三点分别对应 notebook 开发、ODE 求解和 block 级别的系统组装。

7 总结与延伸

这一讲真正建立起来的不是某个具体模型，而是一套建模观。Neuroblox 的目标是把神经科学里常见的机制单元标准化成可组合 block，再用 Julia 的性能和可微性把这些 block 变成可仿真、可拟合、可比较的模型系统。它关心的不是“有没有一个漂亮的静态图”，而是“是否能把机制、干预和行为放在同一条时间轴上解释清楚”。

从研究方法上看，这一讲明确区分了统计相关和机制模拟。前者适合做模式发现、可重复性分析和 decoding；后者适合做干预、长期轨迹和状态演化。Neuroblox 选择的是后者，但并不排斥前者，而是试图把前者的经验转化为后者可使用的结构化输入。

从软件工程上看，这一讲也很清楚地告诉你，Neuroblox 的核心不是“库里有多少函数”，而是“有没有一套共通的模块语义”。只要模块语义统一，社区模型就能被整合、比较、复用和扩展；如果模块语义不统一，再多的模型也只是彼此孤立的艺术品。

后续学习应该抓住的三个主线

第一，先把 Julia 和 notebook 工作流跑通。第二，理解 differential equations、block 连接和 plotting 这三者如何衔接。第三，再回到 Neuroblox 的 block、连接、层级命名和 GUI，理解为什么这个平台能把模型组装成研究系统。

7.1 拓展阅读

如果你想继续顺着这一讲往下走，最值得先看的官方材料是课程主页、MIT OCW 页面，以及后续单元里的 Getting Started、Introduction to Julia、Differential Equations in Julia 和 Plotting with Makie。它们共同组成了这门课的“语言层”和“平台层”。

7.2 本章小结

这一讲的核心结论可以浓缩成一句话：Neuroblox 试图把神经科学中的机制、尺度、干预和比较统一为一个可组合、可微分、可仿真的系统，而 Julia 正是承载这一目标的语言与生态。

课程笔记

MIT RES.9-009 第 02 讲: Neuroblox 图形界面

MIT RES.9-009 课程整理 & Codex

2026 年 4 月 15 日



视频作者/频道: MIT Video Productions; 讲者 Helmut H. Strey

发布日期: 2025-04-10 (讲课日期: 2025-01-13)

视频时长: 20:51

视频链接: https://www.youtube.com/watch?v=X1BRJps84_E

目录

1 GUI 入口、 at brain与协作状态 2

1.1 本章小结 3

2 Harmonic Oscillator: 最小可运行的双节点系统 3

2.1 本章小结 4

3 Cortical demo: 从单块到双区域 5

3.1 本章小结 6

4 总结与延伸 6

4.1 本章小结 6

证据层说明

本讲没有找到独立的官方 slide PDF。正文主要依据字幕、官方课程页和视频帧；课程页中的 logo 首图只承担品牌展示，不承担本讲教学内容，因此已在 `omission_log.jsonl` 中明确记录。

1 GUI 入口、at brain 与协作状态

这一讲一开始就把两件事说得很明确。第一，演示的是 Neuroblox 的新 GUI，而不是 JuliaHub 上那个完全稳定的版本，因为还有一些功能没有完全接通。第二，Neuroblox 现在已经进入可以给学生试用的阶段，但它仍然是一个快速迭代的系统，所以讲者反复提醒，遇到 bug 不要惊讶，要把它当成真实协作的一部分。

官方课程页在这个层面上也给了同样的背景：Neuroblox 被定位成一个基于 Julia 的模块化平台，支持从单个 block 到更大系统的拼接，并允许通过 graphical interface 直接组装模型。第 2 讲的作用，就是把这个抽象定位落到界面上，告诉你到底怎么点、怎么连、怎么跑。

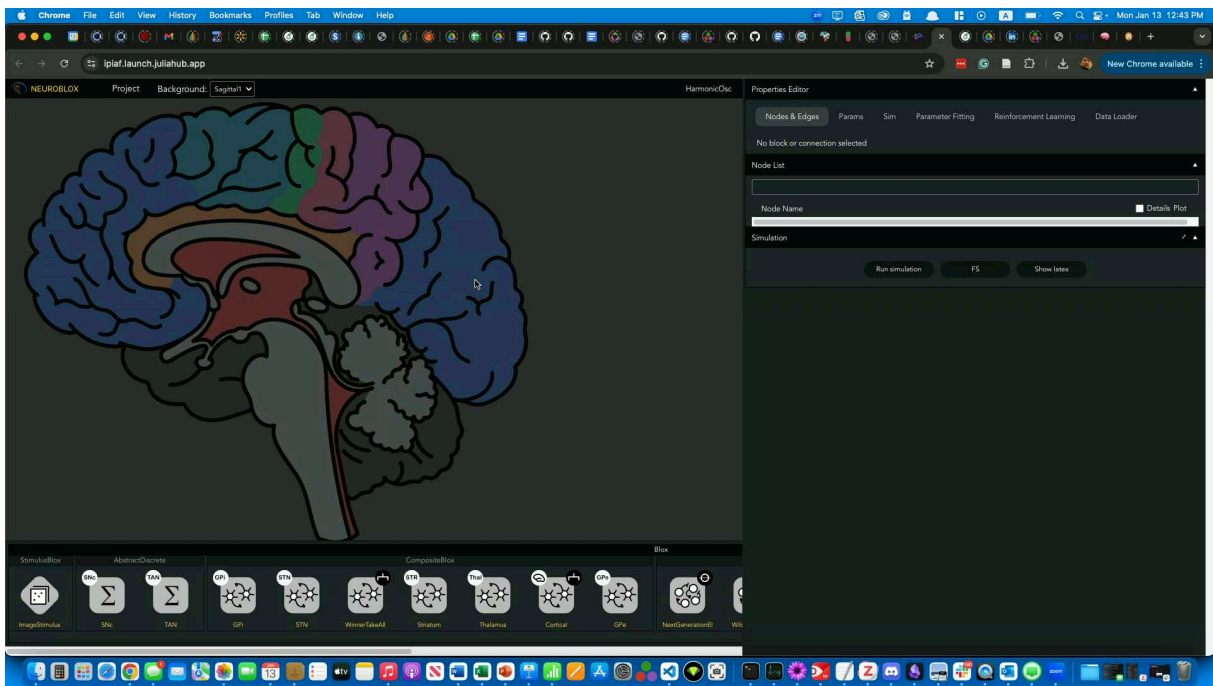


图 1: GUI 总览: flat brain、节点区和右侧属性面板同时可见。这个画面对应讲者解释 canvas、sidebar 和 simulation 面板的段落（约 00:08:05）。

最核心的交互语义其实很简单，但必须记牢。canvas 上的脑区是一个 flat map，也就是把 brain region 展开到平面上；把 block 放到某个区域上以后，你就可以通过 Alt 或 Option 建边。左键点击是选择，左键拖拽边上的 handle 可以改变边的几何形状。换句话说，GUI 里既有“结构放置”，也有“连接编辑”，这两者不是同一个动作。

几个必须记住的操作语义

Alt 或 Option 用来建边；左键点 node 是选 node；左键点 edge 是选 edge；拖拽 edge anchor 可以改边形状。这个交互设计的目的，是让你在不离开 flat brain 的情况下同时编辑结构和动力学。

右侧面板的结构也很明确。Nodes & Edges 用来改节点和连接的属性，Params 用来集中管理参数，Sim 负责仿真设置，Parameter Fitting 和 Reinforcement Learning 目前都还不够稳定，Data Loader 则负责加载数据。讲者特地提醒，不要把还没完全工作的功能当成课程主流程的一部分。

当前可用性提醒

讲者明确说过，parameter fitting 目前不建议依赖，reinforcement learning 也还没有在这个 GUI 上形成稳定经验。教学上应把它们当成未来功能，而不是当前练习的主线。

另一个很现实的细节是云端启动时间。连接 JuliaHub 后，系统要在 AWS 里预留计算资源，随后 Julia 又要进行 JIT 编译，所以第一次点开和第一次跑仿真会慢很多。讲者把这一点解释得很直白：不是 GUI “坏了”，而是云服务和编译开销在前面排队。

你还可以把这一讲里的平台架构理解成三层协作：前端负责界面，graphics server 负责作图，Julia backend server 负责真正的计算。讲者说得很清楚，这三者是互相通信的；等仿真结束，数据先传给图形服务器，再回到界面里显示。这也是为什么 GUI 看起来是“点一下就出图”，但背后其实是一条很长的异步链路。

1.1 本章小结

这一部分不是在讲“GUI 好不好看”，而是在讲 Neuroblox 的操作语义：flat brain 负责空间布局，Alt/Option 负责连边，右侧面板负责参数与仿真，云端和 Julia 则负责执行。把这些语义记牢，后面的示例才有操作基础。

2 Harmonic Oscillator: 最小可运行的双节点系统

讲者接下来选了一个最小但很适合练手的例子：harmonic oscillator neural mass model。它足够简单，能让你快速摸到 GUI 的所有关键操作；它也足够接近动力系统的本体，让你理解为什么 Neuroblox 不是“画图工具”，而是“搭动力学系统的工具”。

最基本的连续形式可以写成

$$\dot{x}(t) = y(t), \quad \dot{y}(t) = -\omega^2 x(t) - \gamma y(t) + I_{\text{couple}}(t).$$

如果只看无阻尼版本，那么把一阶系统消元后就是

$$\ddot{x}(t) + \omega^2 x(t) = 0,$$

解是正弦或余弦，频率由 ω 控制。讲者进一步提醒，若只有阻尼项而没有合适的耦合，这个系统会衰减掉，不会形成你想要的稳定振荡。所以在 GUI 里，他不是只放一个 oscillator，而是放两个互相作用的 oscillator。

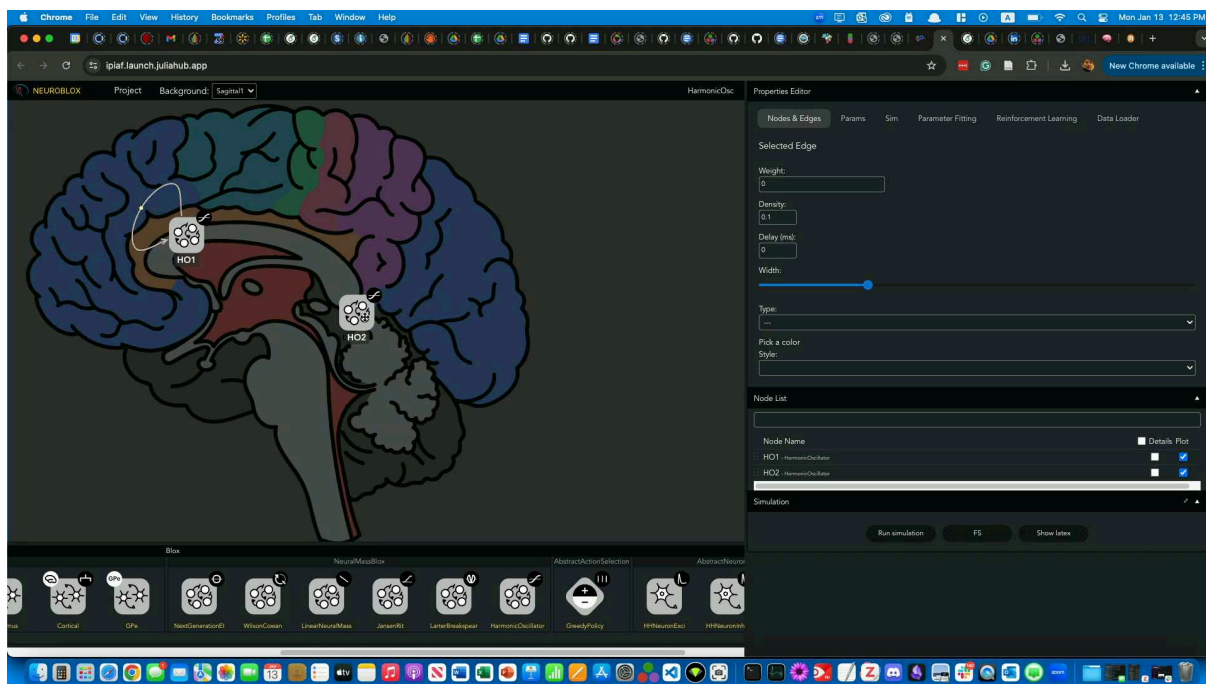


图 2: 两个 harmonic oscillator, HO1 和 HO2 已经放到 canvas 上, 并且能在右侧看到边的属性面板。这个状态对应讲者说明 self-interaction、Alt 拖拽和命名的片段 (约 00:10:02)。

讲者给出的实操顺序很稳定: 先在底部 block 面板里找到 `HarmonicOsc`, 拖到 canvas 上, 重命名为 HO1 和 HO2, 然后给每个 node 加 self-interaction, 再通过 Alt/Option 从一个 node 连到另一个 node。起始参数并不是理论定值, 而是一个经验起点。讲者建议先把 self-interaction 设成 inhibitory, 也就是大约 -1, 再把跨节点连接设成更强一些的正值, 例如 5 和 6, 然后观察是否能把系统推到振荡区间。

为什么要用两个 oscillator

单个 damped oscillator 更像一个会衰减的振动器; 两个相互作用的 oscillator 才更像一个可以通过耦合维持节律的系统。讲者在这里的重点不是某个固定参数, 而是让你感受“自耦合 + 互耦合”如何改变系统的相图。

GUI 里还给了一个很有用的事实: NeurobloX 的时间单位是 milliseconds, 所以输入的 1 不是秒, 而是 1 ms。仿真面板还允许你设定 duration、relative tolerance、absolute tolerance, 以及 stiff / non-stiff solver。讲者的建议是, 先让系统跑通, 再去调这些数值细节; 第一次运行还会慢一些, 因为 Julia 需要先编译相关包, 而云端环境里这一步往往最耗时间。

当仿真跑完以后, 右侧的结果面板会显示轨迹, 你可以把图放大、挪到另一侧、切换查看 node 的参数, 甚至打开 `Show latex` 看系统方程。讲者还顺手提到, 一个想当然的小问题其实很适合作为练习: 如果你已经把这个系统调到会振荡, 如何把它的频率加倍? 这个问题没有在现场给出唯一答案, 它本来就是为了让你理解参数、耦合和频率之间的关系。

2.1 本章小结

这一部分的价值在于把 GUI 操作和动力系统直觉绑在一起。你不是在“摆两个图标”, 而是在构造一个能通过 self-interaction 和 cross-coupling 进入振荡区间的最小系统。会搭这个例子, 后面

的 cortical demo 才不是黑箱。

3 Cortical demo: 从单块到双区域

第三段把系统规模往上推了一层。讲者先新建一个 `cortical demo` 项目，然后从 `composite blocks` 里选出 `Cortical block`，把它放到 `canvas` 上并命名为 `C1`。这个例子说明了一件很重要的事：Neuroblox 的核心不是单个 `neuron`，而是 `blocks of blocks`。你可以把 `WTA`、`cortical region`、`stimulus block` 继续封装，再把这些封装好的块放进更大的系统。

在参数层面，`Cortical block` 会出现若干控制项。最常见的有 `N_wta`、`N_exci`、`G_syn_exci`、`G_syn_inhib` 和 `G_syn_ff_inhib`。讲者的实用建议很直接：如果默认的 `N_wta = 1` 太慢，就先把它减到 5，因为你当前的目标是理解 workflow，不是让一次演示把时间烧完。

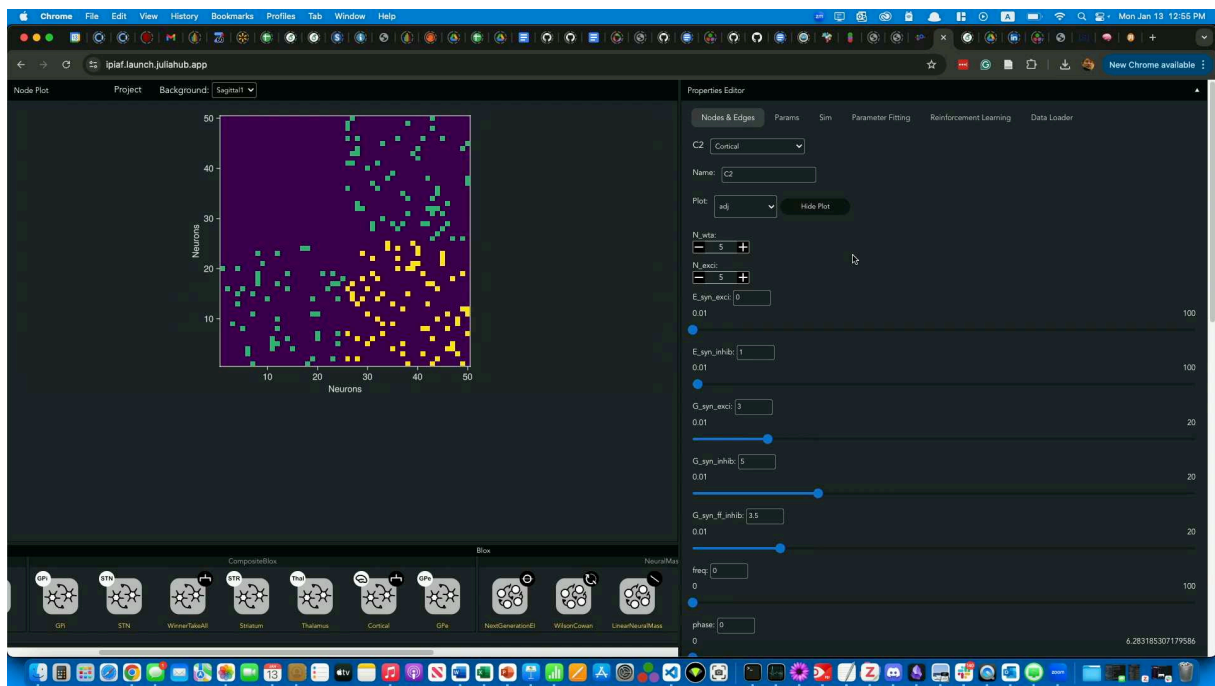


图 3: 双 cortical region 的结果视图：左侧显示 adjacency matrix / cross correlation 类输出，右侧是 C2 的参数面板。这个画面对应 00:19:22 左右的演示。

仿真完成后，GUI 提供的不只是一个时间序列，还包括多个特殊视图：`raster plot`、`stack plot`、`adjacency matrix`、`power spectrum`。讲者强调，`adjacency matrix` 里看到的连接是按 `density` 随机采样出来的，`density` 控制的是连接概率，不是突触权重；权重仍然要另行设置。这个区别非常重要，因为它直接决定你是在调结构稀疏性，还是调动力学强度。

density 不是 weight

`density` 决定“有多少条边会被采样出来”，`weight` 决定“已经采样出来的边有多强”。这两个概念必须分开，不然你会把结构问题和动力学问题混成一件事。

讲者随后把系统扩展成两个 cortical regions, `C1` 和 `C2`。在这个过程中，讲者又一次强调了 GUI 的实用细节：如果结果图还停留在旧参数上，就再点一次 `Run simulation`；如果 region 太大，仿真会明显变慢；如果想看更深的结构，还可以打开 `Show latex` 查看方程。更进一步，如果你愿意，

Neuroblox 还可以把 image stimulus block 接到 visual cortex，再把信号送到 prefrontal cortex。讲者把这部分留给后面愿意继续试的人，而不是把它当成今天必须完成的任务。

3.1 本章小结

这一段把 GUI 从“单个 oscillator 的练习场”推进成了“可拼装的 cortical system”。Cortical block、C1/C2、adjacency matrix 和 power spectrum 共同说明：Neuroblox 的可视化不是装饰，而是模型结构本身的一部分。

4 总结与延伸

这一讲真正教你的不是某个按钮，而是一整套操作顺序。先在 flat brain 上放 block，再用 Alt/Option 建边；先用 harmonic oscillator 验证你是否理解 node / edge / parameter 的关系，再把同样的逻辑推广到 cortical region 和 composite blocks。也就是说，GUI 不是课外附带工具，而是 Neuroblox 的建模入口。

如果把这讲压成三句话，就是：

- flat brain 让你把脑区、block 和连接放到同一个平面上编辑。
- HarmonicOsc 让你验证自己真的会建边、调参、跑仿真。
- Cortical block 让你看到单个 block 如何被组合成更大的回路系统。

这也是为什么讲者最后反复鼓励大家“多玩玩 GUI”。对后面的教程来说，能不能顺利进入 WinnerTakeAll、Cortical、ImageStimulus 等示例，取决于你是否已经把这一讲里的交互语义掌握住了。

课程位置

官方课程总览里,Blox and Connections in Neuroblox 之后紧接着就是 Neurons, Neural Masses and Sources、Circuit Models、Decision Making 等单元。本讲的作用，正是把你从“知道有这些单元”推进到“知道怎么真的在 GUI 里搭起来”。

4.1 本章小结

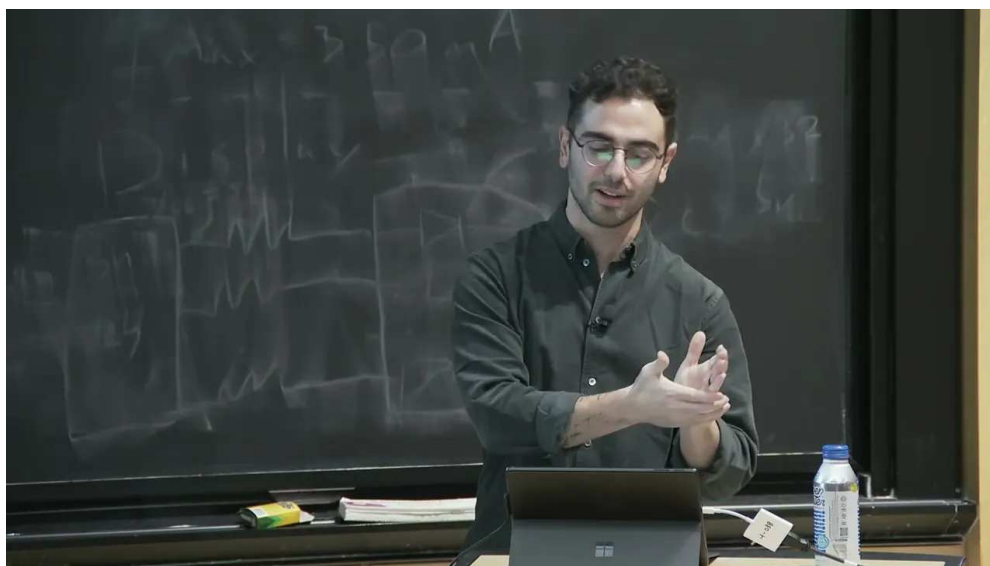
第 2 讲是后续所有教程的操作底座。它把界面、参数、仿真和复合块串成了一条完整的手工流程；只要这条链路打通，后面的更大回路和更复杂实验就只是规模上的扩展，而不是概念上的断层。

课程笔记

课程笔记：课程结构与 Julia 入门

Codex

2026-04-15



视频作者/频道：MIT Video Productions

发布日期：2025-01-13

视频时长：27:26

视频链接：<https://www.youtube.com/watch?v=ekwu47RHCHE>

目录

1	课程开场与安装准备	2
1.1	本章小结	2
2	本周安排与学习方式	2
2.1	本章小结	3
3	Julia 的基本定位	3
3.1	本章小结	3
4	交互式开发: REPL、VS Code 与 Jup ter	4
4.1	本章小结	4
5	表达力: 单位、循环、宏与 Neuroblox	4
5.1	本章小结	5
6	类型系统、多重派发与自定义结构	5
6.1	本章小结	5
7	性能、社区与学习资源	5
7.1	本章小结	6
8	Notebook 实操与课程收尾	6
8.1	本章小结	6
9	总结与延伸	6
9.1	本章小结	7

1 课程开场与安装准备

这一讲一开始并不是直接讲 Julia 语法，而是先把课堂运行环境理顺。讲者先把大家带到课程主页，提醒同学们先确认已经完成 Julia、Neuroblox 以及后续教程所需依赖的安装。这里的关键不是“装过就行”，而是要把课程的项目环境和依赖锁定到当前版本，避免后面练习时因为旧包版本而出问题。

```
1 using Pkg
2 using Downloads
3
4 Downloads.download(
5     "https://raw.githubusercontent.com/Neuroblox/course/refs/heads/main/Project.
6         toml",
7     joinpath(@__DIR__, "Project.toml")
8 )
9
10 pkg"registry add General"
11 Pkg.instantiate()
```

上面这段安装流程的逻辑很简单：先把课程提供的 `Project.toml` 下载到本地课题目录，再添加 Julia 的通用 registry，最后用 `Pkg.instantiate()` 生成与课程匹配的 `Manifest.toml` 和依赖环境。讲者特别强调，如果你在今天之前已经跑过这套步骤，最好把旧的 `Project.toml` 和 `Manifest.toml` 删掉后重跑一次，因为 Neuroblox 和其依赖最近有过更新。

安装时最容易出错的地方

课程要求你每次打开 notebook 时，都在包含 `Project.toml` 与 `Manifest.toml` 的目录下启动 Julia/Jupyter。这样做的目的不是繁琐，而是让每个人都在同一套依赖下运行教程。

1.1 本章小结

这一部分的重点不是 Julia 本身，而是先把“课程计算环境”这件事做对：统一项目目录、重新生成依赖、把 notebook 和项目文件放在同一套可复现的环境里。

2 本周安排与学习方式

讲者随后把整周安排快速过了一遍，并明确了这门课的节奏：每个 session 先有 10 到 15 分钟的概念性介绍，其余时间主要留给 notebook、文本块和代码块。换句话说，这不是一门以长讲座为中心的课，而是一门以动手练习为中心的课。

时间	内容
今天	课程结构 + Julia 入门
明天上午	Julia 中的微分方程与绘图生态（讲者口头写作 <i>Mikey</i> ，应为 Julia 绘图生态）
明天下午	Neuroblox 核心：blocks、connections、如何搭建自己的模块
周三	神经元、神经质量、外部来源，以及 circuit models
周四	decision making 与 reinforcement learning / synaptic plasticity
周五上午	spectral DCM 与模型拟合
周五下午	开放式工作时间：练习、挑战题、提问、继续搭建自己的 circuits

这一整套安排里有两个信息很重要。第一，课程的后续主题都会尽量变成“可运行的 notebook 任务”，而不是只停留在讲解层面。第二，每天的 notebook 末尾都会带有 challenge problems，它们不是额外负担，而是“学完这一部分后还能继续往前走”的建议方向。

课程的工作流

先看网页上的内容，再到 notebook 里跑同样的材料；完成基础练习后，再做 challenge problems。周五的最后一场则故意保持开放，方便大家回头补练习、问问题，或者开始建模自己的电路。

2.1 本章小结

这门课的设计目标很明确：前面用极短的讲解建立框架，后面把时间留给动手。课程安排、练习和 challenge problems 共同组成了同一条学习路径，而不是彼此割裂的任务清单。

3 Julia 的基本定位

讲者接着把话题切到 Julia 本身。Julia 从 2009 年在 MIT 开始开发，直到 2018 年 8 月才发布 1.0。这个时间线的意义在于，它说明 Julia 是一门较年轻但成长很快的语言；而今天的定位，则是“动态、交互式、表达力强，同时又能接近 C 的性能”。

讲者给出的 Julia 画像

Julia 不是“只能写高层表达”的脚本语言，也不是“只能跑得快但很难写”的底层语言。它试图同时兼顾可读性、交互性和性能，这也是“*like Python, runs like C*”这句口号的真实含义。

讲者还特别先提醒了一个会立刻影响上手体验的规则：Julia 是 **one-based indexing**，也就是任何集合的第一个元素索引都是 1，而不是 Python 里的 0。对于从 Python 迁移过来的同学，这一点需要刻意适应。

3.1 本章小结

Julia 的核心卖点可以概括为三层：语法高层、交互友好、性能可观。课程后面所有关于 Neuroblox 的设计，基本都建立在这三个性质之上。

4 交互式开发：REPL、VS Code 与 Jupyter

讲者进一步解释“dynamic and interactive”到底是什么意思。Julia 虽然是编译型语言，但在日常使用中可以像 Python 或 MATLAB 那样按行执行、即时看结果。课程把这种体验放在三个入口里讲：REPL、VS Code 和 Jupyter Notebook。

```
julia --project="." -e "using IJulia; notebook(dir = pwd());"
```

REPL 是最直接的方式，适合快速试验、验证一个表达式是否正确；VS Code 则更适合更长一点的代码开发，因为它能提供补全、文档和绘图面板。讲者专门展示了 Julia 扩展如何在你输入函数名时给出补全，并在旁边显示 `docstring`。如果你在 session 里画图，VS Code 还会打开图窗并保留历史图像，便于回看。

而本课最主要的工作界面是 Jupyter Notebook。讲者强调，它与网页课程内容是同一份材料，只是呈现形式略有不同：网页上是按页面排版的文本与代码，而 notebook 则是文本块和代码块交错的可执行版本。也正因为如此，课程建议大家以 notebook 为主，而不是只看网页。

关于 notebook 的使用建议

讲者建议：如果你第一次打开 Julia intro notebook，先完成前面的基础内容；性能那一节内容很长，短时间内可以先略过，后面有空再回来补。

4.1 本章小结

Julia 的“交互式”不是说它不是编译型语言，而是说它允许你在编译性能和即时反馈之间取得平衡。REPL 适合试错，VS Code 适合开发，Jupyter 适合课程式学习和循序渐进地跑代码。

5 表达力：单位、循环、宏与 Neuroblox

讲者接下来证明 Julia 不只是“能交互”，还“很会表达”。最直观的例子是 `Unitful` 这类包：你可以像写论文一样，把物理单位直接嵌进代码里，例如电阻写成 `100 megaohms` 之类的形式。这样做的好处是，代码更接近人类描述，减少“在脑子里换算单位”的负担。

另一点非常重要：Julia 并不强迫你把循环改写成向量化表达式。讲者明确说，在 Julia 里，循环和向量化操作的性能可以非常接近，因此你可以优先考虑可读性，而不是被某种“必须向量化”的习惯绑定。

更进一步，Julia 允许你通过 `macros` 搭建自己的领域专用语法。讲者在这里举了 `ModelingToolkit` 的例子：用 `@variables`、`@parameters`、`@equations` 这类宏写出符号化的微分方程系统。这样，代码看上去更像数学推导，而不是底层实现。

同样的思路也出现在 `JuMP` 里。你先定义模型、选择优化器，然后再逐步添加变量、目标函数和约束，最后打印或者求解模型。讲者用它说明：Julia 不是只适合“写语法”，它也适合把复杂优化问题写成清晰的高层建模语句。

在 `Neuroblox` 中，这种表达力更直接地服务于建模。讲者展示了一个非常短的例子：定义两个 Hodgkin-Huxley 神经元，创建一个空图，然后把抑制性神经元连到兴奋性神经元上。也就是说，电路模型在代码层面就是图结构上的节点和边，基本概念一一对应。

为什么这对 Neuroblox 特别重要

Neuroblox 的目标不是把你锁进一套复杂 API，而是让“神经元、连接、图、规则”这些建模对象能在 Julia 里自然组合。Julia 的宏、类型和高层语法正好给了它这种组合能力。

5.1 本章小结

这一部分的核心不是某个包，而是 Julia 的建模风格：代码可以既像数学，也像配置，还能保持可执行和可组合。对 Neuroblox 来说，这意味着电路建模可以写得短、清楚，而且容易扩展。

6 类型系统、多重派发与自定义结构

讲者把 Julia 的第二个核心概念讲得很清楚：**multiple dispatch**。简化地说，Julia 允许同一个函数名对应多个方法，而究竟调用哪个方法，不是看函数名字，而是看参数的组合和类型。这个机制是 Julia 设计里的底层支柱之一。

```
1 f(x::Float64, y::Float64) = ...
2 f(x::Number, y::Number) = ...
```

讲者用一个非常短的例子说明派发规则：如果你传入的是 2. 和 3，会匹配到更一般的 `Number` 方法；如果你传入的是两个浮点数，则会命中更具体的 `Float64` 方法。这里的重点在于，`Float64 <: Number` 这类 subtype 关系会直接影响方法选择。

Julia 的类型系统还允许你定义自己的结构体。讲者用一个 `MyType` 的例子说明：结构体里的某些字段可以指定具体类型，某些字段也可以保持泛型，这样既能约束数据，又能保留灵活性。这个机制在后续创建自定义 blocks 时会非常有用，因为 Neuroblox 的许多组件本质上就是类型明确、但仍可扩展的数据结构。

类型和多重派发组合在一起，会产生一种很 Julia 的现象：两个原本没打算互相配合的类型和函数，可能只需要一条方法定义，甚至不需要额外代码，就突然能一起工作。讲者把这种现象称为一种“emergent”式的语言能力。

这和面向对象的差别

讲者把 Julia 的设计与 Python 式面向对象做了对比：Julia 更强调“按参数类型组合方法”，而不是把所有行为都塞进对象层级里。对数值计算和科学建模来说，这通常更自然。

6.1 本章小结

类型系统和 multiple dispatch 不是 Julia 的附加功能，而是它的编程核心。理解这两点，后面看 Neuroblox 的自定义 block、连接规则和模型组织方式时，就会顺很多。

7 性能、社区与学习资源

讲者最后补了两块很实用的内容：一是性能，二是学习资源。性能部分的核心信息是，Julia 在很多基准上可以非常接近 C 的行为，而不像 MATLAB、R 或 Octave 那样离底层实现更远。讲者用线性代数和递归类基准说明：Julia 既能处理数值密集计算，也能在高层抽象下维持相当不错的速度。

除了语言本身，Julia 社区也被强调为一个优势。讲者提到 [Discourse.julialang.org](https://discourse.julialang.org)、Julia Slack，以及开发者调查里大家最常使用的学习资源。这里最值得记住的是：官方 manual 在 Julia 生态里并不是摆设，而是最常被拿来学习和查找细节的材料之一。

资源类型	课程建议
官方 manual	优先阅读；讲者明确建议把它当作首要参考
JuliaCon / YouTube / 书籍	适合补充背景、看范例、查不同作者的讲法
课程 Introduction to Julia 页面	已经整理了若干相关链接，适合继续扩展
社区论坛与 Slack	遇到具体问题，先搜索再提问，通常能很快得到反馈

7.1 本章小结

性能说明 Julia 不是“写起来快但跑不动”，社区和资源则说明它不是“只有语言没有生态”。对于真正要在科研里持续使用的工具，这两点都很关键。

8 Notebook 实操与课程收尾

在概念讲完之后，讲者把大家拉回到实际操作：进入 Introduction to Julia 页面，打开第一个 notebook `julia_intro`。如果你是第一次用 Jupyter，就按正常流程进入 notebooks 文件夹，然后打开对应页面即可。后续每个 notebook 的进入方式都类似。

讲者还给了一个很实用的时间管理建议：`julia_intro` 这份 notebook 很长，但并不要求一次性全部做完。前面的核心部分，尤其是 `functions` 之后的内容，值得认真跟；但性能那一段可以先跳过，等你完成前面的内容再回来补。这样可以保证第一次上手时不被过长的 notebook 拖住。

最后，讲者再次强调：课程网页和 notebook 的内容是一致的，所以你可以在网页上预览，再到 notebook 里执行同一套材料。遇到问题时，既可以在 Slack 提问，也可以直接举手。

8.1 本章小结

这门课的实际执行方式非常明确：先用网页建立概念，再在 notebook 里跑同样的材料；先抓住主线内容，再回头处理性能和细节。这样做的目的，是让你尽快进入“边看边跑边改”的状态。

9 总结与延伸

这一讲的真正任务，不是完整教会 Julia，而是让你建立一个足够稳的起点：知道课程怎样运行，知道 notebook 怎么打开，知道 Julia 为什么适合 Neuroblox，也知道哪些概念会在后续反复出现。

如果把整讲压缩成三句话，就是：

- 先把环境和项目文件弄对，保证后续练习可复现；
- 再记住 Julia 的三个关键词：交互、表达力、性能；
- 最后把 multiple dispatch、类型和宏当成后续理解 Neuroblox 的基础设施。

对后续内容来说，今天的目标已经足够清楚：你不需要立刻掌握 Julia 的全部细节，但你必须能在 notebook 里开始跑，知道自己在看什么，也知道自己下一步该去哪里找答案。

9.1 本章小结

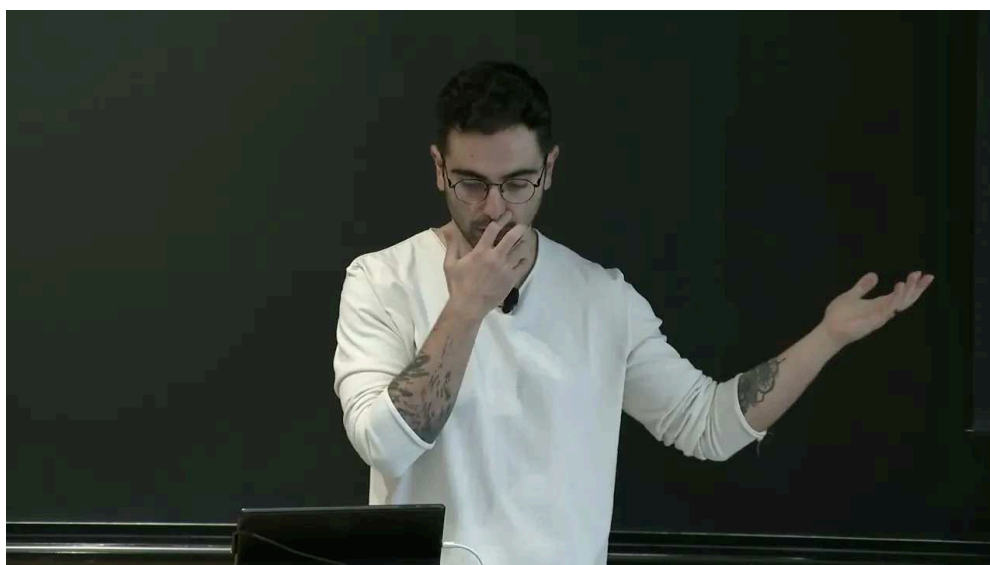
课程结构、Julia 语言特性和 notebook 工作流在这一讲里被连接起来了。后面的每一讲，都会把今天打下的这些基础继续展开到具体模型和具体电路上。

课程笔记

MIT RES.9-009 第 04 讲：Julia 中的微分方程与绘图

MIT RES.9-009 课程整理 & Codex

2026 年 4 月 15 日



视频作者/频道：MIT Video Productions

发布日期：2025-04-10（讲课日期：2025-01-14）

视频时长：12:57

视频链接：<https://www.youtube.com/watch?v=6EaolLVhnug>

目录

1	课程定位与 workflow	2
1.1	本章小结	2
2	ModelingToolkit: 从符号系统到 Lotka-Volterra	2
2.1	本章小结	4
3	I hkevich 手工尖峰、参数改写与外部电流	4
3.1	本章小结	5
4	动态电流与结构化化简	5
4.1	本章小结	7
5	Julia 生态协作与不确定性传播	7
5.1	本章小结	7
6	Makie: 后端、布局与组合绘图	7
6.1	本章小结	9
7	Spike plotting: 把仿真结果变成统计图	9
7.1	本章小结	10
8	总结与延伸	1
8.1	本章小结	11

证据范围

本讲没有找到官方 slide PDF；正文依赖两条主证据链：课程页 HTML/结构化页面，和嵌入视频的字幕与元数据。为了避免把缺失资源当成教学内容，相关缺口已记录到 `omission_log.jsonl`，正文只展开可核验的课程材料。

1 课程定位与 workflow

这一讲把两件事放在一起讲：**微分方程的符号建模和结果的布局式绘图**。前半部分来自 `intro_diffeq.ipynb`，后半部分来自 `intro_plot.ipynb`；它们分别对应 ModelingToolkit 与 Makie，正好构成 Neuroblox 里最基础也最常用的两条工作线。

讲者的意思很直接：如果你要在 Julia 里做计算神经科学，先要会用符号方式写系统，再要会把系统的结果组织成可读的图。前者解决“模型怎么写”，后者解决“模型怎么讲给别人看”。这两步连起来，才是一个完整的 research workflow。

官方课程页没有把所有内容做成单独 slide PDF，但页面本身已经给出了 notebook 入口、文字说明、代码块和图片。因此本讲的结构不按“屏幕截图顺序”展开，而按“建模 -> 求解 -> 改参数 -> 动态输入 -> 绘图 -> 统计”这条主线整理。

这讲的核心接口

ModelingToolkit 负责把 ODE/DAE 写成符号系统，`OrdinaryDiffEq` 负责求解，Makie 负责把轨迹、布局和 spike 统计画出来。只要掌握这三层接口，就已经能完成本讲的 notebook。

1.1 本章小结

本讲不是在讲两个无关主题，而是在讲同一个工作流的上下两端：先用符号建模和数值求解得到时间序列，再用 Makie 把时间序列转成结构清楚的图。

2 ModelingToolkit：从符号系统到 Lotka-Volterra

ModelingToolkit 的关键价值，在于它允许你用符号方式定义方程系统，而不是先手写一堆索引再把它们塞进黑箱求解器。课程页面先拿 Lotka-Volterra 作为入门例子，目的不是生态学本身，而是让你先建立“两个变量互相影响、系统随时间演化”的直觉。

$$\dot{x} = \alpha x - \beta xy, \quad \dot{y} = \delta xy - \gamma y \quad (1)$$

这类写法对应的不是静态关系，而是状态随时间的更新规则。真正重要的是，变量名、参数名和微分算子都保留在符号层；这样一来，后续求解、替换参数、提取变量和绘图都能直接围绕这些符号展开。

```

1 using ModelingToolkit
2 using ModelingToolkit: t_nounits as t, D_nounits as D
3 using OrdinaryDiffEq
4
5 @variables x(t) y(t)
6 @parameters a b g d

```



```

7 eqs = [D(x) ~ a*x - b*x*y,
8       D(y) ~ d*x*y - g*y]
9
10 sys = ODESystem(eqs, t, [x, y], [a, b, g, d])
11 prob = ODEProblem(structural_simplify(sys), [x => 5.0, y => 2.0], (0.0, 10.0),
12                  [a => 1.5, b => 1.0, g => 3.0, d => 1.0])
13 sol = solve(prob)
14 plot(sol)

```

Listing 1: ModelingToolkit 的最小工作流

讲者还特别点出，求解后你可以像访问字典一样访问具体变量，例如 `sol[x]`。这很重要，因为它说明求解器返回的不是一团匿名数组，而是保留符号语义的 `solution object`。

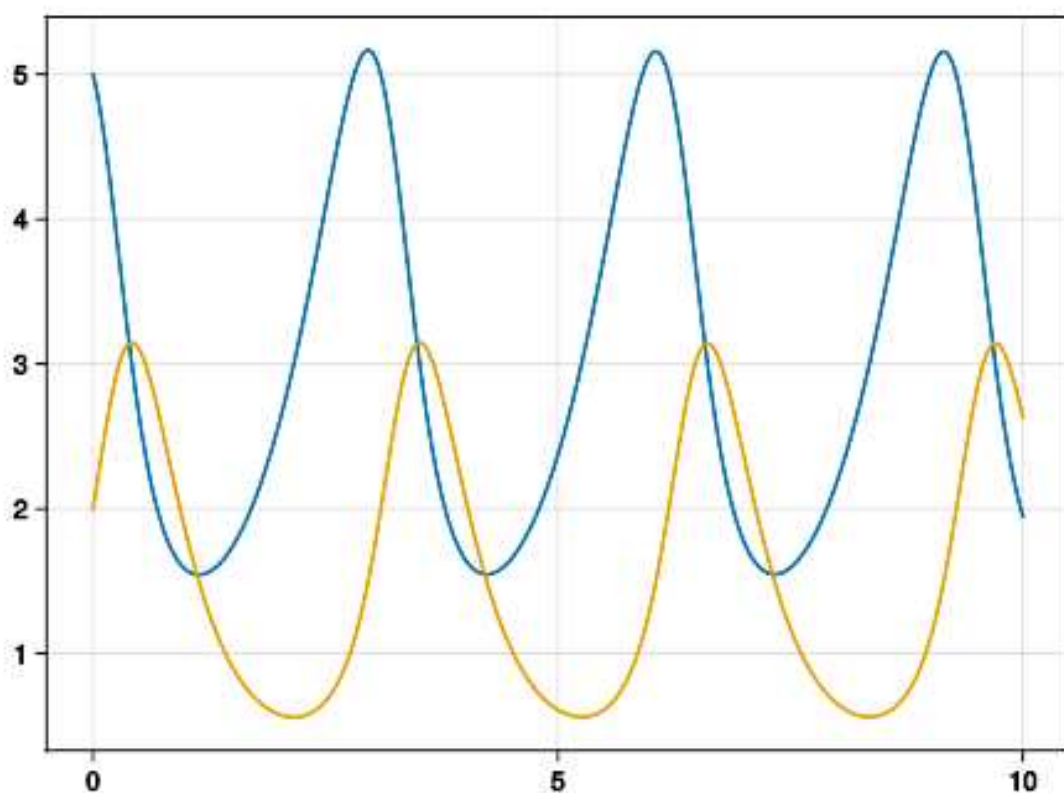


图 1: Lotka-Volterra 的时间轨迹示意。课程用这个例子先建立“符号方程 -> 求解 -> 变量访问 -> 作图”的基本链条。

讲者还借这个阶段强调了 ModelingToolkit 的模块化风格。一个系统可以先作为独立模块定义出来，再通过 `port` 接口连到另一个模块，最后再组装成更大的系统。视频里用 `mass` 和 `damper` 的类比说明：单个部件可以先写好变量和方程，再把接口暴露出来，最后把两个模型通过连接端口拼成一个新的系统。

模块化建模的意义

这一层不是为了“多写几层抽象”，而是为了让一个复杂系统可以从单元模型逐步组合出来。课程特别强调 `model macro`、嵌套模型和端口连接，就是在说明 Neuroblox 也沿着同样的设计哲学工作。

课程随后把视角扩展到 SCIMLE 生态: ModelingToolkit 只是其中一部分, PDE、UQ、Universal Differential Equations 等工具都在同一生态里协同工作。这里的关键结论是, Julia 里不同包虽然可以由不同团队独立维护, 但仍然能在同一个建模流程里互操作。

2.1 本章小结

ModelingToolkit 的重点不是“能不能写 ODE”，而是“能不能用符号保留系统结构”。一旦结构保住了，求解、替换参数和后续分析就都顺手了。

3 Izhikevich: 手工尖峰、参数改写与外部电流

课程接着把同样的建模方式推进到 Izhikevich neuron。这里的重点不再是两变量动力学本身，而是尖峰事件要如何被“手工”表达出来：当电压越过阈值时，系统要执行 `reset`，把电压拉回更极化的值，同时可能还要更新其他变量。

$$D(v) \sim 0.04v^2 + 5v + 140 - u + I, \quad D(u) \sim a(bv - u)$$

尖峰事件可以写成阈值触发加重置规则：

$$v > 30.0 \Rightarrow [u \sim u + d, v \sim c]$$

这段内容的教学重点有两个。第一，尖峰并不是连续 ODE 自动“长出来”的，需要用离散事件显式描述。第二，不同参数会让同一个神经元进入不同 spiking regime，因此参数本身就是模型的一部分。

```

1 @variables v(t)=-65 u(t)=-13
2 @parameters a=0.02 b=0.2 c=-50 d=2 I=10
3 event = (v > 30.0) => [u ~ u + d, v ~ c]
4
5 izh_system = ODESystem(eqs, t, [v, u], [a, b, c, d, I]; discrete_events = event)
6 izh_simple = structural_simplify(izh_system)
7
8 izh_prob2 = remake(izh_prob; p = [a => 0.1, b => 0.2, c => -65, d => 2])

```

Listing 2: 阈值事件与参数回填的思路

讲者随后演示了 `remake` 的意义：如果 `ODEProblem` 已经定义好，只改参数或初始条件，就能很方便地重跑模拟。视频里强调的是从 `chattering` 切换到 `fast spiking`，这说明参数组合的变化会直接改变放电模式。

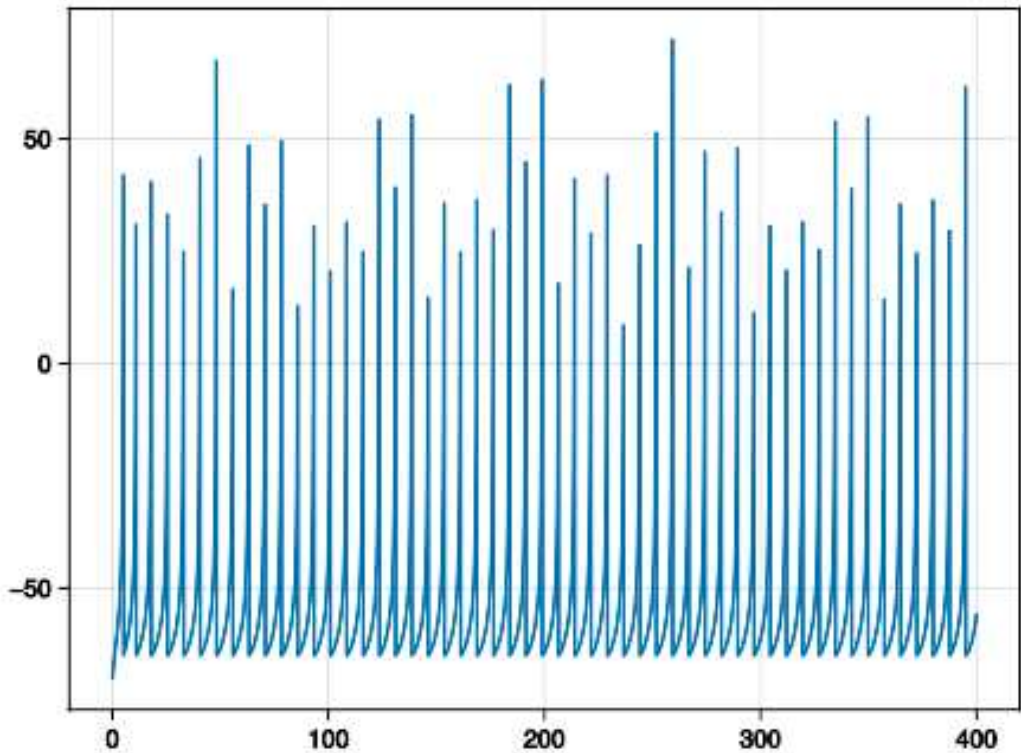


图 2: 改参数后的 Izhikevich 放电轨迹。这里能直观看到参数变化如何把尖峰模式从一种 regime 推到另一种。

3.1 本章小结

尖峰模型的核心不只是 ODE 系统本身，而是“连续动力学 + 离散事件 + 参数重写”三者一起工作。ModelingToolkit 的优势就在于把这三层都放在同一套符号语义里。

4 动态电流与结构化化简

下一步，课程把外部电流从常量参数改成了时间相关变量 $I(t)$ 。这一步很小，但非常关键，因为它把“输入”从静态常数变成了系统状态的一部分，也就更接近真实脑内输入的变化方式。

更重要的是，`structural_simplify` 会把原来含有代数方程的系统化成更容易积分的形式。视频里特意强调：原始系统里电流方程还在，但在化简后的系统里，它会被代入主方程，从而把一个 DAE 变成更容易求解的 ODE。

$$\text{DAE} \xrightarrow{\text{structural_simplify}} \text{ODE}$$

这不是纯粹的符号炫技，而是数值效率问题。ODE 的积分通常比 DAE 更直接、更稳定，也更适合后续的参数扫描与可视化。

讲者顺手也补了一个很实用的判断标准：不同系统适合不同求解器。视频里用一个相对简单的两维系统和一个更高维、更 stiff 的系统作对比，说明求解器性能、刚性和容差设置会直接改变运行表现。课程页给出的 solver guide 就是为了把这种“在什么场景用什么 solver”的判断具体化。

```
1 @variables v(t)=-65 u(t)=-13 I(t)
2 @parameters a=0.02 b=0.2 c=-65 d=8
```

```

3 eqs = [D(v) ~ 0.04*v^2 + 5*v + 140 - u + I,
4       D(u) ~ a*(b*v - u),
5       I ~ 10sin(0.5t)]
6
7 izh_system = ODESystem(eqs, t, [v, u, I], [a, b, c, d])
8 izh_simple = structural_simplify(izh_system)
9 tspan = (0.0, 400.0)
10 izh_prob = ODEProblem(izh_simple, [], tspan)
11 izh_sol = solve(izh_prob)
12 plot(izh_sol; idxs=[v, I])

```

Listing 3: 动态电流进入状态方程

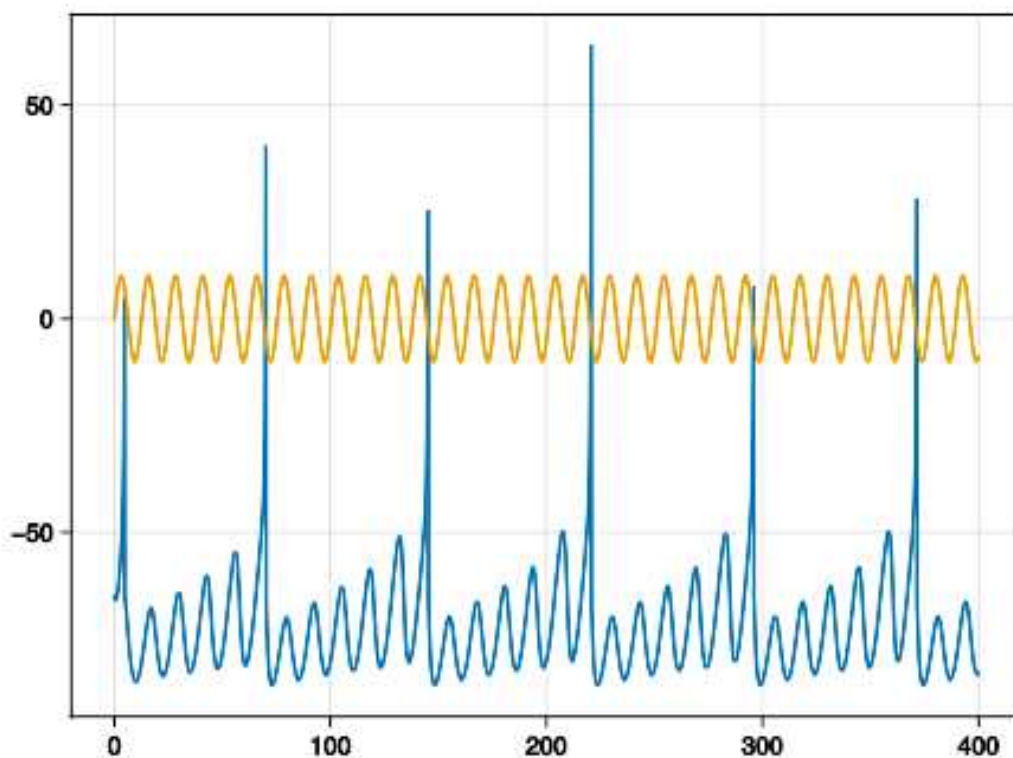


图 3: 动态外部电流进入系统后, 电压与输入一起被画出来。这里对应的是“把 I 从参数改成变量”这一步。

讲者还提到, 参数值和初始条件上的 $\pm$ 变化其实可以来自测量不确定性; 把这些不确定性显式带入模型, 得到的就不再是一条线, 而是传播后的边界或带状区域。这一点在 Julia 生态里依赖的是 `Measurements` 一类工具, 后面会接着讲。

官方课程页在这一段后面给出了一个明确的练习: 把 `generalized integrate-and-fire neuron` 写成 `ODESystem` 并模拟, 然后选一个你熟悉的神经元模型做 `sensitivity analysis`, 最后把参数变化对 `spike pattern` 的影响用一张或几张图总结出来。这个练习和前面的主线完全一致, 因为它要求你把“方程定义、参数改写、数值求解和可视化”连成闭环。

4.1 本章小结

把外部电流从常量变成变量之后，模型就从“固定输入的神经元”变成了“带动态驱动的神经元”。`structural_simplify` 的作用则是让这个系统在数值上更可解。

5 Julia 生态协作与不确定性传播

课程中间有一段很值得单独拎出来：讲者不是只在讲 `ModelingToolkit` 本身，而是在讲 Julia 生态为什么能让不同包协作。`ModelingToolkit` 和 `Measurements` 并不是一开始就为彼此设计的，但 Julia 的组合式设计让它们可以无缝协同。

这段话的含义很具体。参数值和初始条件里的 $\pm$ 量，不只是“误差条”的装饰，而是测量不确定性的编码；一旦这些量进入方程，求解器就会把它们一并传播到最终轨迹里。于是你得到的不是单条曲线，而是带边界的解。

不确定性不是附加项

在这个工作流里，不确定性不是解算完再贴上去的阴影，而是直接作为输入的一部分进入系统。这样得到的 solution 才能反映“测量误差如何穿过动力学被放大或压缩”。

讲者顺手也把这一点和 Julia 的包生态联系起来：不同包可以分别由不同团队开发，但由于语言层面的可组合性，它们仍能协作。这也是 `Neuroblox` 依赖 Julia 的一个实际原因。

课程前面提到的性能对比也在这里得到回扣：讲者明确说，`SCIMLE` 的 solver 工具链文档非常完整，能覆盖不同 stiffness 和 tolerance 需求；同一颜色编码在不同语言示例里保持一致，说明这个生态的目标不是“换一种语法而已”，而是让你把已有模型平滑迁移到 Julia 并复核结果。

课程页的参考文献也很直接：一条是 Izhikevich 2003 的经典尖峰模型论文，另一条是 `ModelingToolkit` 的 composable graph transformation 论文和官方文档。这说明本讲并不是临时拼出来的教程，而是直接对齐了 SciML 的方法论文和文档链条。

这段的核心结论

`ModelingToolkit` 管系统，`Measurements` 管不确定性，而 Julia 让两者能在同一条方程链上工作。结果是你不仅能模拟，还能把测量误差一起送进模型。

5.1 本章小结

这部分实际上讲的是 Julia 生态为什么适合科学建模：不是单个包“很强”，而是不同包之间能自然拼接，因此模型、输入和不确定性可以一起进入同一个求解流程。

6 Makie: 后端、布局与组合绘图

进入绘图部分后，课程先把 Makie 讲成一个生态，而不是单个函数库。Makie 负责统一绘图接口，真正的渲染则由后端完成。课程页点名了四个后端，讲者则明确说在这门课里会用 `CairoMakie`，因为它适合 2D 静态图，而且不需要 GPU。

表 1: Makie 讲到的几个后端

后端	典型场景	课程里的定位
CairoMakie	2D 静态图、出版质量输出	本讲默认选择
GLMakie	GPU 加速的交互式 2D/3D 图	适合重交互或重计算场景
WGLMakie	Web 交互式图	适合浏览器展示
RPRMakie	光线追踪 / 更拟真渲染	仍偏实验性

Makie 的一个优点是布局很自然。讲者反复强调, Figure 可以当作一个二维网格来用, `fig[1,1]`、`fig[2,1]` 之类的位置能直接嵌入各自的 Axis。这样就能在一个窗口里同时放多张图, 而不是分散成很多窗口。

```

1 using CairoMakie
2
3 fig = Figure(size = (700, 500))
4 ax1 = Axis(fig[1, 1], title = "Line_plot")
5 ax2 = Axis(fig[2, 1], title = "Scatter_plot")
6 lines!(ax1, seconds, measurements)
7 scatter!(ax2, seconds, measurements)
8 hidexdecorations!(ax1, grid = false)
9 fig

```

Listing 4: 一个最常见的 Makie 布局骨架

课程还特别提到, Makie 的布局和普通 `lines(seconds, measurements)` 这种简写相比, 多了对坐标轴和排版控制。若要做正式汇报或论文图, 这种控制就非常值钱。

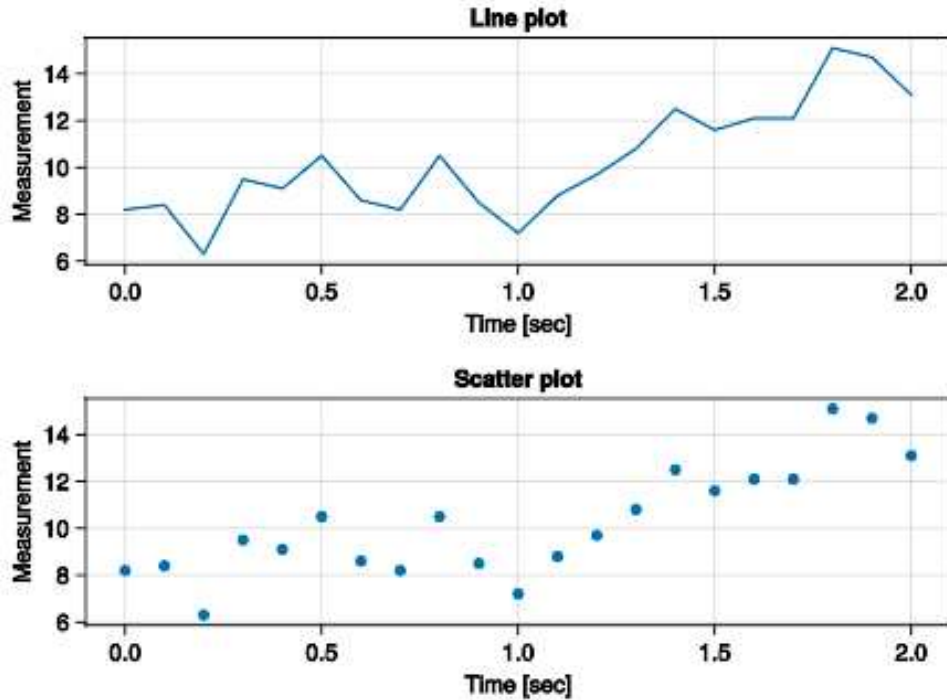


图 4: Makie 的两面板布局示意。课程用它说明 Figure 和 Axis 可以像网格一样组合。

讲者还补充说，这种布局能力正是新用户容易上手的地方。一个 panel 里还可以再嵌套多个 subplot，形成更复杂的分块结构；课程里提到的 panel A、legend、brain image 和多个子图，都是同一套布局系统的自然结果。若你来自 R 的 ggplot2，可以把 AlgebraOfGraphics 看成类似 grammar of graphics 的入口；若你来自 Python，base Makie 的调用风格会更像 matplotlib，只是排版和组合更统一。

Makie 的 extended ecosystem 也不是摆设。课程明确提到 neuroscience 里的 power spectrum、scalp map、source localization、event-related potentials 和 pair plots，甚至 multibody 与 robotic arms 这类例子。换言之，Makie 不是只能画一般曲线，它更像一套可以跨领域复用的视觉语法。

6.1 本章小结

Makie 不是单纯的“画图包”，而是一个统一接口加多后端的绘图生态。对 Neuroblox 来说，这意味着仿真结果、布局和发表级输出可以在同一套语义下完成。

7 Spike plotting: 把仿真结果变成统计图

最后一段把前面的两条主线合到一起：先仿真，再画更有信息量的图。课程选的是 fast-spiking Izhikevich 神经元，并在特定时刻改变外部电流，然后按窗口统计 spike 数。

这个例子很适合说明 Makie 的实际价值。你不只是把电压轨迹画成一条线，而是把 spike 计数、刺激时刻和时间窗都组织成一张图。这样一来，观众不仅看到“有尖峰”，还能看到“刺激前后尖峰统计如何变化”。

```

1 spikes = zeros{Int, length(t_range) - 1}
2 for i in eachindex(t_range[1:end-1])
3     idxs = findall(t -> t_range[i] <= t <= t_range[i + 1], timepoints)

```



```

4     spikes[i] = count(V_izh[idxs] .> spike_threshold)
5 end

```

Listing 5: 按时间窗统计 spike 的核心思路

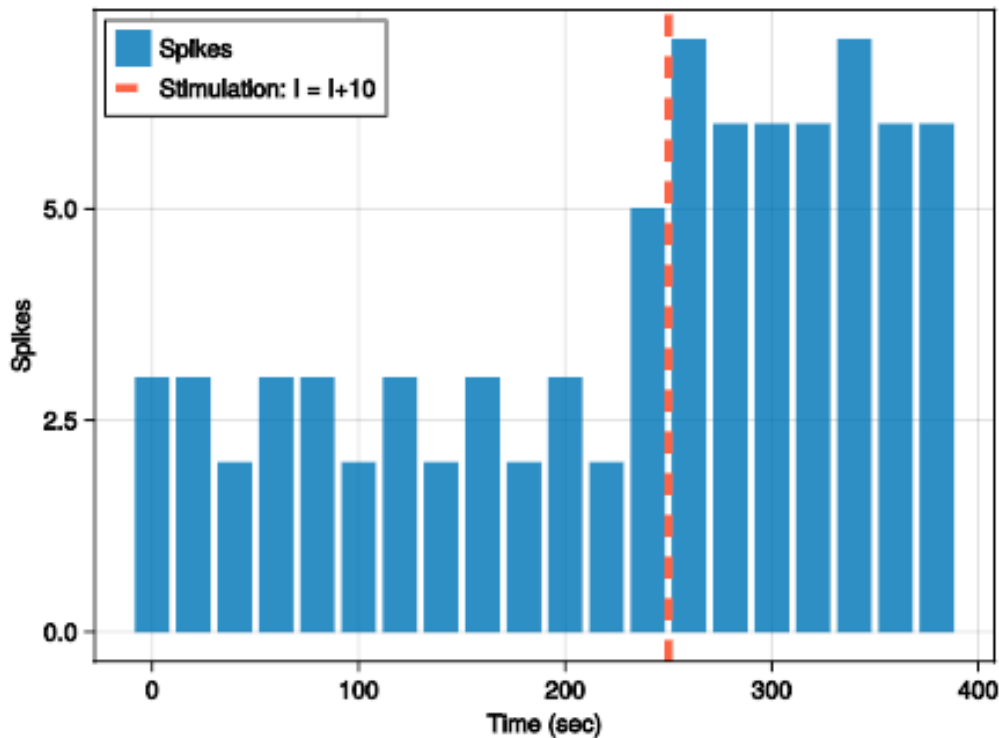


图 5: spike 统计图：红色虚线标出刺激开始时刻，柱状图则显示不同时间窗内的 spike 数。

讲者还顺带提醒，Makie 的这个例子可以直接从课程网站上的 notebook 复现：同样的材料同时存在于网页和 notebook 里，所以最稳妥的做法就是先照着 `intro_diffeq.ipynb` 做，再切到 `intro_plot.ipynb`。

网页上的 plotting challenge 也把你重新指回前一部分：如果你要继续练习，就先回到 Differential Equations in Julia 的 notebook，把前面定义的神元系统补完整，再回来把结果换一种图法呈现。

7.1 本章小结

这一段把“模拟”和“展示”真正接上了：不是只看单条轨迹，而是把事件计数、刺激窗口和时间轴放进同一张图里，让结果的结构一眼可见。

8 总结与延伸

这讲的主线很统一：用 ModelingToolkit 写动力系统，用 OrdinaryDiffEq 求解，用 Makie 展示结果。Lotka-Volterra 用来建立符号 ODE 的直觉，Izhikevich 用来说明尖峰事件和参数重写，动态电流和 `structural_simplify` 用来展示 DAE 到 ODE 的化简，Makie 则把布局和 spike 统计变成清晰的视觉结果。

课后顺序

先做 `intro_diff_eq.ipynb`，再做 `intro_plot.ipynb`。课程页已经把同样的材料放在网页和 notebook 两边，最好的学习方式就是按这个顺序把整套流程跑一遍。

官方课程页给了两类参考：一类是 Izhikevich neuron 的经典论文，另一类是 ModelingToolkit 与 Makie 的文档。就这讲而言，最重要的不是死记某个函数名，而是把“符号建模 -> 求解 -> 参数修改 -> 动态输入 -> 布局绘图”这个完整链条记住。

讲者在结尾也明确说了，Notebook 和 website 的内容是对齐的，所以如果你在课堂上没跟上，最稳妥的补课顺序就是先把网页和 notebook 各跑一遍，再回头看这讲的几个关键图。

8.1 本章小结

如果只记一句话，这讲要传达的是：Neuroblox 的基础能力不是“会不会画图”，而是“能不能把动力学模型写对、解对、改对、画对”。这四件事在 Julia 里正好可以连成一条流水线。

课程笔记

MIT Neuroblox 教材笔记 06：神经元、神经团块与源

MIT Neuroblox 课程组 & Codex

2026 年 4 月 15 日



视频作者/频道：MIT Video Productions

发布日期：2025-04-10；课程标题标注 2025-01-15

视频时长：12 分 54 秒

视频链接：<https://www.youtube.com/watch?v=Ptqv16fh0tg>

目录

1 导论：学习目标与对象总览

本讲把 Neuroblox 的三类基础对象放在同一张地图上：神经元、神经团块 (*neural mass*) 和 源 (*source*)。视频开场先明确：在前面几讲已经讲过 Julia 和 Neuroblox 的编程语法之后，这一讲开始回到神经科学本体，先看最基本的神经元，再看群体平均后的神经团块，最后看如何用外部源驱动模型。

这节课真正关心的不是“画什么图”，而是“变量如何通信”

Neuroblox 中三类对象的核心差别，不是名字，而是它们如何把自己的状态送进下游方程：

- 神经元通常通过连续状态或事件触发来影响别的 Blox；
- 神经团块把群体行为压缩成更少的状态变量，强调平均活动、同步/去同步和参数区间；
- 源本身没有输入，只负责向下游注入连续驱动、脉冲列或刺激协议。

这也是本讲所有例子的共同骨架。

对象	通信方式	代表例子	适合回答的问题
Neuron	连续状态 / 事件触发	QIF, HH, Izhikevich	单细胞兴奋性、发放、
Neural mass	群体平均 / 振荡器 / 解析导出	Wilson-Cowan, Jensen, next generation	平均活动、同步状态、
Source	仅输出、无输入	ConstantInput, PoissonSpikeTrain, DBS	外部驱动、实验协议、

表 1: 本讲三类 Blox 的接口差异。

课程页在首页就把学习目标写得很直接：先可视化神经元与神经团块的仿真结果，再把外部源作为 Blox 引入，最后用这些源去驱动单神经元与群体活动。这个目标虽然朴素，但它实际上定义了后面所有 API 的使用边界：先建模，再连接，再看输出。

1.1 本章小结

本节只做了框架搭建：后面的所有细节，都可以看成是在回答同一个问题——“不同层级的神经模型，究竟通过什么接口把状态送到下游？”

2 神经元：单元动力学、发放率与随机性

神经元部分先从生物结构讲起：树突接收输入，胞体整合信号，轴突负责把动作电位传给别的神经元。课程视频特意强调了一个数量级事实：人脑里神经元数量极大，而且多数神经元并不是孤立工作的，树突平均会接收到很多其他神经元的输入。这个背景解释了为什么单神经元模型既要足够简洁，又要保留“输入-整合-输出”这条基本链路。

动作电位在课堂里被描述成一个从化学信号转成电信号的过程：突触前释放递质，递质进入突触间隙，被突触后受体接收，受体打开离子通道，离子流入引发突触后电位。也就是说，单神经元模型里最关键的不是“会不会放电”这个结论，而是受体、通道、离子流和膜电位之间怎样协同。

2.1 Hodgkin-Huxley 的意义：不仅是生物细节，也是等效电路

课程视频把 Hodgkin-Huxley 模型放在很核心的位置：它不仅刻画了受体动力学和膜电位变化，还提供了一个等效电路视角。这个视角很重要，因为它把“生物过程”翻译成了“受体、电容、电源”组成的电路，使得单神经元可以既像生物系统，又像一个可计算的动力学系统。

```

1 using Neuroblox
2 using OrdinaryDiffEq
3 using CairoMakie
4 using Random
5 Random.seed!(1)
6
7 @named qif = QIFNeuron(; I_in=4)
8 sys = system(qif; discrete_events = [60] => [qif.I_in ~ 10])
9 tspan = (0, 100) # ms
10 prob = ODEProblem(sys, [], tspan)
11 sol = solve(prob, Tsit5())
12
13 fig = rasterplot(qif, sol; threshold=-40)
14 fig = frplot(qif, sol; threshold=-40, win_size=20)
15 V = voltage_timeseries(qif, sol)
16 fig = lines(V)
17
18 using StochasticDiffEq
19 @named hh = HHNeuronExci_STN_Adam_Blox(; sigma = 2)
20 sys = system(hh)
21 prob = SDEProblem(sys, [], (0, 1000))
22 sol = solve(prob, RKMil())
23 fig = powerspectrumplot(hh, sol; sampling_rate=0.01)

```

Listing 1: QIF 与 HH 的标准仿真接口示意。

上面的代码块把本讲最重要的三种可观测量放在一起了：`rasterplot` 看离散放电，`frplot` 看滑动窗意义下的发放率，`powerspectrumplot` 则把随机扰动下的频谱结构显示出来。课程页还特地指出，随机 HH 神经元的模型中含有 brownian noise 项，它会改变频谱上的能量分布，所以只看时域波形是不够的。

U V Z Å 6' bi2'THQi

U#V Z Å 6'BM;@' iB THQi

m R, Z Å 6 ¥ æ~ Vj~j b[æ: æ ¥ B □ “ o^ ?bp—o?b
qM~p^ “] b

m k, >>* ¥ q b[æ: □ æ4 .2i " ^o pæ † b

kXk AMi2;'+iQ2bQM iQAwL ^ em

j /" * s BMi2;'+iQ2bQM b[æ: □ 6A +m'p2us7 b
BMi2;'+iQ2bQM b[æ: □ 6A +m'p2us7 b
{~S¶=+< ?b tV æuW“ ~ Ab~ b
B ¥1o ^s, 7^[æj < ¥ #B7m'+4iBjBm* YV¿M
V[Æ a«p?ba #m'bi?b>WM—b— ~“ 6AwL s·m ^,c 7

是神经元动力学的相图。

2.3 Izhikevich 模型的教学价值

视频用 Izhikevich 作为例子，原因很朴素：二维系统足够简单，所以可以真的把很多 regime 画出来。课程页也延续这个思路，用 QIF / Izhikevich 类模型演示 raster、firing rate 和电压轨迹。相比 HH 这样更细的生物模型，Izhikevich 的优势是参数和变量更少，便于把“模型能做什么”先看清楚；代价则是生物解释力更弱。

这一讲对单神经元模型的态度

更简单的模型不一定更“假”。如果你的研究问题是发放模式、阈值效应、分岔或噪声如何改变群体行为，那么更小的模型常常更有解释力，因为你能直接看见它的 regime 变化。

2.4 本章小结

神经元部分真正传递的是一个方法论：先把单细胞动力学抽象成可计算的状态，再用合适的可视化去读它的 regime。发放率曲线、raster、电压轨迹和频谱分别回答不同的问题，不能互相替代。

3 神经团块：平均场、振荡器与下一代模型

神经团块模型比单神经元更上一层抽象楼梯。它们要描述的不是某个细胞逐毫秒的膜电位，而是群体平均活动、同步性和振荡结构。视频里给出的第一个例子是 Wilson-Cowan，它用兴奋/抑制两个变量就能描述一个神经群体的平均 E-I 平衡。

```

1 using Neuroblox
2 using OrdinaryDiffEq
3 using CairoMakie
4 using Random
5 Random.seed!(1)
6
7 @named nm = WilsonCowan()
8 sys = system(nm)
9 tspan = (0, 100) # ms
10 prob = ODEProblem(sys, [], tspan)
11 sol = solve(prob, Tsit5())
12
13 fig, ax = plot(sol)
14 axislegend(ax)
15 fig
16
17 E = state_timeseries(nm, sol, "E")
18 fig = lines(E)
19 fig

```

Listing 2: Wilson-Cowan Blox 的标准仿真与状态提取。

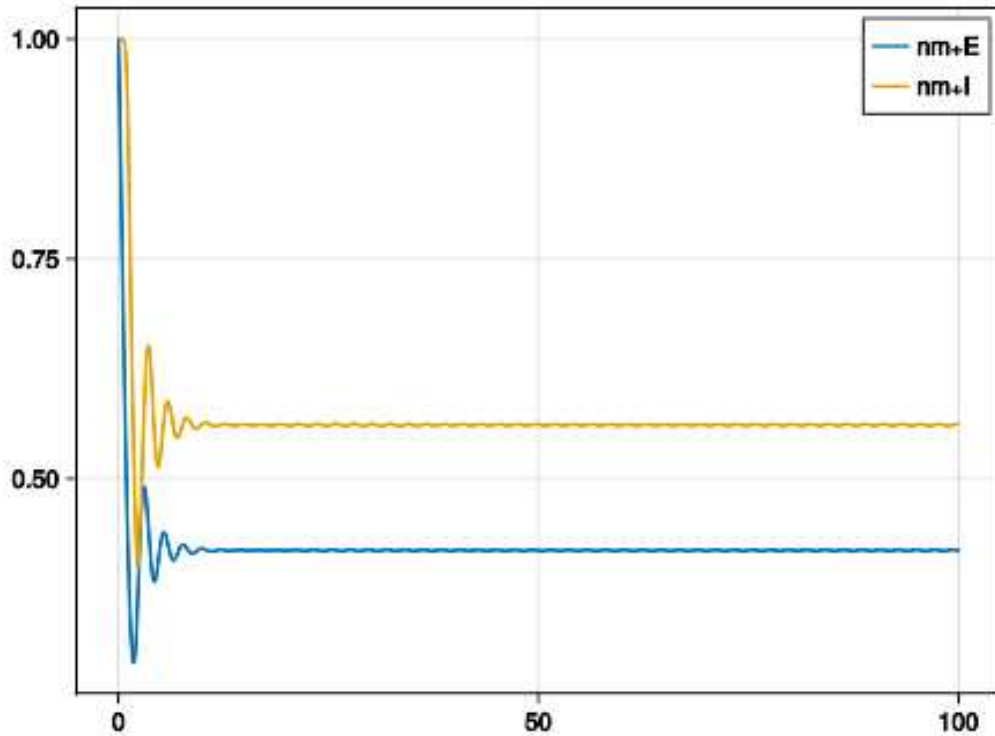


图 4: ConstantInput 连接后 Wilson-Cowan 的输出。输入把原先的 E-I 平衡整体推移了。

这张图的教学意义很直接：source 不只是“多加一个常数”，而是可以把整个系统的平衡点拖到新的位置。视频也明确指出，后续课程会用更复杂的 circuit 来继续研究 E-I balance 的变化。

4.2 事件触发 spike 源

下一类 source 是事件触发式的 spike source。课程页先给出 PoissonSpikeTrain：先定一个时间区间 `tspan`，再给固定 spike rate；或者把 rate 设成一个带 `distribution` 和 `dt` 的结构，让 rate 自己也随时间随机变化。

```

1 tspan = (0, 200) # ms
2 spike_rate = 0.01 # spikes / ms
3 @named spike_train_rate = PoissonSpikeTrain(spike_rate, tspan)
4
5 using Distributions
6 spike_rate = (distribution = Normal(1, 0.1), dt = 10)
7 @named spike_train_dist = PoissonSpikeTrain(spike_rate, tspan)

```

Listing 4: 固定速率与随机速率的 PoissonSpikeTrain。

课程页接着给出一个自定义事件源的完整示例：BernoulliSpikes。实现思路很清楚：先定义 source 结构体，再重载 `generate_spike_times` 来生成 spike 时间，最后重载 `connection_spike_affects` 来指定每次 spike 到来时如何修改下游神经元的方程。

```

1 struct BernoulliSpikes <: SpikeSource
2     name
3     namespace
4     tspan

```

```

5     probability_spike
6     dt
7     function BernoulliSpikes(probability_spike, tspan, dt; name, namespace=nothing)
8         new(name, namespace, tspan, probability_spike, dt)
9     end
10 end
11
12 import Neuroblox: generate_spike_times, connection_spike_affects
13
14 function generate_spike_times(source::BernoulliSpikes)
15     t_range = source.tspan[1]:source.dt:source.tspan[2]
16     t_spikes = Float64[]
17     for t in t_range
18         if rand(Bernoulli(source.probability_spike))
19             push!(t_spikes, t)
20         end
21     end
22     return t_spikes
23 end
24
25 function connection_spike_affects(source::BernoulliSpikes, ifn::IFNeuron, w)
26     eqs = [ifn.I_in ~ ifn.I_in + w]
27     return eqs
28 end

```

Listing 5: BernoulliSpikes 与事件回调的自定义示例。

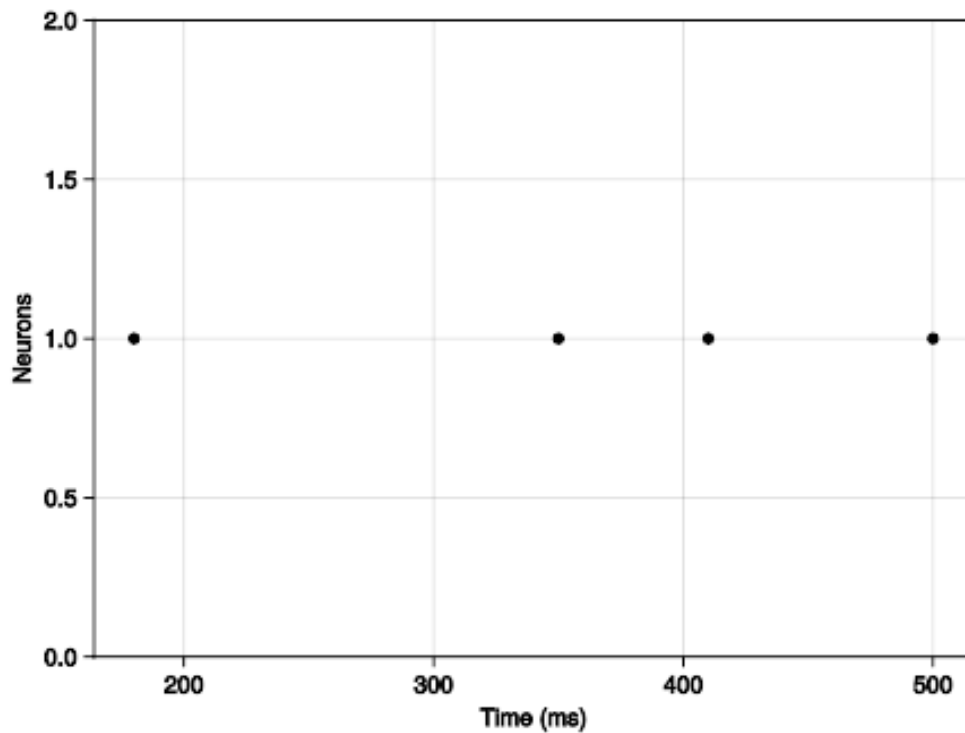


图 5: Bernoulli / Poisson 风格 spike source 驱动 IF 神经元的输出。

课程笔记

MIT Neuroblox 教材笔记 07: 回路模型总览

MIT Neuroblox 课程组 & Codex

2026 年 4 月 16 日



视频作者/频道: MIT Video Productions (讲者: Haris Organzidis)

发布日期: 课程标题标注 2025-01-15; YouTube 上传 2025-04-10

视频时长: 13:16

视频链接: <https://www.youtube.com/watch?v=ih7IELQ5W5>

目录

1 导论：课程定位与两个回路示例 2

1.1 本章小结 2

2 总览页的真正用途：把读者分流到两条详细建模路线 2

2.1 本章小结 3

3 命名空间与复合 Blox 3

3.1 本章小结 5

4 Corticostriatal 微组装：从局部竞争到层级回路 5

4.1 本章小结 6

5 PING：节律性抑制与注意 6

5.1 本章小结 7

6 总结与延伸 7

6.1 本章小结 8

证据说明

本讲以 MIT Neuroblox 官方课程页 [circuits.html](#)、对应的官方字幕，以及课程页中的代码和示意图为主证据。当前工作区没有找到独立的官方 slide PDF，因此正文以“课程页文本 + 字幕 + 代码块 + 官方课程图像”作为覆盖层；缺失项已在侧车日志中显式记录。

1 导论：课程定位与两个回路示例

这一讲承接前面“神经元、神经团块与外部源”的内容，目标不是再发明一种新对象，而是把前面已经学过的三类对象组合成 *circuits*。讲者一开始就把范围说得很清楚：这一讲只做两个文学来源明确的例子，一个是 biomimetic corticostriatal micro-assembly，另一个是 PING，也就是 pyramidal-interneuron gamma network。

更重要的是，这一讲的学习目标不是“看懂一个漂亮的图”，而是建立一个层级式建模观。课程页把目标写得很直接：从系统神经科学文献里抽取复杂模型，构造带层级结构的 composite Bloxs，并用更高级的 plotting recipes 去看这些复合块和它们形成的回路。

示例	主要结构	这一讲想让你学会什么
Corticostriatal micro-assembly	多层 composite Blox + 多尺度输入	从局部竞争扩展到层级回路
PING	兴奋-抑制环路 + 节律驱动	用节律性抑制解释 gamma / beta 注意机制

表 1: 本讲两个回路示例的教学分工。

这一讲的核心转折

前面几讲关心的是“单个 Blox 怎么定义、怎么连线、怎么仿真”。这一讲关心的是“当 Blox 里面还有 Blox 时，外层名字如何被正确继承、连接项如何被正确汇总、以及如何把多个层次的生物过程放进同一个系统里”。

字幕里最早的几句也把这个转折点说明了：讲者说，前面已经看过如何定义和仿真 neurons、neural masses 和 external sources，现在要把这些元素不同组合起来，做成 circuits。这个问题设置很重要，因为它决定了后面所有例子的共同骨架。

1.1 本章小结

这一部分只做了一件事：把课程主题从“单元对象”推进到“层级回路”。后面的所有技术细节，都是围绕如何组织层级结构、如何解释两类典型回路展开的。

2 总览页的真正用途：把读者分流到两条详细建模路线

如果只把 *circuits* 页面当成一个“短介绍页”，就会低估它的教学作用。官方页面虽然内容不长，但它承担的是一个路由器（routing page）的角色：它告诉读者，本讲的回路建模主要沿两条路径展开，一条进入 *CS_circuit*，一条进入 *PING_circuit*。换句话说，这一页并不是要把所有细节一次讲完，而是要给出“从总览到深入”的阅读结构。

入口页	对应深入单元	继续阅读时应该重点追踪什么
<code>circuits</code>	<code>CS_circuit</code>	层级 composite Bloxs、视觉输入、 上行调制、群体绘图 recipe
<code>circuits</code>	<code>PING_circuit</code>	E/I 环路、gamma/beta 节律、注 意与时序锁定

表 2: 官方总览页给出的两条后续阅读路线。

从教材组织的角度看，这种写法有两个好处。第一，它把“回路模型”定义成一个比单神经元更高的教学层级，但又不要求你在总览页一次掌握所有细节。第二，它为后续单元建立了一个明确的分工：`CS_circuit` 负责讲层级结构与多尺度视觉回路，`PING_circuit` 负责讲节律性 E/I 环路及其功能解释。因此，`circuits` 页真正传递的，不只是两个例子名称，而是“阅读时先抓哪条主线”的方法提示。

为什么总览页仍然值得单独成章

因为它把建模语言提升了一层。前几讲更多是在讲对象如何被定义，这一页则开始讲“对象如何被组织成课程后半段真正关心的回路单元”。如果跳过这一步，读者会直接掉进两个专项案例，而失去它们之间的共同语法。

2.1 本章小结

总览页的功能不是替代后续专题，而是把后续专题的阅读顺序、角色分工和共同建模语言先交代清楚。把这一步讲透，后面的 `corticostriatal` 与 `PING` 两章就不会显得像两份彼此无关的案例报告。

3 命名空间与复合 Blox

官方课程页的“Namespaces”小节是这一讲最关键的建模约定。对于 `atomic Blox`，‘namespace = nothing’ 通常就够了，因为它们没有子结构；但一旦进入 `composite Blox`，尤其是多层 `composite Blox`，外层名字就不能省。原因很直接：`Neuroblox` 要靠这个外层命名空间，把所有连接项累积起来，再正确地匹配到每个子块的输入变量上。

为什么复合块必须显式命名

`atomic Blox` 只有一个层级，变量名直接对应动力学状态；`composite Blox` 内部还有 `Blox`，因此变量名必须能表达“我属于哪一个外层块、哪一个中层块、哪一个局部神经元”。命名空间的作用就是把这种层级关系编码进变量名里，并让连接方程在展开后仍然可追踪。

类型	是否有自己的动力学	是否需要 namespace
<code>atomic Blox</code>	有	通常不需要
<code>CompositeBlox</code>	没有，内部只组织子块	需要，尤其是多层结构
<code>Source</code>	只有输出，没有输入	视连接上下文而定

表 3: 命名空间约定在不同类型 Blox 上的含义。

官方页给出的例子是两个 ‘WinnerTakeAllBlox’, 把外层名字设成 `model_name = :g`, 再用 `system_from_graph` 生成最终系统。这里的关键不是语法, 而是语义: 同一个图对象里可以并排存在多个 composite Blox, 而 `model_name` 则是最终被写入系统名的外层标识。

```

1 model_name = :g
2
3 @named
4   wta1 = WinnerTakeAllBlox(namespace = model_name)
5
6 @named
7   wta2 = WinnerTakeAllBlox(namespace = model_name)

```

Listing 1: 两个 WTA 复合块的命名与建系统写法。

```

1 g = MetaDiGraph()
2 add_edge!(g, wta1 => wta2, weight=
3 10
4 )
5
6 sys = system_from_graph(g, name = model_name)

```

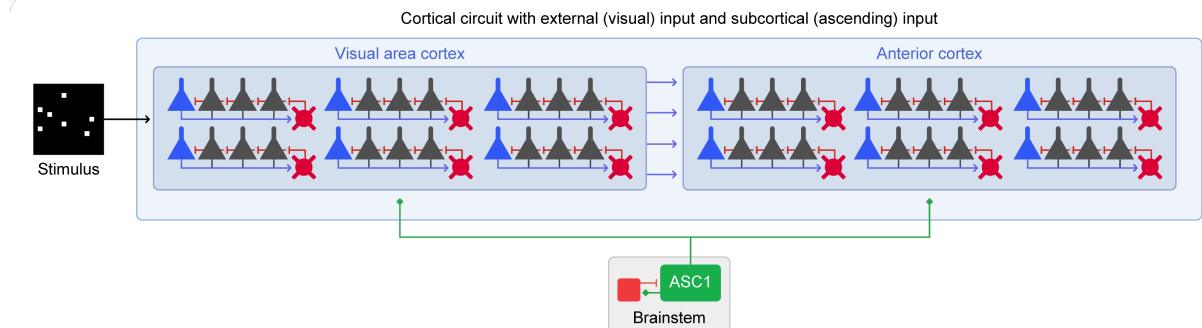
Listing 2: 把图结构转换成最终系统。

这两段代码的工作方式很简单, 但非常有代表性。第一段先实例化两个 WTA 复合块, 并让它们共享同一个外层命名空间 `model_name`。第二段把这两个复合块放进 `MetaDiGraph()`, 再用 `add_edge!` 声明块之间的连接权重。最后 `system_from_graph(g, name = model_name)` 把整个图编译成一个最终的方程系统。换句话说, 图不只是画出来, 而是直接成为模型定义本身。

对照课程页里的变量名, ‘wta1 exci1 V’ 和 ‘wta2 exci2 V’ 是两个局部神经元的膜电位; 一旦外层命名空间 ‘g’ 被加进去, 它们就变成 ‘g wta1 exci1 V’ 和 ‘g wta2 exci2 V’。这里每一个 ‘ ’ 都表示向外多挂一层命名空间, 读法是从右往左回溯: 先是局部 neuron, 再是 WTA block, 再是最外层 model name。

$$\text{wta1 exci1 V, wta2 exci2 V} \longrightarrow \text{g wta1 exci1 V, g wta2 exci2 V} \quad (1)$$

其中, ‘g’ 是外层模型名, ‘wta1’ / ‘wta2’ 是两个复合块, ‘exci1’ 是 WTA 里的一个兴奋性神经元, ‘V’ 是膜电位状态变量。这个例子本质上是在告诉你: 层级化建模时, 变量名本身就是结构信息。



为什么这一段不是“语法演示”而已

如果只把这些代码看成 Julia 语法，就会漏掉重点。这里真正重要的是：复合块之间的连接不是手工把方程一条条写死，而是先建立图，再让系统从图中展开。命名空间就是保证这种自动展开正确无误的索引机制。

3.1 本章小结

这一节把讲座最底层的约定钉住了：atomic Blox 可以简单，composite Blox 必须显式命名，而 `system_from_graph` 则负责把层级图编译成最终系统。只要这个约定没搞错，后面的所有回路都能沿着同一条路线扩展。

4 Corticostriatal 微组装：从局部竞争到层级回路

讲者把 corticostriatal 模型定位成一个多尺度、强生物约束的例子。它不是为了展示某个单一方程，而是为了说明一个完整的神经系统模型如何从突触、单神经元、局部集合一路扩展到脑区和更大尺度的回路。字幕里说得很明确：这个模型从神经受体层面一直跨到行为层面，输入可以是外界图像，输出可以是左/右决策。

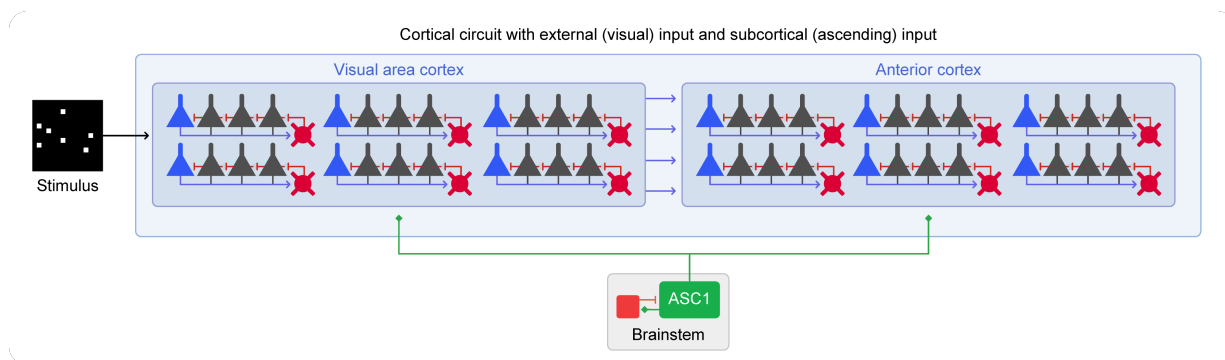


图 1: 官方课程页中的 corticostriatal 扩展示意。它把视觉输入、两个 cortical block 和 brainstem 的并行调制放在同一张图上，最能说明这一讲的层级化建模思路。

这张图的教学价值在于：它把“模型从底层开始搭”这句话变成了可视化结构。最小层是 superficial cortical layer 的 lateral inhibition；再往上是多个 winner-take-all blocks 组成的 cortical block；再往上是两个 cortical blocks 彼此连接，一个代表 visual cortex，一个代表 anterior cortex；最后，image stimulus 和 brainstem 作为外部源接入，前者连到 visual cortex，后者作为 ascending system 同时调制两个 cortex。

多尺度不是口号，而是连接约束

讲者强调这个模型“约束于 biology”，不仅体现在参数上，也体现在结构上。也就是说，你不是随便堆积模块，而是要让连接规则、层级关系和外部源的类型都尽量贴近文献里的机制假设。

在这个例子里，winner-take-all 不是抽象概念，而是局部 lateral inhibition 的直接结果：四个兴奋性神经元和一个 feedback interneuron 互相作用，最后通常是某一个 excitatory neuron 赢得竞争并继续放电。讲者解释得很清楚，最初更频繁地发放的那个神经元，更容易通过自己驱动 interneuron，再由 interneuron 抑制其他兴奋性神经元，从而把竞争锁定下来。

层级	作用	对行为的意义
superficial cortical layer	局部 WTA / lateral inhibition	决定哪个局部群体先赢
cortical block	聚合多个 WTA 结构	支持更复杂的表征与任务分工
visual/anterior cortex	跨区耦合	把局部竞争放进更大回路
image stimulus + brainstem	外部输入与上行调制	让模型对感知与状态敏感

表 4: corticostriatal 例子中各层级的功能分工。

课程视频还特别强调了一个可视化层面的事实：Neuroblox 提供的不只是仿真实接口，还有适合复合块的绘图方式。讲者提到可以把一个 composite block 直接喂给 stack plot，也可以传入一个神经元向量，或者做 mean field plot，再或者看 power spectrum。这个设计很重要，因为复合模型的状态太多，单看某一条轨迹很容易丢失整体结构。

为什么要用多种 plot recipe

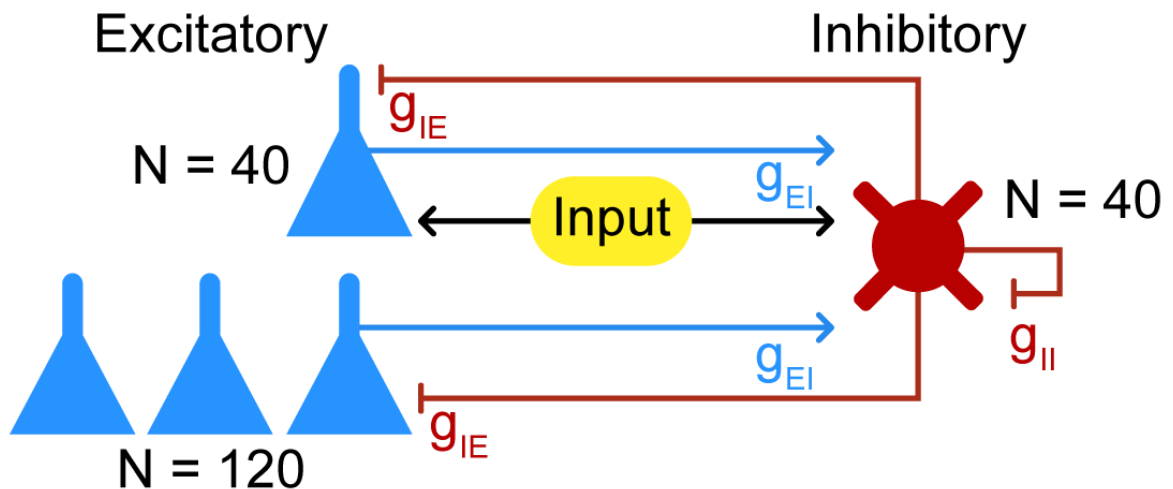
stack plot 更像结构视图，mean field plot 更像群体平均视图，power spectrum 则更适合看节律或频域特征。对这种层级回路来说，三者回答的问题不同，不能互相替代。

4.1 本章小结

这一节的重点是“从局部竞争到层级回路”的建模路径。corticostriatal 例子把 WTA、层级 cortical block、图像输入和脑干调制串成一个完整故事，也把“复合 Blox 如何嵌套”讲成了一个可以运行的系统。

5 PING：节律性抑制与注意

第二个例子是 PING，Pyramidal-Interneuron Gamma network。讲者把它描述成一个由 pyramidal neurons 和 interneurons 组成的网络，目标是生成 gamma rhythms，并用这种节律性解释 attention。这里的核心思想不是“哪个神经元最强”，而是“抑制-兴奋的时序关系如何把某一群神经元锁定到合适的节律上”。



这个图的作用很直接：它把兴奋性群体、抑制性群体和外部输入放在一起，说明 gamma rhythm

是怎么从 E/I 反馈里冒出来的。讲者在视频里说得很明确：某些 excitatory neurons 会对刺激更敏感，在与 interneurons 的相互作用中，它们会对齐节律并与外部刺激同步；一旦峰值和 stimulus 对齐，就可以认为模型“在注意这个刺激”。

这里的 attention 是一个节律假说

讲者并不是在给注意做完整的认知理论，而是在采用一个工作性假设：如果节律峰值与刺激对齐，那么该刺激更容易被处理。于是 attention 在这个模型里就等价于 rhythmic alignment。

更进一步，视频还提到了原始工作里的一个重要细节：gamma rhythm 其实嵌在 beta rhythm 之中，也就是 beta rhythm 驱动 gamma。教程里先让你生成 gamma rhythm，如果想在上面再叠加 beta，就只需要再加一个外部源，形式上是一个 beta 频率的 sinusoidal current。

$$\text{gamma rhythm} \subset \text{beta rhythm} \quad (2)$$

这里不是在做严格的包含关系证明，而是在表达一个建模事实：beta 作为慢一些的调制层，会影响 gamma 的出现时机和相位。讲者接着强调，真正起作用的是 interneurons 对 pyramidal neurons 的负反馈，以及这些 spike timing 的细节。换句话说，节律不是背景噪声，而是注意机制的一部分。

在讲到 tutorial 时，视频也给出了非常实用的提醒：先把 gamma rhythm 搭出来，beta 可以之后再加；而课程页面上的 Introduction 部分对于 namespace 有重要说明，应该先读，因为后面做 circuit models 时会频繁遇到层级变量名的展开问题。这个提醒和前一节的命名空间讨论是连在一起的。

PING 例子要抓住的两个层次

第一层是结构：兴奋性群体和抑制性群体如何构成竞争-反馈环。第二层是时序：为什么 rhythmic inhibition 会比完全 asynchronous 的 inhibition 更有效地把注意“锁”到某个刺激上。

5.1 本章小结

PING 这一节说明，回路模型不只是空间上的连接图，也可以是时间上的节律组织。gamma / beta 的关系把 E/I 反馈、刺激对齐和注意机制绑在一起，让“节律性抑制”成为可计算的假说。

6 总结与延伸

这一讲的技术主线其实只有一条：当模型从单个神经元走向复合回路时，最重要的不是再多加几个方程，而是把层级关系、命名空间和外部输入的角色定义清楚。corticostriatal 例子展示了如何把局部 WTA、多个 cortical block、视觉输入和上行调制拼成一个大系统；PING 例子则展示了如何用节律性抑制把“注意”写成一个可运行的动力学过程。

阅读顺序的建议

讲者在结尾明确提醒：先去官方网站的 Circuit Models Introduction 页面，确认 namespace 的规则，再回头做 corticostriatal model。这个顺序不是形式主义，而是为了避免在层级模型里把变量名和连接项展开错。

如果把这一讲压缩成一句话，那就是：从 atomic Blox 到 composite Blox，再到多层 circuit model, Neuroblox 的关键能力是把结构信息直接编码进系统构造过程。这也是为什么这一讲虽然只给了两个例子，却已经足够覆盖后面更复杂的电路建模路线。

最后，视频也给了作业层面的预告：这些示例和 challenge problems 会比较长，不需要一次做完；先把基础的 namespace 和层级结构弄明白，再继续往后推进即可。

6.1 本章小结

这一节做的是收束：回到最初的目标，理解层级结构、命名空间和节律性反馈如何成为同一套建模语言。读完这一讲后，后续课程页里的更复杂电路，本质上只是这套语言的进一步组合。

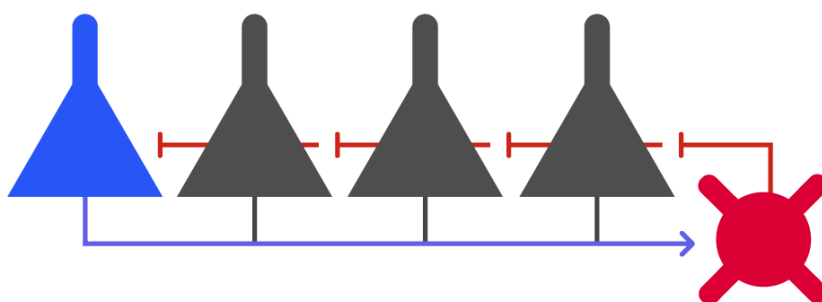
课程笔记

MIT RES.9-009 第 08 讲：皮层-纹状体微组 装的仿生模型

五道口纳什 & Codex

2026 年 4 月 15 日

Superficial cortical lateral inhibition circuit



视频作者/频道：MIT OCW Neuroblox course page

发布日期：January IAP 2025

视频时长：课程页 / notebook

视频链接：<https://ocw.mit.edu/courses/res-9-009-introduction-to-computational-neuroscience/>

目录

1 课程目标与层级建模 2

1.1 本章小结 2

2 Winner-Take-All 与侧抑制回路 2

2.1 本章小结 4

3 浅层皮层微回路与上行调制 4

3.1 本章小结 6

4 扩展视觉处理模型 6

4.1 本章小结 8

5 练习、敏感性分析与研究联系 8

5.1 本章小结 8

6 总结与延伸 9

6.1 本章小结 9

1 课程目标与层级建模

这讲的目标，不是把一个孤立回路讲成一个“能跑起来的例子”，而是把 Neuroblox 的层级建模思想讲清楚：先从单个神经元对象出发，再把若干神经元封装成一个局部功能块，最后把多个功能块再组合成更高一级的皮层模型。官方课程页明确给出三层路线：从 *Neuron Blox* 到包含神经元对象的 *CompositeBlox*，再到包含前者的更大 *CompositeBlox*。这个自下而上的路线很重要，因为后面要做的 category learning 不是靠一个神经元回路完成的，而是靠若干层结构叠加出微组装、节律调制和视觉输入的联动。

先看结构，再看代码

这一讲的关键不在“代码跑通”，而在于理解每一级封装解决了什么问题。单神经元层负责可解释的电生理动力学，WTA 层负责局部竞争，CorticalBlox 负责把许多局部竞争拼成浅层皮层块，最后再加上 ASC1 与图像刺激，才得到可用于后续 category learning 的扩展系统。

Superficial cortical lateral inhibition circuit

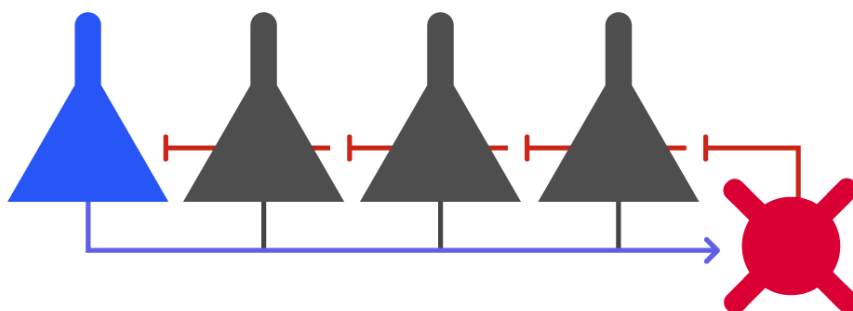


图 1: WTA 侧抑制回路示意图，也是本讲的封面图。

这也解释了为什么课程页在开头就要求先处理 `CS_circuit.ipynb`：它不是一个“附录式练习”，而是后续带塑性、带分类学习的大模型的结构前导。换句话说，这讲真正教的是一种搭建顺序，而不是单个对象的 API 使用。

1.1 本章小结

本讲先把“神经元-微回路-皮层块-扩展输入”这条层级链条立起来。只要把这个链条抓住，后面的 WTA、ASC1 和视觉输入就不是零散技巧，而是同一个建模框架中的不同层级。

2 Winner-Take-All 与侧抑制回路

第一段主体内容是手工搭建一个侧抑制回路，也就是 winner-takes-all (WTA) 回路。官方页把它作为 Figure 1，并明确说这个回路受浅层皮层结构启发。电路里有 5 个兴奋性神经元，它们都和同一个抑制性中间神经元形成互惠连接：兴奋性神经元把活动送给抑制神经元，抑制神经元再把抑

制回送给各个兴奋性神经元。这样一来，局部的共同兴奋会被一个共享抑制器“裁决”，最终往往只留下少数赢家。

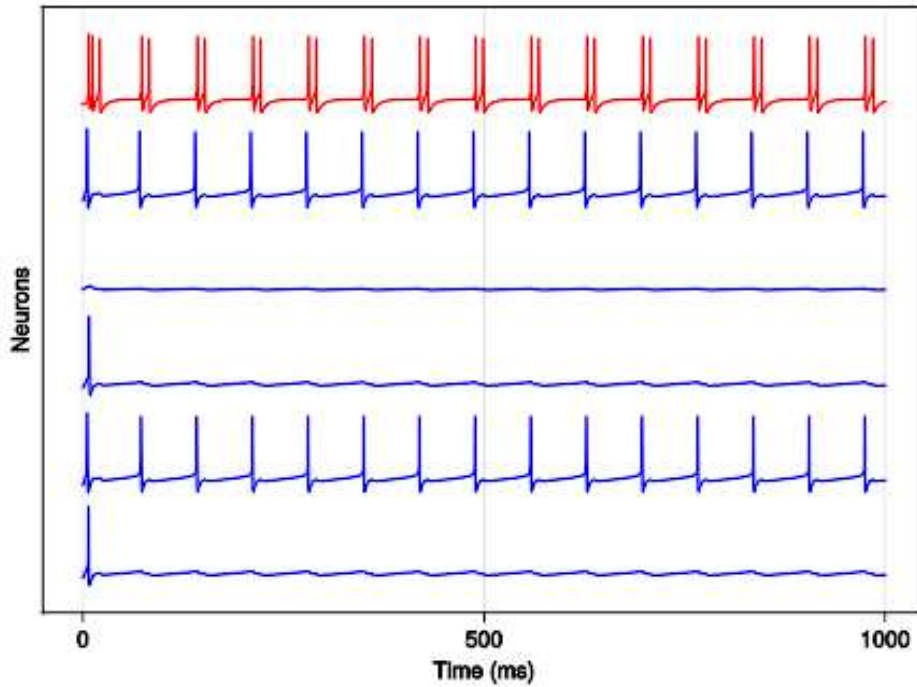


图 2: 5 个兴奋性神经元与 1 个抑制性神经元构成的 stackplot。

```

1 using Neuroblox
2 using OrdinaryDiffEq
3 using Random
4 using CairoMakie
5
6 model_name = :g
7 @named inh = HHNeuronInhibBlox(namespace=model_name, G_syn = 4.0)
8 @named exci1 = HHNeuronExciBlox(namespace=model_name, I_bg = 5 * rand())
9 @named exci2 = HHNeuronExciBlox(namespace=model_name, I_bg = 5 * rand())
10 @named exci3 = HHNeuronExciBlox(namespace=model_name, I_bg = 5 * rand())
11 @named exci4 = HHNeuronExciBlox(namespace=model_name, I_bg = 5 * rand())
12 @named exci5 = HHNeuronExciBlox(namespace=model_name, I_bg = 5 * rand())
13
14 g = MetaDiGraph()
15 for exci_neuron in [exci1, exci2, exci3, exci4, exci5]
16     add_edge!(g, inh => exci_neuron, weight = 1)
17     add_edge!(g, exci_neuron => inh, weight = 1)
18 end
19
20 @named sys = system_from_graph(g)
21 prob = ODEProblem(sys, [], (0.0, 1000), [])
22 sol = solve(prob, Vern7())
23 fig = stackplot([exci1, exci2, exci3, exci4, exci5, inh], sol)

```

Listing 1: 手工搭建 WTA 回路的核心逻辑

这段代码有三个层次的语义。第一，‘HHNeuronExciBlox’ 和 ‘HHNeuronInhibBlox’ 不是把电生理细节压扁成一个黑箱，而是把兴奋/抑制神经元作为可组合对象暴露出来。第二，‘MetaDiGraph()’ 让连接关系先于系统方程存在，图决定了后续 `system_from_graph` 会生成怎样的动力学系统。第三，‘stackplot’ 只是可视化接口，但它在教学上很重要，因为它把每个神经元的时序活动按固定偏移堆叠起来，方便看出谁先活跃、谁被抑制。

‘density’ 代表什么

在把两个 WTA 进一步连接时，‘density’ 不是权重，而是连接概率。官方页明确说，它对应于从一个 WTA 的兴奋性神经元到另一个 WTA 的兴奋性神经元是否连边的 Bernoulli 成功概率。换句话说，‘density’ 控制的是结构稀疏性，‘weight’ 才控制突触强度。

```

1 N_exci = 5
2 @named wta1 = WinnerTakeAllBlox(namespace=model_name, I_bg = 5, N_exci = N_exci)
3 @named wta2 = WinnerTakeAllBlox(namespace=model_name, I_bg = 4, N_exci = N_exci)
4
5 g = MetaDiGraph()
6 add_edge!(g, wta1 => wta2, weight = 1, density = 0.5)
7
8 sys = system_from_graph(g, name = model_name)
9 prob = ODEProblem(sys, [], (0.0, 1000), [])
10 sol = solve(prob, Vern7())
11 neuron_set = get_neurons([wta1, wta2])
12 fig = stackplot(neuron_set, sol)

```

Listing 2: 把 WTA 封装成复合块并连接两个 WTA

这里真正值得学的，不是两个 WTA 连到一起这件事本身，而是 ‘WinnerTakeAllBlox’ 把 “一个局部竞争单元” 封装成了可复用模块。这样一来，上层模型就不再关心每个神经元之间的细枝末节，而只需要调节块与块之间的连线密度和背景输入。课程页还提示可以改变兴奋性神经元个数，或让连接拓扑保持不变，这实际上是在让你体会 “局部竞争单元的规模” 和 “跨单元耦合” 各自对群体动力学的作用。

2.1 本章小结

WTA 回路的核心机制是共享抑制带来的局部竞争。Neuroblox 的价值在于，它把这个机制封装成能复用的 ‘Blox’，并让你用图结构和概率连边去控制模块间耦合，而不是手写一大堆方程和索引。

3 浅层皮层微回路与上行调制

从两个 WTA 走向十个 WTA，课程页开始构造一个更像 “浅层皮层块” 的系统。官方页把它称为 SCORT，并展示了多个 WTA 互联后的皮层结构图。这里多出的角色是 feedforward interneuron (前馈抑制性中间神经元)，它会连接到模块内的所有兴奋性细胞，同时接受来自上层或相邻块的输入。它的教学意义在于：它不是简单地再加一个抑制神经元，而是把节律控制从局部 WTA 提升到了多微回路的群体层面。

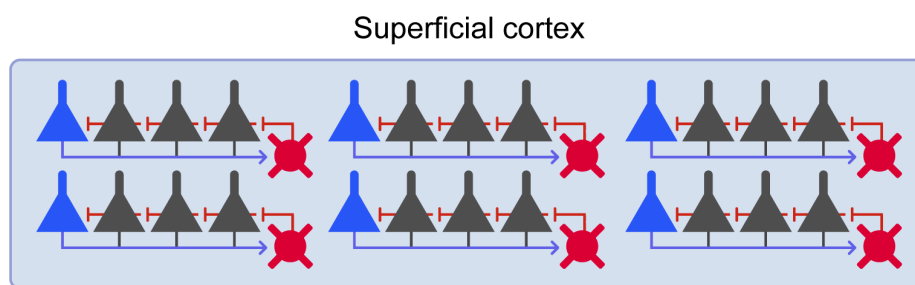


图 3: 由多个 WTA 微回路组成的浅层皮层块示意图。

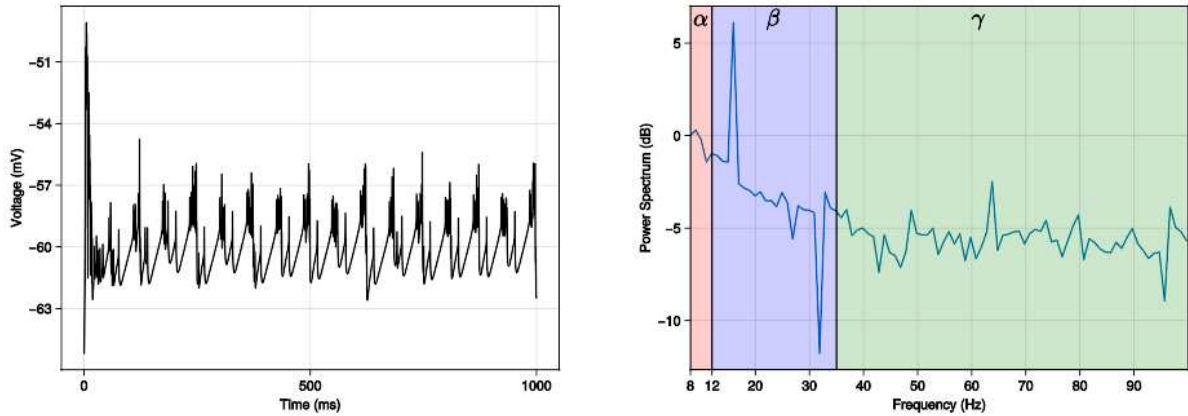
```

1 N_wta = 10
2 N_exci = 5
3 G_syn_exci = 3.0
4 G_syn_inhib = 4.0
5 G_syn_ff_inhib = 3.5
6 I_bg = 5.0
7 density = 0.01
8
9 wtas = [WinnerTakeAllBlox(name = Symbol("wta$i"),
10                             namespace = model_name,
11                             N_exci = N_exci,
12                             G_syn_exci = G_syn_exci,
13                             G_syn_inhib = G_syn_inhib,
14                             I_bg = I_bg)
15         for i in 1:N_wta]
16
17 @named n_ff_inh = HHNeuronInhibBlox(; namespace=model_name, G_syn=G_syn_ff_inhib)
18
19 @named ASC1 = NextGenerationEIBlox(; namespace=model_name,
20     Ce = 2*26, Ci = 26,
21     v_syn_ee = 10.0, v_syn_ei = -10.0,
22     v_syn_ie = 10.0, v_syn_ii = -10.0,
23     alpha_inv_ee = 10.0/26, alpha_inv_ei = 0.8/26,
24     alpha_inv_ie = 10.0/26, alpha_inv_ii = 0.8/26,
25     k_ei = 0.6*26, k_ie = 0.6*26)

```

Listing 3: 浅层皮层块与 ASC1 上行系统的核心参数

这段代码的语义可以分成两层。第一层是 WTA 簇本身：每个簇仍然维护自己的局部侧抑制。第二层是上行调制器 ASC1，它被建模成 ‘NextGenerationElBlox’，也就是下一代兴奋-抑制神经质量模型。课程页特别强调参数被固定到一个 16 Hz 的调制频率，这不是随便选的，而是为了把可见的频带峰值和皮层节律联系起来。换句话说，这里不是在做“任意振荡器”，而是在做“能驱动皮层块节律的上行系统”。



(a) 平均膜电位

(b) 功率谱

图 4: ASC1 作用下的群体平均活动与频谱, 峰值位于约 16 Hz。

```

1 @named CB = CorticalBlox(N_wta = 10, N_exci = 5, density = 0.01,
2                           weight = 1, I_bg_ar = 7; namespace = model_name)
3
4 g = MetaDiGraph()
5 add_edge!(g, ASC1 => CB, weight = 44)
6
7 sys = system_from_graph(g, name = model_name)
8 prob = ODEProblem(sys, [], (0.0, 1000))
9 sol = solve(prob, Vern7())
10
11 neuron_set = get_neurons(CB)
12 fig = stackplot(neuron_set[1:50], sol)
13 fig_mean = meanfield(CB, sol)
14 fig_pow = powerspectrumplot(CB, sol; sampling_rate = 0.01)

```

Listing 4: 把 ASC1 连接到 CorticalBlox 并分析群体统计

这里最值得注意的是 ‘meanfield’ 和 ‘powerspectrumplot’。前者把大量神经元的时域活动压缩成一个平均膜电位轨迹, 后者进一步把频率内容提取出来。课件在这里做了一个很重要的教学动作: 它不满足于 “网络在动”, 而是让你用频谱去确认 “网络为什么在某个节律上动”。这比单看栅格图更接近机制分析。

3.1 本章小结

浅层皮层块的关键不是更多神经元, 而是更多层级的抽象: WTA 负责局部竞争, ‘CorticalBlox’ 负责把竞争单元拼成皮层块, ‘NextGenerationEIBlox’ 则把上行调制和节律峰值显式带进模型。这样一来, 结构、频率和行为就被统一到同一个图模型里。

4 扩展视觉处理模型

最后一步是把视觉处理接进来, 形成更接近任务驱动的扩展模型。课程页引入了 ‘VAC’ (visual area cortex) 和 ‘AC’ 两个 ‘CorticalBlox’, 并通过 ‘ImageStimulus’ 把图像样本作为外部输入源接

入。这里最重要的概念是：图像不是一个“背景图片”，而是一个由像素变量组成的输入场，每个像素都被映射到视觉皮层中的一个神经元或一组神经元上。

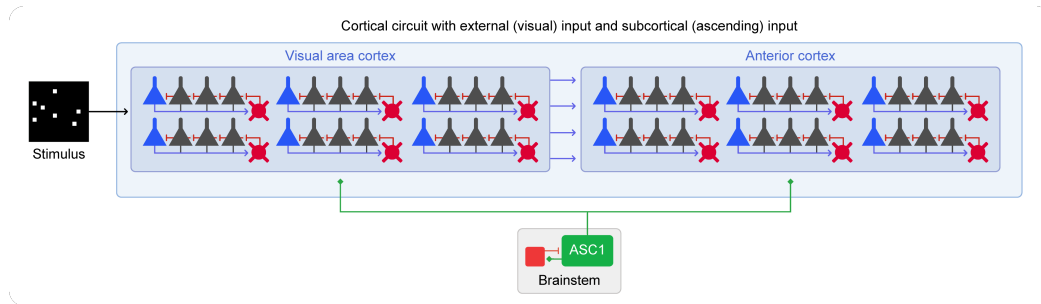


图 5: 扩展后的皮层模型，包含 Cortex、Brainstem 和 ImageStimulus。

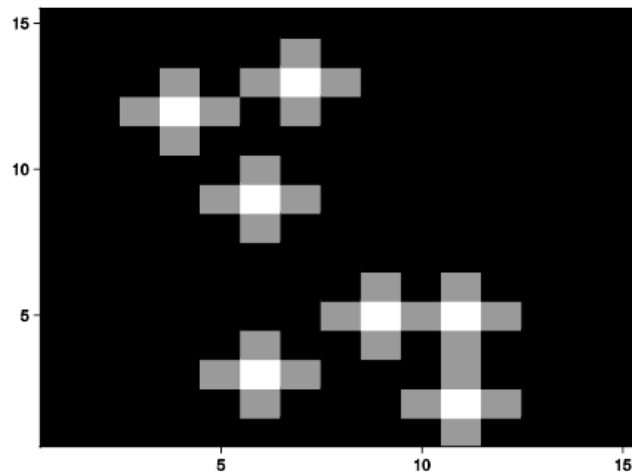


图 6: 图像样本与像素连接的示意。像素不再是静态图片，而是网络输入变量。

```

1 @named VAC = CorticalBlox(namespace = model_name, N_wta = 10, N_exci = 5,
2                               density = 0.01, weight = 1)
3 @named AC = CorticalBlox(namespace = model_name, N_wta = 10, N_exci = 5,
4                               density = 0.01, weight = 1)
5
6 using CSV
7 using DataFrames
8 using Downloads
9 image_set = CSV.read(Downloads.download(
10     "raw.githubusercontent.com/Neuroblox/NeurobloxDocsHost/refs/heads/main/data/
11     image_example.csv"), DataFrame)
12
13 image_sample = 2
14
15 @named stim = ImageStimulus(image_set[[image_sample], :], namespace = model_name,
16                               t_stimulus = 1000, t_pause = 0)
17
18 pixels = Array(image_set[image_sample, 1:end-1])
19 pixels = reshape(pixels, 15, 15)
20 fig = heatmap(pixels, colormap = :gray1)

```

```

19
20 g = MetaDiGraph()
21 add_edge!(g, stim => VAC, weight = 14)
22 add_edge!(g, ASC1 => VAC, weight = 44)
23 add_edge!(g, ASC1 => AC, weight = 44)
24 add_edge!(g, VAC => AC, weight = 3, density = 0.08)
25
26 sys = system_from_graph(g, name = model_name)
27 prob = ODEProblem(sys, [], (0.0, 1000))
28 sol = solve(prob, Vern7())

```

Listing 5: 视觉输入与扩展皮层模型

这段代码把两个层次的输入都体现出来了：一类是外部图像刺激 ‘stim’，另一类是上行系统 ‘ASC1’。它们分别控制视觉皮层的驱动和节律背景，而 ‘VAC => AC’ 的稀疏连接又把局部特征流向更高层级。课件建议你尝试不同图像样本，并观察 ‘VAC’ 和 ‘AC’ 的空间放电模式变化，这其实就是在提醒你：这个模型不是单纯的节律模型，而是一个能承载感知任务的层级电路。

4.1 本章小结

扩展模型的意义在于把“视觉输入”从一个外部文件，变成了可以沿着图图结构传播的神经动力学变量。到这里，WTA、浅层皮层块、上行调制和图像刺激都被放进一个统一框架，后续 category learning 才有了可计算的底座。

5 练习、敏感性分析与研究联系

课程页最后给出的练习非常有价值，因为它们不是机械复现，而是参数敏感性和拓扑敏感性的入口。第一类练习是改变 ASC1 的参数，把 16 Hz 调到 5 Hz、10 Hz 或 20 Hz，观察功率谱峰值如何移动。第二类练习是加入第二个上行系统 ‘ASC2’，让 ‘ASC1’ 只连 ‘VAC’，‘ASC2’ 只连 ‘AC’，再看两个调制器叠加后是否出现双峰。第三类练习是改变刺激时长 `t_stimulus` 和静息时长 `t_pause`，测试刺激停止后皮层块是否还能在反馈环路里维持活动。

这些练习的共同点，是它们把“节律生成”“输入持续时间”和“反馈拓扑”拆成了三个可独立操作的控制变量。对教材学习者而言，这比只记住“模型会振荡”更重要，因为它告诉你振荡不是魔法，而是参数、连接和驱动共同塑造出来的。

这里哪些是课件原话，哪些是教材补充

上面的练习项都来自官方课程页；但如果你进一步去推导“为什么某个参数会把频率从 16 Hz 推到 10 Hz”，那就已经进入教材补充和研究解释层面，不再是页面的直接陈述。阅读时要把“课件说明”与“我对机制的解释”分开。

5.1 本章小结

这一讲真正想训练的，不是按图索骥地复现单个回路，而是把局部竞争、节律调制和视觉输入做成可调的模块化系统。练习题之所以重要，是因为它们把模型从“演示”推进到“实验对象”。

6 总结与延伸

如果把这一讲压缩成一句话，那就是：Neuroblox 不是在教你写一个神经网络脚本，而是在教你用层级化对象组织生物物理模型。WTA 提供局部竞争，CorticalBlox 把竞争单元组织成浅层皮层块，ASC1 把上行节律注入系统，ImageStimulus 则把视觉输入接入动力学网络。四者合起来，就形成了一个能承载后续 category learning 的底层平台。

从研究视角看，这种建模方式有两个直接价值。第一，它让你能在保持生物解释性的前提下快速搭建不同层级的电路。第二，它让你把结构、节律和输入分开调节，从而做出更像“实验”的模型分析，而不是只看一次仿真的截图。课程页所引用的 Pathak 等工作 and Next-generation neural mass 文献，实际上都在提醒你：真正有解释力的模型，往往来自对层级和接口的精确定义，而不只是更复杂的方程。

6.1 本章小结

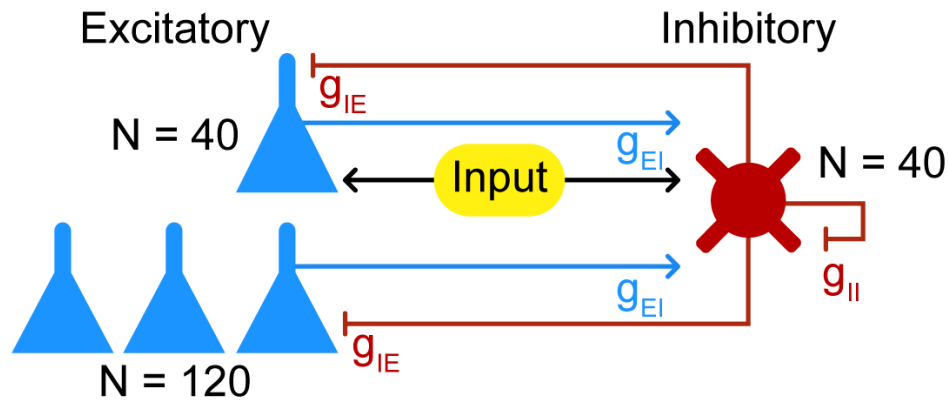
本讲的核心收获是三个接口：局部竞争接口、皮层块接口和上行调制接口。只要这三个接口清楚，后续任何更大的分类学习模型，都可以沿着同样的方法继续堆叠和验证。

课程笔记

MIT RES.9-009 第 09 讲： Pyramidal-Interneuron Gamma 网络

五道口纳什 & Codex

2026 年 4 月 15 日



视频作者/频道: MIT OCW Neuroblox course page

发布日期: January IAP 2025

视频时长: 课程页 / notebook

视频链接: MIT OCW course page

目录

- 1 课程定位与网络直觉 2
 - 1.1 本章小结 2
- 2 动力学方程、细胞类型与参数 2
 - 2.1 本章小结 3
- 3 Neuroblox 实现 3
 - 3.1 导入与初始化 3
 - 3.2 定义神经元与连接图 4
 - 3.3 邻接矩阵构图与求解 4
 - 3.4 本章小结 5
- 4 结果解读与练习 5
 - 4.1 本章小结 6
- 5 总结与延伸 7
 - 5.1 参考文献 7
 - 5.2 本章小结 7

1 课程定位与网络直觉

这一讲只有官方课程页与 notebook 作为证据，没有嵌入式视频，因此最稳妥的读法是把它当成一个“从纸面说明到可运行实现”的 PING 网络教程。页面标题和封面图已经把问题说得很清楚：我们要搭一个 pyramidal-interneuron gamma (PING) 网络，目标不是抽象地讨论振荡，而是复现 Börgers, Epstein, and Kopell 的经典回路，看看 Neuroblox 如何把这个回路写成可求解的图结构系统。

PING 的核心直觉很直接：兴奋性神经元向抑制性神经元送出驱动，抑制性神经元再把抑制回送给兴奋性神经元，形成一个闭环。只要外部驱动和抑制强度处在合适的平衡区间，就会出现稳定的 gamma 频段振荡；如果平衡偏移，群体同步性就会变差。官方页面把这个回路画成结构图，强调驱动输入只作用于部分兴奋性神经元，而另一部分兴奋性神经元则主要靠背景噪声和回路动力学被动参与。

先抓住 E/I 平衡

这页的关键不是“有多少个神经元”，而是“驱动、抑制和噪声如何共同决定相位锁定”。后面所有代码都在围绕这个平衡做工程化实现。

1.1 本章小结

这一讲的目标是把经典 PING 回路写成 Neuroblox 可求解的图系统。理解网络结构和 E/I 平衡，比记住任何单条代码都更重要。

2 动力学方程、细胞类型与参数

官方页在概念部分给出了 Hodgkin-Huxley 形式的膜电位方程。它虽然没有把所有细胞动力学细节都展开，但已经足够说明这不是简化到只剩阈值的玩具模型，而是保留了通道电流、门控变量和外部驱动的生物物理模型：

$$C \frac{dV}{dt} = g_{Na} m^3 h (V_{Na} - V) + g_K n^4 (V_K - V) + g_L (V_L - V) + I_{ext}.$$

这里的含义很标准： C 是膜电容， V 是膜电位， m, h, n 是门控变量， g_{Na}, g_K, g_L 是离子通道最大电导， V_{Na}, V_K, V_L 是反转电位， I_{ext} 是外部输入电流。页面明确说明，兴奋性细胞近似为 reduced Traub-Miles cell，抑制性细胞近似为 Wang-Buzsáki cell；如果要追公式细节，需要回到 Börgers et al. 的 SI Appendix 中的 Eq. 12–14。

网络规模也被固定得很明确：总共 200 个神经元，其中 40 个是 driven excitatory neurons, 120 个是 other excitatory neurons, 40 个是 inhibitory neurons。连接结构是有向图：兴奋性神经元驱动抑制性神经元，抑制性神经元再反馈抑制兴奋性神经元，同时允许自抑制，但不允许自兴奋。这个设定正是 gamma 振荡最常见的最小闭环之一。

参数为什么这样设

页面里的参数不是随便填的。 $\mu_E, \mu_I, \sigma_E, \sigma_I, g_{EI}, g_{IE}, g_{II}$ 大体上沿用或微调自原始论文的附录，而 $I_{undriven}$ 和 I_{bath} 则是为了在 Neuroblox 的实现里重新平衡 E/I 关系。换言之，这一页既在“复现”，也在“重标定”。

2.1 本章小结

PING 网络依赖的是一个带门控变量的 Hodgkin-Huxley 动力学，而不是简单的阈值放电。参数的作用是维持合适的驱动、噪声和抑制比例，从而让 gamma 振荡出现。

3 Neuroblox 实现

3.1 导入与初始化

官方 notebook 的实现从五个包开始：Neuroblox 负责对象与图，OrdinaryDiffEq 负责求解，Distributions 负责随机电流，Random 负责随机种子，CairoMakie 负责图形输出。随后先固定随机种子，再设置参数与电流分布。

```

1 using Neuroblox
2 using OrdinaryDiffEq
3 using Distributions
4 using Random
5 using CairoMakie
6
7 Random.seed!(42)
8
9 mu_E = 0.8
10 sigma_E = 0.15
11 mu_I = 0.8
12 sigma_I = 0.08
13
14 NE_driven = 40
15 NE_other = 120
16 NI_driven = 40
17 N_total = NE_driven + NE_other + NI_driven
18
19 N = N_total
20 g_II = 0.2
21 g_IE = 0.6
22 g_EI = 0.8
23
24 I_base = Normal(0, 0.1)
25 I_driveE = Normal(mu_E, sigma_E)
26 I_driveI = Normal(mu_I, sigma_I)
27 I_undriven = Normal(0, 0.4)
28 I_bath = -0.7

```

Listing 1: 导入依赖、固定随机种子并设置参数

这段初始化里最值得注意的地方有两个。第一，页面没有把所有驱动都设成完全确定的常数，而是通过 Distributions.jl 把部分电流写成随机变量，这样每个神经元的初始状态都带有轻微差异。第二，I_bath 明确引入了一个外部抑制性浴流，它对整体 E/I balance 的影响在后面的结果里会变得很明显。

3.2 定义神经元与连接图

接下来是 Neuroblox 的关键步骤：先定义神经元对象，再定义图，再让图生成 ODE 系统。官方代码里把兴奋性神经元分成 driven 和 undriven 两组，前者命名为 ED1, ED2, ..., 后者命名为 E01, E02, ...；抑制性神经元则命名为 ID1, ID2, ...。这样做的好处是，后面在 raster plot 里很容易区分不同群体。

```

1 exci_driven = [PINGNeuronExci(name=Symbol("ED$i"), I_ext=rand(I_driveE) + rand(
    I_base)) for i in 1:NE_driven]
2 exci_other  = [PINGNeuronExci(name=Symbol("E0$i"), I_ext=rand(I_base) + rand(
    I_undriven)) for i in 1:NE_other]
3 exci = [exci_driven; exci_other]
4 inhib = [PINGNeuronInhib(name=Symbol("ID$i"), I_ext=rand(I_driveI) + rand(I_base) +
    I_bath) for i in 1:NI_driven]
5
6 g = MetaDiGraph()
7 for ne in exci
8     for ni in inhib
9         add_edge!(g, ne => ni; weight=g_EI/N)
10        add_edge!(g, ni => ne; weight=g_IE/N)
11    end
12 end
13
14 for ni1 in inhib
15     for ni2 in inhib
16         add_edge!(g, ni1 => ni2; weight=g_II/N)
17     end
18 end

```

Listing 2: 定义神经元并写出图连接的基本形式

这里的图不是单纯的“连边示意”，而是真正决定后续方程系统结构的对象。`MetaDiGraph()` 先把网络拓扑装进去，再由 `system_from_graph` 把拓扑翻译成可解的动态方程。这个设计很 Neuroblox：先写对象关系，再让框架自动生成求解对象。

3.3 邻接矩阵构图与求解

官方页还给了一个替代方案：如果网络很大，可以先把节点一次性加进去，再用邻接矩阵生成边。这个版本和上面的双重循环在功能上等价，只是更适合规模更大的情形。

```

1 g = MetaDiGraph()
2 add_blox!.(Ref(g), [exci; inhib])
3 adj = zeros(N_total, N_total)
4
5 for i in 1:NE_driven + NE_other
6     for j in 1:NI_driven
7         adj[i, NE_driven + NE_other + j] = g_EI/N
8         adj[NE_driven + NE_other + j, i] = g_IE/N
9     end
10 end
11

```



```

12 for i in 1:NI_driven
13     for j in 1:NI_driven
14         adj[NE_driven + NE_other + i, NE_driven + NE_other + j] = g_II/N
15     end
16 end
17
18 create_adjacency_edges!(g, adj)
19
20 tspan = (0.0, 300.0)
21 @named sys = system_from_graph(g, graphdynamics=true)
22 prob = ODEProblem(sys, [], tspan)
23 sol = solve(prob, Tsit5(), saveat=0.1)

```

Listing 3: 用邻接矩阵一次性生成同样的图并求解

graphdynamics=true 的用途

这个选项会启用 GraphDynamics.jl 的编译路径。页面已经点明：它并不适用于每个模型，但一旦可用，通常会让大规模网络或重复调参时的编译速度更快。

这里的求解器选择也值得点出来。Tsit5() 是一个对非刚性系统很常用的高效显式求解器，官方页把它类比为 Matlab 里的 ode45；如果要更高精度，可以尝试 Vern7()。这说明作者不是只想“跑出一张图”，而是在提醒你如何在精度、速度和可复现性之间取舍。

3.4 本章小结

Neuroblox 的实现思路是“先对象，后图，最后方程”。随机电流、图结构和求解器选择共同决定了 PING 网络是否能稳定地进入 gamma 振荡区间。

4 结果解读与练习

最后一步是把解画出来。官方页用 raster plot 同时显示兴奋性和抑制性神经元，以便直接观察群体同步性和放电节律。

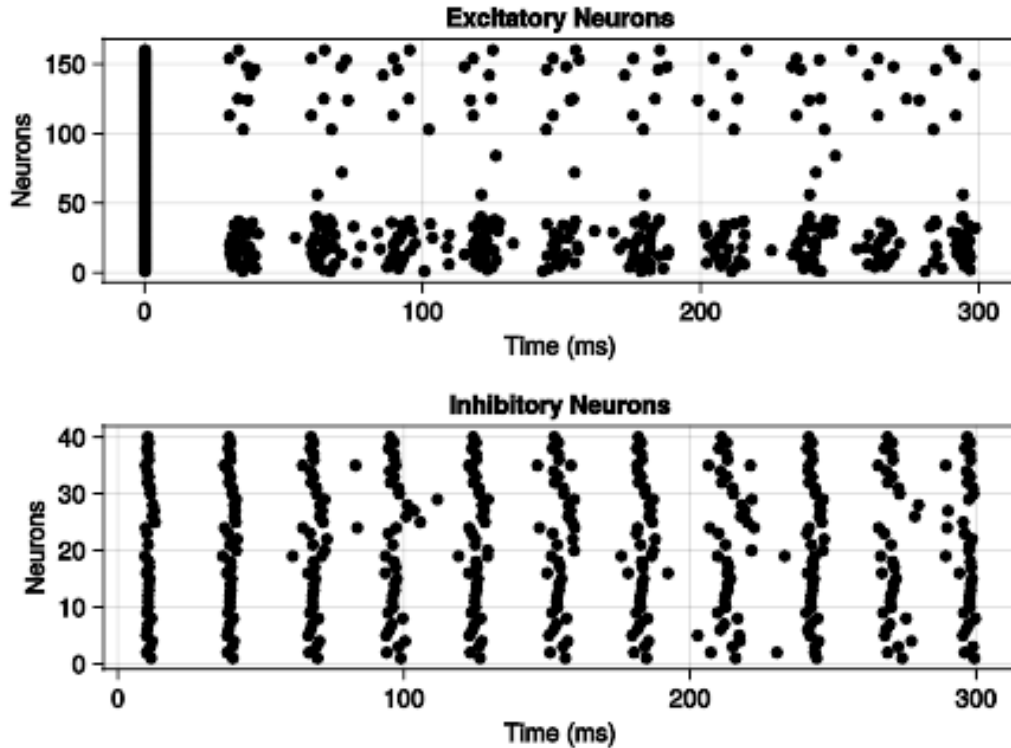


图 1: PING 网络的 raster plot。上图是兴奋性神经元，下图是抑制性神经元；它们共同展示了接近 Börgers et al. Figure 1 的群体放电节律。

从结果上看，上方面板里驱动兴奋性神经元会形成比较集中的放电簇，而未驱动的兴奋性神经元则更稀疏、更受噪声影响。下方面板的抑制性神经元则表现出更规则的同步爆发。这正是 PING 网络的典型图景：兴奋性群体先被驱动拉起，随后抑制性回路把节律重新“切”出来，形成近似 gamma 频段的群体振荡。

为什么末尾会略微去同步

官方页明确提示，模拟末端的兴奋性和抑制性群体会比原论文略微更不整齐。原因不是代码坏了，而是实现里对兴奋驱动和 inhibitory bath 的处理与原始论文略有差异，导致整体 E/I balance 发生了轻微偏移。

页面给出的练习非常有价值：可以提高抑制性浴流，或者降低被输入驱动的兴奋性神经元比例，观察同步性如何变化。这个练习的真正目的，是让你意识到 PING 不是一个“固定波形”，而是一个对输入统计和网络结构都很敏感的动力系统。

4.1 本章小结

结果图把前面的结构、参数和求解设置统一到一起：驱动兴奋性群体先被拉起，抑制性回路再塑造节律。稍微调整 E/I balance，就会显著改变同步程度。

5 总结与延伸

这一讲最重要的收获有三个。第一，PING 网络并不神秘，它本质上是一个以 E/I 回路为核心的 gamma 振荡最小模型。第二，Neuroblox 的价值在于把这个模型拆成神经元对象、连接图和求解器三个层次，分别管理生物物理、结构和数值求解。第三，所谓“复现论文图像”并不等于机械抄代码，而是要把原论文里的参数和外部驱动逻辑翻译成框架可执行的对象关系。

如果继续往下做，最直接的两个方向就是：一是系统性扫描 I_{bath} 、 g_{EI} 、 g_{IE} 和驱动比例，看看 gamma 同步区间如何移动；二是把这个最小 PING 网络放到更复杂的层级模型里，观察它如何和其他局部回路耦合。这样你就不只是“跑过一个例子”，而是真正掌握了为什么这个例子会成立。

5.1 参考文献

参考文献

- [1] Börgers C, Epstein S, Kopell NJ. *Gamma oscillations mediate stimulus competition and attentional selection in a cortical network model*. Proc Natl Acad Sci U S A. 2008;105(46):18023–18028. DOI: 10.1073/pnas.0809511105.
- [2] Traub RD, Miles R. *Neuronal Networks of the Hippocampus*. Cambridge University Press, 1991. DOI: 10.1017/CBO9780511895401.
- [3] Wang X-J, Buzsáki G. *Gamma oscillation by synaptic inhibition in a hippocampal interneuronal network model*. J Neurosci. 1996;16(20):6402–6413. DOI: 10.1523/JNEUROSCI.16-20-06402.1996.
- [4] Protter M. *GraphDynamics.jl – Efficient dynamics of interacting collections of modular subsystems* (v0.2.2). Zenodo, 2024. DOI: 10.5281/zenodo.14183153.

5.2 本章小结

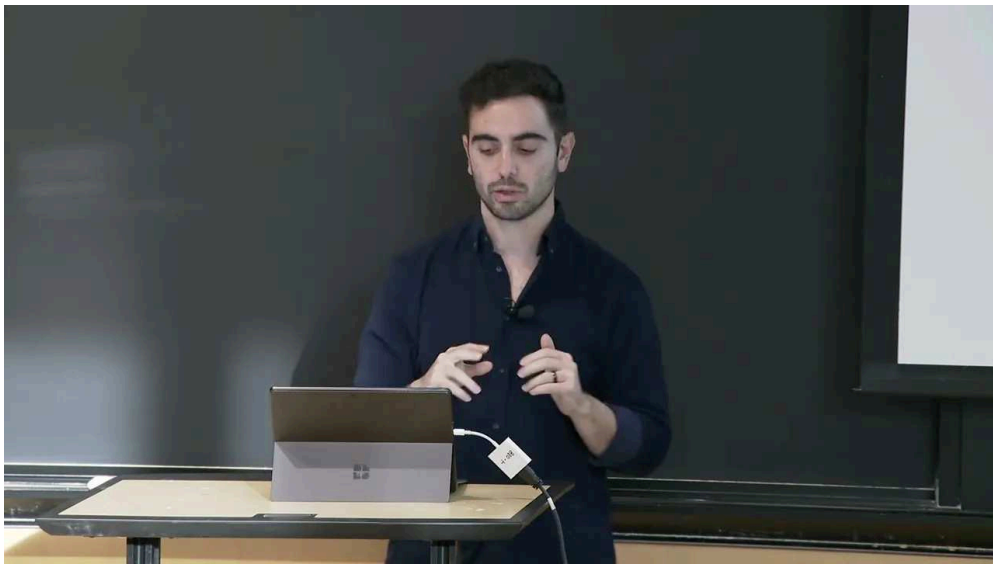
PING 讲的是一个最小但足够真实的 gamma 节律回路。把它写成 Neuroblox 代码之后，结构、参数和数值方法就可以一起被检查、复现和继续扩展。

课程笔记

MIT RES.9-009 第 10 讲：回路模型中的决策

五道口纳什 & Codex

2026 年 4 月 15 日



视频作者/频道：MIT Video Productions（讲者：Haris Organzidis）

发布日期：课程日期 2025-01-16；YouTube 上传 2025-04-10

视频时长：14:03

视频链接：<https://www.youtube.com/watch?v=ULAe2VvQgms>

目录

1 讲座定位：从塑性与强化学习转向感知决策	2
1.1 问题语义：先验类别已知，难点在于证据分辨	2
1.2 漂移-扩散模型的教材化写法	2
1.3 本章小结	3
2 随机点运动任务：用纯感知刺激驱动左/右选择	3
2.1 coherence 控制难度	3
2.2 本章小结	3
3 Wang 回路模型：把决策写成竞争网络	3
3.1 回路结构	3
3.2 为什么它像 winner-take-all	4
3.3 缩放后的参数与自由变量	4
3.4 本章小结	4
4 模型定义：显式 receptor dynamics 与电流分解	4
4.1 代码骨架：参数、输入与神经元对象	4
4.2 电压方程与 receptor 组成	6
4.3 本章小结	6
5 系统构建与仿真：从图结构到 ODEProblem	7
5.1 连接图与求解器	7
5.2 本章小结	7
6 结果解读：竞争如何在图上显现	8
6.1 Spike raster：谁先赢，谁就把对方压下去	8
6.2 Firing-rate 视图：把竞争幅度看得更清楚	9
6.3 coherence 增加时会发生什么	9
6.4 本章小结	9
7 挑战问题：把讲义变成可做实验	1
7.1 本章小结	10
8 参考文献与收束	1
8.1 本章小结	10
9 总结与延伸	11
9.1 本章小结	11

证据说明

本讲正文以 MIT Neuroblox 官方课程页 [decision_making.html](#)、YouTube 官方字幕，以及课程页中嵌入的代码与结果图为主证据。当前工作区未找到独立的官方 slide PDF，因此正文采用“课程页文字 + 字幕 + 代码 + 图像”的覆盖方式；缺失项已写入 `omission_log.jsonl`。

1 讲座定位：从塑性 with 强化学习转向感知决策

这一讲延续课程前几节“把神经动力学变成可运行回路”的路线，但对象换成了感知决策。讲者一开始就把它描述成一个更专门的应用：前面讲过 reinforcement learning 和 plasticity，这一讲则要搭建一个真正“做决策”的 circuit model，并把它放进行为任务里。

1.1 问题语义：先验类别已知，难点在于证据分辨

这里研究的不是 category learning 那种“靠反馈学类别”的问题。对于 perceptual decision making，类别通常是 *a priori* 已知的，参与者不需要先学哪一类正确，而是要在模糊刺激里把两个可能答案分开。课程强调的核心，是随着时间积累 evidence，直到足够有把握地判断刺激更像 A 还是 B。

决策任务的关键差异

感知决策里，困难主要来自刺激本身的不确定性；类别学习里，困难则更多来自“类别边界要靠经验学出来”。前者是 evidence accumulation，后者是 feedback-based learning。

1.2 漂移-扩散模型的教材化写法

讲者把 drift-diffusion family 作为这一类任务的经典起点。它的吸引力在于：当你要求模型达到某个准确率时，可以把它调成“在这个准确率下最快”的模型；反过来，在固定时间或固定证据量下，它也能被调成“最准确”的模型。这个速度-准确率折中，是整类模型最重要的原则之一。

教材化地写，最简 drift-diffusion 过程可表示为

$$dx_t = \mu dt + \sigma dW_t, \quad \tau = \inf\{t \geq 0 : x_t \in \{-B, +B\}\}. \quad (1)$$

其中：

- x_t 是随时间累积的证据变量；
- μ 是漂移项，表示证据朝某一方向的平均偏置；
- σdW_t 是扩散噪声，表示不确定性；
- B 是决策边界；
- τ 是第一次碰到边界的时间，也就是决策时刻。

如果再加上非决策时间 t_{nd} ，观测到的 response time 往往写成 $RT = \tau + t_{nd}$ 。课程里明确说，非决策时间可以吸收真实反应里和“思考”无关的部分，例如 motor execution。

1.3 本章小结

这一段先把问题的语义钉牢：这一讲不是在学一般分类器，而是在学一个带时间维度、带边界条件、带速度-准确率权衡的决策系统。后面所有回路结构，都是为这个语义服务的。

2 随机点运动任务：用纯感知刺激驱动左/右选择

课程选择 random dot motion task 作为行为范式，是因为它非常适合把“感知不确定性”和“证据积累”绑定在一起。参与者看到一个点云在 aperture 里移动，需要判断平均运动方向是 left 还是 right。

2.1 coherence 控制难度

实验者可以操控 dot coherence。coherence 为 0% 时，所有点随机运动，任务几乎只剩噪声；当 coherence 提高到 50% 时，一半 dot 朝正确方向运动，另一半随机；如果更高，方向会越来越清楚，任务也更容易。

课程里的关键判断

这类任务的难点不是“学会正确类别”，而是“在噪声中把方向差异抠出来”。因此它是检验 perceptual decision-making 的经典范式。

讲者还强调，这个任务是 purely perceptual。reward feedback 在这里不是主要难点来源，主要难点是如何从时间上逐步分辨不确定刺激。也正因为如此，它是 drift-diffusion 和后续 circuit model 的理想测试床。

2.2 本章小结

random dot motion task 把抽象的 evidence accumulation 具体化成“点云朝哪边动”的判断。coherence 越高，平均证据越清楚；coherence 越低，模型越依赖积累过程和内部竞争机制。

3 Wang 回路模型：把决策写成竞争网络

课程把重点放在 Wang 2002 的经典回路模型上。它不是纯行为学模型，而是一个明确的 circuit abstraction：两个选择对应两个 selective excitatory populations，再配一个 non-selective excitatory population 和一个 inhibitory population，让 A/B 之间发生竞争。

3.1 回路结构

这个模型可以直接翻译成四类功能单元：

- selective excitatory population A，对应一个候选类别；
- selective excitatory population B，对应另一个候选类别；
- non-selective excitatory population，作为共享兴奋性背景；
- inhibitory population，提供竞争所需的抑制回路。

每个 selective population 都接收独立的 spike train 输入: `stim_A` 与 `stim_B`。课程解释它们分别表示对 left / right dot direction 的视觉编码。与此同时, 所有 Bloxs 都收到来自未显式建模上游区域的 background Poisson spike trains。

3.2 为什么它像 winner-take-all

讲者明确指出, 这个 circuit 带有 winner-take-all 的味道: 一个群体逐渐赢得竞争, 另一个群体逐渐被压下去。与此同时, 他也提醒这并不是非常 biologically realistic 的密集连接, 而更像对生物结构的抽象。它之所以值得保留, 是因为它能抓住行为结果, 并且能把 receptor dynamics 单独拎出来研究。

3.3 缩放后的参数与自由变量

为了让缩小后的模型仍然保留原始 Wang 模型的竞争行为, 课程使用缩放因子去调整 conductance 参数:

$$s_E = \frac{1600}{N_E}, \quad s_I = \frac{400}{N_I}. \quad (2)$$

这里, N_E 与 N_I 分别是兴奋性与抑制性神经元总数。课程还给出两种突触权重的设置: $w_+ = 1.7$, 而

$$w_- = 1 - f \frac{w_+ - 1}{1 - f}, \quad (3)$$

其中 f 是 selective excitatory neurons 在兴奋性群体里的比例。这个表达式的作用, 是让缩小版电路仍保持原始模型里那种 “一个群体胜出、另一个群体被压制” 的结构。

3.4 本章小结

Wang 模型的重点不只是 “有两个选择”, 而是它把行为上的决策竞争写成了一个明确的网络竞争问题。缩放、输入和权重设置共同决定了这个竞争能否在缩小模型里继续成立。

4 模型定义: 显式 receptor dynamics 与电流分解

这一讲的模型定义部分, 真正值得记住的不是某个单独参数, 而是它的建模层次: 行为任务先被翻译成输入流, 再被翻译成带 receptor dynamics 的 LIF population, 最后再拼成图结构。

4.1 代码骨架: 参数、输入与神经元对象

下面保留课程页里的关键代码骨架, 帮助你把 “文字描述” 对上 “可执行对象”。

```

1 using Neuroblox
2 using OrdinaryDiffEq
3 using Distributions
4 using CairoMakie
5 using Random
6
7 Random.seed!(1)
8
9 model_name = :g
10 tspan = (0, 1000)  ## Simulation time span [ms]
```



```

11 spike_rate = 2.4    ## spikes / ms
12
13 N = 150              ## total number of neurons
14 f = 0.15             ## ratio of selective excitatory to non-selective excitatory
                        neurons
15 f_inh = 0.2          ## ratio of inhibitory neurons to all neurons
16 N_E = Int(N * (1 - f_inh))
17 N_I = Int(ceil(N * f_inh))    ## total number of inhibitory neurons
18 N_E_selective = Int(ceil(f * N_E))
19 N_E_nonselective = N_E - 2 * N_E_selective
20
21 # We use two distinct weight values as per Wang [1]
22 w_plus = 1.7
23 w_minus = 1 - f * (w_plus - 1) / (1 - f)

```

Listing 1: 模型参数、随机种子与输入对象

这段代码最重要的不是变量名，而是它把“尺度缩放”和“结构比例”显式写出来：总神经元数、选择性兴奋性比例、抑制性比例、以及从原始 Wang 模型抽取出来的两类权重。

$$w_- = 0.8764705882352941 \quad (\text{在课程参数下的实际数值}) \quad (4)$$

```

1  exci_scaling_factor = 1600 / N_E
2  inh_scaling_factor = 400 / N_I
3
4  coherence = 0          # random dot motion coherence [%]
5  dt_spike_rate = 50     # update interval for the stimulus spike rate [ms]
6
7  mu_0 = 40e-3           # mean stimulus spike rate [spikes / ms]
8  rho_A = rho_B = mu_0 / 100
9
10 mu_A = mu_0 + rho_A * coherence
11 mu_B = mu_0 - rho_B * coherence
12 sigma = 4e-3           # standard deviation of stimulus spike rate [spikes /
                        ms]
13
14 spike_rate_A = (distribution=Normal(mu_A, sigma), dt=dt_spike_rate)
15 spike_rate_B = (distribution=Normal(mu_B, sigma), dt=dt_spike_rate)
16
17 @named background_input = PoissonSpikeTrain(spike_rate, tspan; namespace =
                        model_name)
18 @named stim_A = PoissonSpikeTrain(spike_rate_A, tspan; namespace = model_name)
19 @named stim_B = PoissonSpikeTrain(spike_rate_B, tspan; namespace = model_name)
20
21 @named n_A = LIFExciCircuitBlox(; namespace = model_name, N_neurons = N_E_selective
                        , weight = w_plus, exci_scaling_factor, inh_scaling_factor)
22 @named n_B = LIFExciCircuitBlox(; namespace = model_name, N_neurons = N_E_selective
                        , weight = w_plus, exci_scaling_factor, inh_scaling_factor)
23 @named n_ns = LIFExciCircuitBlox(; namespace = model_name, N_neurons =
                        N_E_nonselective, weight = 1.0, exci_scaling_factor, inh_scaling_factor)

```

```
24 @named n_inh = LIFInhCircuitBlox(; namespace = model_name, N_neurons = N_I, weight
    = 1.0, exci_scaling_factor, inh_scaling_factor)
```

Listing 2: 缩放因子、随机 dot coherence 与神经元块

这里三个值得直接对照的点。第一，`stim_A` 和 `stim_B` 的均值随 coherence 分离，正好把刺激不确定性编码进输入。第二，background input 让未显式建模的上游区域以 Poisson spike trains 的方式进入系统。第三，课程用 `LIFExciCircuitBlox` 与 `LIFInhCircuitBlox` 直接表达“一个选择性兴奋性群体 / 一个抑制性群体”的语义。

4.2 电压方程与 receptor 组成

讲者在字幕里说，他想展示的是“model equations”，重点是这套模型显式包含 *receptor dynamics*。为了把这个口头说明整理成教材化表达，可以写成

$$C_m \frac{dV}{dt} = -g_L(V - E_L) - I_{\text{syn}}. \quad (5)$$

其中 synaptic current 可以按课程叙述拆成

$$I_{\text{syn}} = I_{\text{ext}}^{\text{AMPA}} + I_{\text{rec}}^{\text{AMPA}} + I_{\text{rec}}^{\text{NMDA}} + I^{\text{GABA}}. \quad (6)$$

对应的符号含义如下：

- V 是膜电位；
- C_m 是膜电容；
- g_L 和 E_L 分别是漏电导与漏电反转电位；
- $I_{\text{ext}}^{\text{AMPA}}$ 表示代表视觉刺激的外部 AMPA 电流；
- $I_{\text{rec}}^{\text{AMPA}}$ 与 $I_{\text{rec}}^{\text{NMDA}}$ 表示来自其他兴奋性神经元的 recurrent excitatory currents；
- I^{GABA} 表示来自抑制性中间神经元的抑制电流。

课程特别强调，NMDA 的动力学比 AMPA 和 GABA 更复杂，但这种复杂性是值得保留的，因为它让模型能够显式研究不同 receptor 对证据积累的作用。

为什么这个模型虽然粗糙却仍然有价值

讲者直说：这个网络不算非常 biologically realistic，尤其因为它的连接是 dense 的。不过它的价值在于，它能把行为层面的竞争结果和 receptor-level dynamics 绑在一起，从而让“谁在起作用”这件事变得可测。

4.3 本章小结

这一节完成了从任务到方程的翻译：随机点运动的输入被压缩成 selective Poisson spike trains，回路动力学被展开成包含 AMPA、NMDA 和 GABA 的 LIF 方程。模型的抽象程度不低，但它保留了最重要的可解释层次。

5 系统构建与仿真：从图结构到 ODEProblem

模型写出来后，课程把关键动作交给图结构和数值求解器。这里的重点不是“把代码抄一遍”，而是看清楚 graph 先于 system、system 先于 solver 的建模顺序。

5.1 连接图与求解器

```

1 g = MetaDiGraph()
2 add_edge!(g, background_input => n_A; weight = 1)
3 add_edge!(g, background_input => n_B; weight = 1)
4 add_edge!(g, background_input => n_ns; weight = 1)
5 add_edge!(g, background_input => n_inh; weight = 1)
6
7 add_edge!(g, stim_A => n_A; weight = 1)
8 add_edge!(g, stim_B => n_B; weight = 1)
9
10 add_edge!(g, n_A => n_B; weight = w_minus)
11 add_edge!(g, n_A => n_ns; weight = 1)
12 add_edge!(g, n_A => n_inh; weight = 1)
13
14 add_edge!(g, n_B => n_A; weight = w_minus)
15 add_edge!(g, n_B => n_ns; weight = 1)
16 add_edge!(g, n_B => n_inh; weight = 1)
17
18 add_edge!(g, n_ns => n_A; weight = w_minus)
19 add_edge!(g, n_ns => n_B; weight = w_minus)
20 add_edge!(g, n_ns => n_inh; weight = 1)
21
22 add_edge!(g, n_inh => n_A; weight = 1)
23 add_edge!(g, n_inh => n_B; weight = 1)
24 add_edge!(g, n_inh => n_ns; weight = 1)
25
26 sys = system_from_graph(g; name=model_name, graphdynamics = true)
27 prob = ODEProblem(sys, [], tspan)
28 sol = solve(prob, Euler(); dt = 0.01)

```

Listing 3: 图结构、系统构造与数值求解

课程页特别提醒，`system_from_graph` 和后续 `ODEProblem` 的构造会花几分钟，因为系统规模比前面的例子大得多。这里打开 `graphdynamics=true`，是为了启用更快的编译路径；讲者还补充说，并不是所有模型都兼容 `GraphDynamics.jl`，但兼容时通常能显著加速。

5.2 本章小结

图结构把连接关系从方程里剥离出来，先让“谁连谁”变成显式对象；然后求解器再把这个图变成可运行动力学。对于大型回路，这种建模顺序比手工写方程更清晰，也更便于调参。

6 结果解读：竞争如何在图上显现

课程的结果部分非常直接：给定某个 coherence 和输入设置，A/B 两个 selective excitatory populations 会表现出互相竞争的行为。一个群体上升，另一个群体下降，抑制群体维持较稳定的 tonic activity。

6.1 Spike raster：谁先赢，谁就把对方压下去

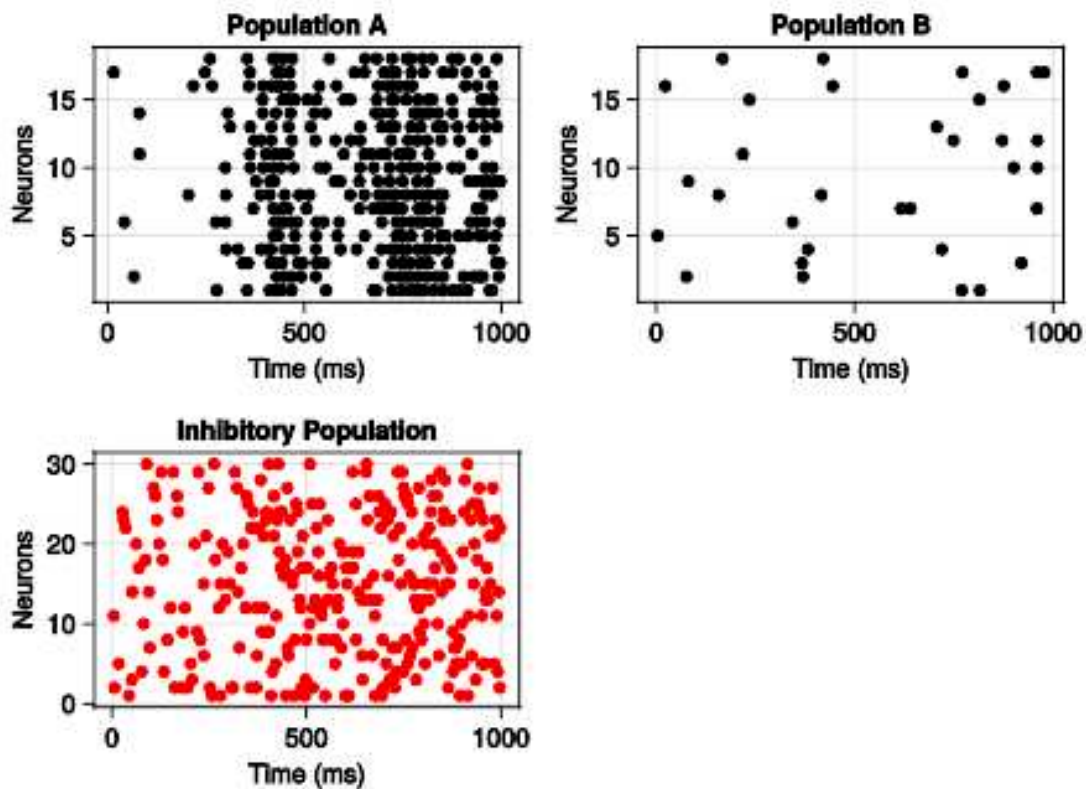


图 1: Population A、Population B 与 inhibitory population 的 spike raster。图源：MIT 官方课程页。

这个图最重要的不是“看到了 spike”，而是看到了时间上的竞争逻辑。课程字幕明确说，在一个 excitatory population 迅速 ramp up 的同时，另一个 population 会同步下降；抑制群体则提供维持竞争的 tonic activity。也就是说，胜负不是只看最终谁更强，而是看竞争过程中的时间轨迹。

6.2 Firing-rate 视图：把竞争幅度看得更清楚

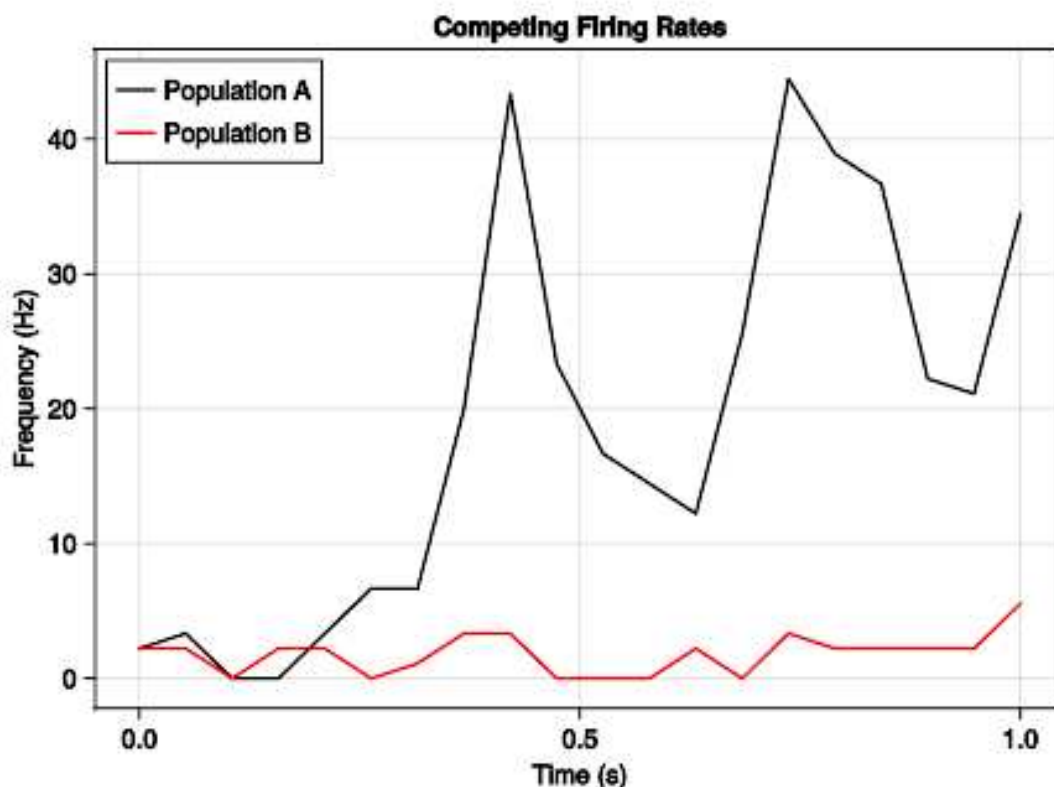


图 2: Population A 与 Population B 的 firing rate 对比。图源：MIT 官方课程页。

第二张图把同样的竞争过程改写成 firing-rate 轨迹。课程强调，单轴图更容易看清两群体之间的差距，因此更适合观察 competition magnitude。和 raster 相比，firing rate 图更适合看“谁赢得更多”，而 raster 更适合看“谁在什么时刻开始压制对方”。

6.3 coherence 增加时会发生什么

在 0% coherence 时，两条输入电流分布高度重叠，因此 A/B 的初期活动也很相似；到 6% 或更高 coherence 时，两类电流的均值差逐渐增大，分布分离得更明显，决策也更容易。课程最后还提到，论文里可以进一步得到 response time distributions 和 psychophysical curves：前者是自由反应任务里非常常见的指标，后者则说明 accuracy 会随 coherence 改变而形成典型的 sigmoid-like 曲线。

6.4 本章小结

结果部分验证了前面的建模目标：这个回路不是只会“有活动”，而是真的会产生竞争和胜负分化。coherence 越高，胜负越早分出；coherence 越低，竞争越接近随机过程。

7 挑战问题：把讲义变成可做实验

课程最后没有停在“看懂结果”上，而是直接给了两个开放式练习。这两个问题其实就是把 lecture 变成 notebook 实验的入口。

- **响应时间分布：**电路是 real-time decision model。你需要记录何时到达决策边界，再把这些时刻整理成 response time distribution。进一步可以改变 dot coherence，观察 RT 分布如何整体前移或后移。
- **受体消融：**比较 NMDA、AMPA 与 GABA 哪一种 receptor 对竞争行为最关键。课程提示你去看 LIFExcNeuron 和 LIFInhNeuron 的方程，并直接改 receptor conductance 做 intervention。

实现提示

这两个练习的核心不是“猜答案”，而是让你把 circuit 的可解释组件拆开：速度-准确率关系靠什么维持，竞争行为靠哪类 receptor 撑住，哪些改动会让模型从竞争状态退回到平庸状态。

课程页还把这些练习和论文里的更多结果连在一起：不同 coherence 下的 RT 统计、accuracy 曲线，以及你对受体 conductance 的干预，都是可以继续复现实验的方向。

7.1 本章小结

挑战题的意义在于把 lecture 变成一个“可以动手验证”的模型。真正值得做的是比较干预前后的曲线，而不是只看单次仿真。

8 参考文献与收束

官方课程页列出的两篇参考文献都很关键。Wang 2002 是这讲的主理论来源，而 GraphDynamics.jl 则解释了为什么课程能把较大的回路在可接受时间内编译和仿真出来。

- Wang XJ. *Probabilistic decision making by slow reverberation in cortical circuits*. Neuron. 2002 Dec;36(5):955–968. DOI: 10.1016/S0896-6273(02)01092-9. PMID: 12467598.
- Protter, M. (2024). *GraphDynamics.jl – Efficient dynamics of interacting collections of modular subsystems* (v0.2.2). Zenodo. DOI: 10.5281/zenodo.14183153.

讲者在结束时提醒大家，到 course 页的决策页面打开 `decision_making.ipynb`，再去做题和改参数。这个收尾很典型：不是把 lecture 当成一个终点，而是把它当成 notebook 实验的起点。

8.1 本章小结

这一讲把 perceptual decision making 变成了一个 receptor-explicit circuit model：先用 drift-diffusion 的语义理解速度-准确率权衡，再用 Wang 式竞争回路和显式 receptor dynamics 复现行为结果。下一步最自然的工作，就是在 notebook 里改变 coherence、边界、以及 NMDA/AMPA/GABA 的 conductance，看看 decision behavior 如何变化。

9 总结与延伸

从教材视角看，这一讲最重要的贡献，是把“决策”这件事拆成了三个可分别理解、也可重新组合的层面：行为层的 drift-diffusion intuition，回路层的 recurrent competition，以及实现层的 receptor-specific conductance 与图结构求解。它说明一个好的计算神经科学模型，不只是把行为曲线拟合出来，还要保留足够清晰的机制接口，让你能回答“为什么这个群体赢了”“为什么 coherence 会改变 response time”“为什么改一个 receptor 就会改整个竞争格局”。

继续往后做时，最值得扩展的方向有三个。第一，把自由反应范式里的 response time distribution 真正算出来，并与不同 coherence 条件下的准确率一起看。第二，系统地干预 NMDA、AMPA 与 GABA conductance，比较它们对 winner-take-all dynamics 的影响。第三，把这讲的竞争回路与课程后面出现的 plasticity、parameter fitting 和 causal modeling 连接起来，看看哪些行为差异可以被解释为参数改变，哪些则必须回到回路结构本身。

9.1 本章小结

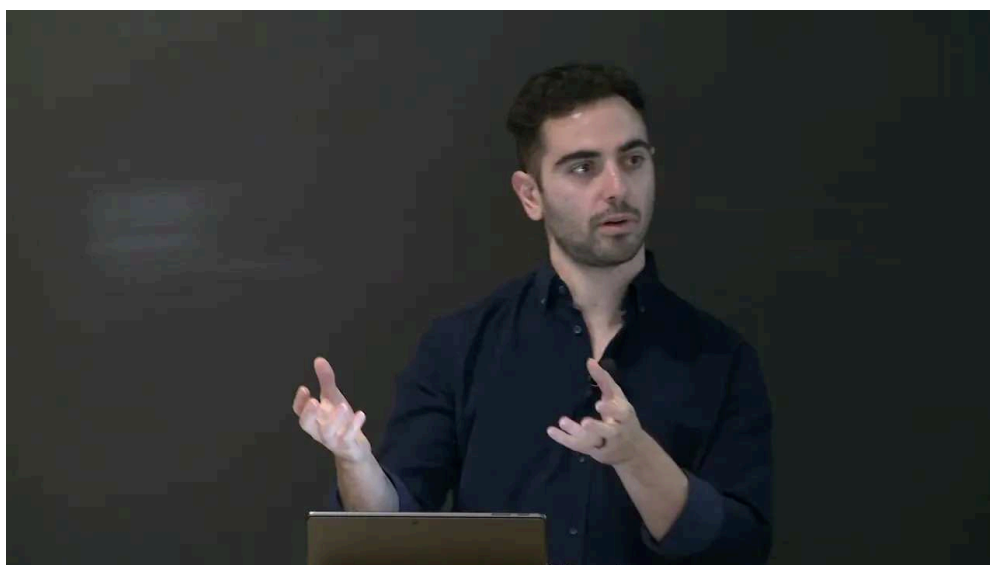
这一讲的真正价值，不在于给出一个“会选 A 还是 B”的黑箱，而在于给出一个能够把速度-准确率权衡映射到可解释神经回路上的工作范式。这也是 Neuroblox 课程整体的一条主线：用模块化、可组合的建模方式，把神经动力学、行为任务和可操作的实验假设接起来。

课程笔记

MIT RES.9-009 第 11 讲: Synaptic Plasticity and Reinforcement Learning

五道口纳什 & Codex

2026 年 4 月 15 日



视频作者/频道: MIT Video Productions (讲者: Haris Organzidis)

发布日期: 课程日期 2025-01-16; YouTube 上传 2025-04-10

视频时长: 10:52

视频链接: <https://www.youtube.com/watch?v=pBvgcIHK6GY>

目录

证据说明

本讲正文优先依据 MIT OCW 官方课程页 `learning.html`、YouTube 官方字幕与课程 `notebook` 的代码块摘要。课程页中存在的商业许可提示、以及无法获得的官方 `slides PDF`，不作为教学正文展开；相关遗漏记录在 `omission_log.jsonl` 中。

1 突触可塑性：从生物学直觉到学习规则

这讲的核心不是先讲“强化学习算法”，而是先把学习放回到突触层面：神经元之间的连接不是静态常数，而是会随着活动历史改变的参数。课程从最基础的生理直觉切入：**potentiation** 让连接更容易传递 spike，**depression** 则让连接变弱。这个变化可以发生在毫秒级，也可以跨越更长时间尺度，因此它既能解释快速适应，也能解释更慢的经验累积。

1.1 Hebbian 规则的最小形式

课程给出的经典入口是 Hebb 的命题：*neurons that fire together wire together*。如果两个通过同一 synapse 相连的神经元在时间上高度相关，那么这个 synapse 倾向于被强化。作为教材化写法，可以把最小的 Hebbian 更新概括为

$$\Delta w_{ij}(t) \propto s_i(t) s_j(t), \quad (1)$$

其中 s_i 与 s_j 分别表示 pre-synaptic 和 post-synaptic activity。这个式子不是课程里逐字写出的唯一公式，但它准确对应了课程对“同放电、同强化”的结构化解释。

关键理解

Hebbian learning 不是 reward learning。它只要求活动相关性，不要求外部奖励信号。因此它更像“局部统计规律”，而不是“任务目标导向”的优化器。

1.2 为什么突触可塑性会进入计算神经科学

课程强调，在建模时我们不只是想得到一个能跑的 ODE 系统，而是想要一个能够被数据检验的 dynamical system。也就是说，模型中的参数必须允许我们从观测信号反推出系统结构，或至少区分竞争性假设。突触可塑性之所以重要，是因为它把“连接强弱随时间变化”这件事直接变成了可计算、可验证的动力学。

教材化重写的边界

下面的更新式和解释使用了课程语义与标准计算神经科学记号做统一表达，便于自学；它们不是视频里逐行出现的唯一公式写法。

1.3 本章小结

这一部分建立了两件事：第一，synaptic plasticity 是学习的生物学载体；第二，Hebbian 规则提供了最基本的“活动相关性 $\rightarrow$ 权重更新”机制。后续章节所有 reinforcement learning 和 reward-modulated learning 的理解，都依赖这个局部更新直觉。

2 类别学习：从 cortico-cortical LTP 看到任务结构

课程把抽象的可塑性规则放入一个具体行为任务：category learning。动物看到一个 dot pattern，需要判断它属于 left 还是 right 类别。这个实验的关键并不只是“分类正确率”，而是它迫使模型在 trial-by-trial 的反馈中逐步形成稳定的 decision boundary。

2.1 任务语义与模型结构

官方讲述把这个任务翻译成了一个具身的 circuit：视觉输入进入 visual cortex，随后驱动 cortex-to-cortex 与 cortex-to-striatum 的连接，再由 action selection 形成左/右选择，最后通过 reward / no reward 反馈回到学习规则里。换句话说，模型不是纯抽象分类器，而是一个带有神经回路语义的 agent-environment system。

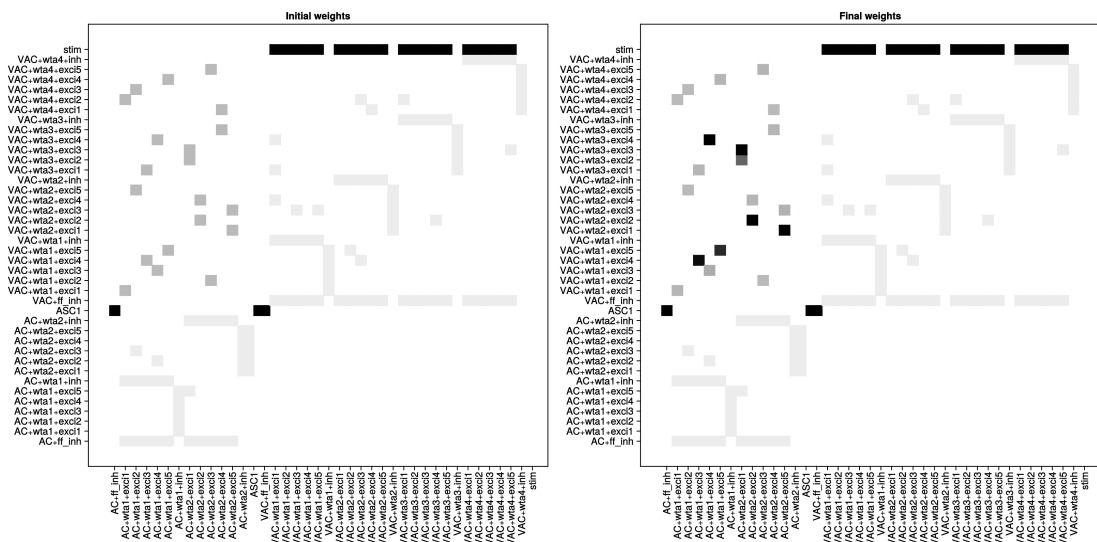


图 1: 皮层-皮层学习前后的连接矩阵。左图为 initial weights，右图为 final weights。图源：MIT Neuroblox 官方课程页。

2.2 cortico-cortical LTP

课程指出，cortico-cortical synapses 中的 LTP 属于典型 Hebbian learning：pre-synaptic 和 post-synaptic activity 越相关，权重越倾向上升。更重要的是，这里并不只是“相关就增强”，而是有一个与 dopamine 相关的反转项，使得奖励结果可以在试次级别上改变权重走向。教材化地写成

$$\Delta w_{ij}^{cc}(t) = \eta s_i(t) s_j(t) (1 + \gamma \delta_t), \quad \delta_t = r_t - \hat{r}_t, \quad (2)$$

会更便于记忆。这里的 δ_t 就是 reward prediction error。注意，这个表达是标准化的教学重写；课程里对它的口头表述是“dopamine 相关的 reversal term”和“actual reward minus expected reward”。

奖励误差的角色

在课程语境中，dopamine 不是单纯的“奖励多一点/少一点”的口号，而是把行为反馈转成 synaptic update direction 的调制信号。正奖励可以维持或增强 potentiation, reward omission 或 punishment 则可能把权重推向相反方向。

2.3 本章小结

这一部分的重点是：课程把 Hebbian plasticity 放进具体的 category-learning 任务中，说明“局部相关性”可以通过 reward gating 变成“任务导向的学习”。图上的 initial/final weights 不是装饰，而是在显示回路如何从稀疏、未定向的连接走向能支撑行为的结构。

3 cortico-striatal 回路：奖励调制的强化学习

如果说前一节还停留在 cortex-to-cortex 的局部规则，那么这里就进入完整的 reinforcement learning 语义：视觉输入到 cortex，随后到 striatum，再到 action，再回到 reward feedback。课程把这个过程与 macaque 的 category learning 文献直接对齐，因此它并不是“任意设计的 toy problem”，而是一个生物学动机很强的 behavioral task。

3.1 强化学习的回路映射

在这个模型里，Agent 由图结构构成；Environment 则是 ClassificationEnvironment，里面包含多个刺激、每个刺激的真实类别，以及每个 trial 的结果。策略部分使用 greedy policy：比较 STR1 与 STR2 的活动，选择更高者所对应的行动。这个设计把抽象 RL 语义映射成了 Neuroblox 的具体对象。

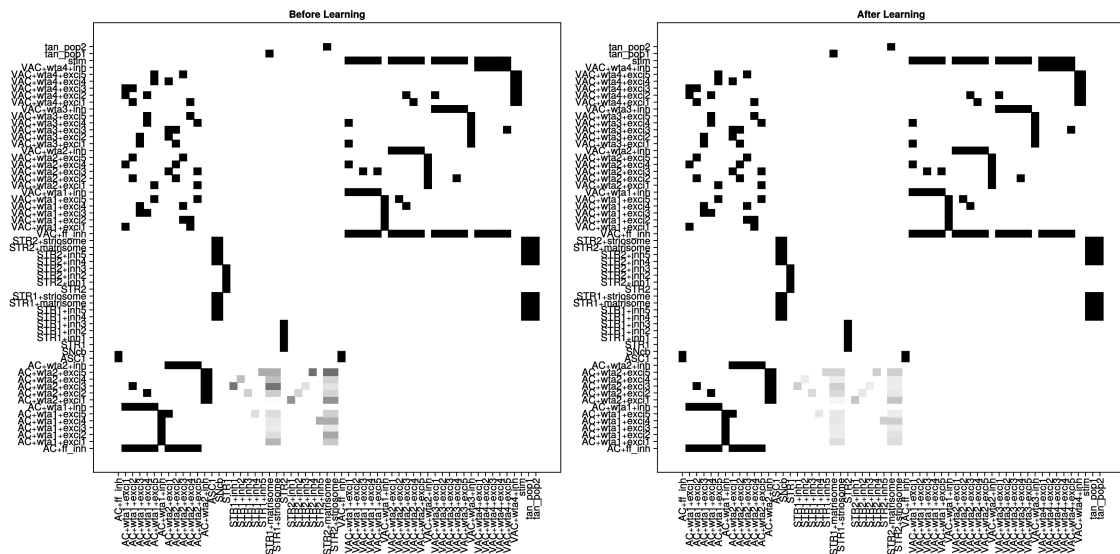


图 2: 完整 corticostriatal RL 模型的初始与学习后连接矩阵。左图为 Before Learning, 右图为 After Learning。图源：MIT Neuroblox 官方课程页。

3.2 奖励误差与 dopamine

课程里最重要的桥梁，是把 reward prediction error 与 dopamine 调制联系起来。讲者明确说明， ϵ 表示 actual reward 与 expected reward 的差，并把它与 Q-learning、temporal difference learning 的误差项并列起来看。用标准 RL 记号重写，可以写成

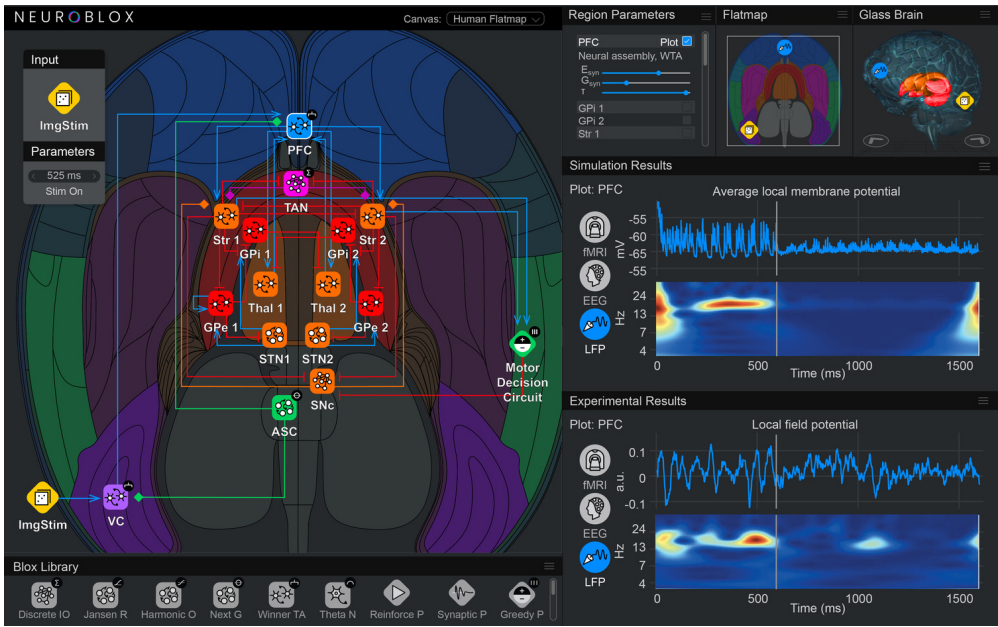
$$\epsilon_t = r_t - \mathbb{E}[r_t], \quad \Delta w_{ij}(t) \propto \epsilon_t s_i(t) s_j(t). \quad (3)$$

课程笔记

MIT RES.9-009 第 12 讲: Parameter Fitting using Optimization

五道口纳什 & Codex

2026 年 4 月 16 日



视频作者/频道: MIT Neuroblox 官方课程页

发布日期: 官方课程页最后修改 2025-07-22

视频时长: 无嵌入视频

视频链接: MIT course page

目录

1 逆问题：参数拟合到底在解决什么 2

1.1 本章小结 2

2 模型定义：四个耦合参数和带噪观测 2

2.1 本章小结 3

3 初始猜测：让误差先暴露出来 3

3.1 本章小结 4

4 用优化求解参数：损失函数、有限差分与 LBFGS 4

4.1 本章小结 5

5 结果：参数回收与轨迹对齐 5

5.1 本章小结 6

6 从 synthetic recover 到 real-data work ow: 这一讲真正教会了什么 7

6.1 本章小结 7

7 参考文献 7

7.1 本章小结 8

7.2 拓展阅读 8

8 总结与延伸 8

证据说明

本讲没有官方嵌入视频，教学内容主要来自 MIT Neuroblox 官方课程页 [optimization](#) 及其结构化页面提取。官方 slide PDF 目前仍未找到，因此没有把缺失的 PDF 当作正文内容展开；这类缺口已经记录在 `omission_log.jsonl` 中。正文中的两张图都来自官方页面资产，而不是额外生成的示意图。

1 逆问题：参数拟合到底在解决什么

课程先从一个很明确的对比开始：我们前面的工作主要是在解 **forward problem**，也就是给定参数以后去构造并模拟一个微分方程系统；这一讲则转向 **inverse problem**，也就是从数据反推模型参数。换句话说，前者回答“这个系统会怎样演化”，后者回答“什么参数能让模型尽量复现观测数据”。

用数学语言说，参数拟合要找的是

$$\mathbf{p}^* = \arg \min_{\mathbf{p}} \mathcal{L}(\mathbf{p}; \mathcal{D}), \quad (1)$$

其中 $\mathcal{D}$ 是观测数据， $\mathbf{p}$ 是待估计的参数向量， $\mathcal{L}$ 是衡量模拟轨迹和数据差异的损失函数。

这里最重要的教学点是：课程并没有直接上真实数据，而是先建议用 *fictive data*。做法是先用一个已知参数的模型生成假数据，再尝试把这些参数“找回来”。这一步的意义不是为了省事，而是为了检验模型和数据是否足以支持参数回收。如果连假数据都拟合不回来，那真实数据上的拟合就更没有把握。

为什么要先用假数据

假数据拟合本质上是在做可辨识性检查。它能告诉我们当前模型、观测变量和时间采样方式是否足以把参数区分出来，而不只是“让优化器跑完”。

课程还补了一句：如果换成真实数据， workflow 几乎不变，只是把“先生成数据”改成“从文件读取数据”，例如用 `CSV.jl` 和 `DataFrames.jl`。

1.1 本章小结

这一段建立了全讲的主线：参数拟合不是单纯求一个最优数值，而是在检验模型是否有能力从数据中恢复机制参数。假数据只是方法论上的第一道门槛。

2 模型定义：四个耦合参数和带噪观测

这一讲使用的是一个 **next generation neural mass model**，也就是前面已经见过的兴奋-抑制平衡模型。课程把四个耦合系数当作未知参数来拟合：

$$k_{ee}, \quad k_{ii}, \quad k_{ei}, \quad k_{ie}.$$

它们分别对应兴奋-兴奋、抑制-抑制、兴奋-抑制和抑制-兴奋四种耦合方向。课程用的 ground truth 是

$$\mathbf{p}_{\text{gt}} = [2, 1.5, 5.5, 7].$$

为了让例子更接近真实问题，课程在假数据上加入了观测噪声。代码里用的是均值为 0、标准差为 0.1 的高斯噪声，也就是

$$\varepsilon_{t,s} \sim \mathcal{N}(0, 0.1^2), \quad x_{t,s}^{\text{data}} = x_{t,s}^{\text{sim}} + \varepsilon_{t,s}. \quad (2)$$

这里：

- t 表示时间点。
- s 表示状态变量。
- $x_{t,s}^{\text{sim}}$ 是无噪声模拟轨迹。
- $x_{t,s}^{\text{data}}$ 是加噪后的观测数据。

代码结构也很清楚。课程先用 `Random.seed!(1)` 固定随机种子，确保例子可复现；然后定义时间区间 `tspan = (0, 100)`，并把保存点设成整数时间步 `0:100`。接着构造 `ODEProblem`，用 `Tsit5()` 先把 `ground truth` 参数对应的轨迹解出来，再把噪声加到结果上，形成后续拟合的目标。

这一段真正做了什么

模型定义的目标不是“把模型写出来”这么简单，而是先制造一个可控的逆问题环境：参数已知、轨迹可生成、噪声可注入，这样后面才有资格谈拟合是否成功。

2.1 本章小结

这一段把拟合问题具体化成一个可执行的实验：四个耦合参数、一个确定的时间窗、一个带噪观测数据集。后续所有优化步骤都只是在这个环境里更新参数而已。

3 初始猜测：让误差先暴露出来

课程接下来强调了一件很实在的事：大多数优化方法都需要一个 **initial guess**。这里给出的初值是

$$\mathbf{p}_0 = [0.2, 3.3, 2, 3.5].$$

这个起点和 `ground truth` 相比差得并不小，这正是教学上想要的效果，因为它能把“未优化时的偏差”直接展示出来。

课程用了一个 `setp setter`，把系统里的四个待拟合参数绑定起来，这样优化过程中每换一个参数向量，就能立刻更新 `ODEProblem`。随后重新求解模型，并把数据轨迹和初始猜测下的模拟轨迹画在同一张图里。

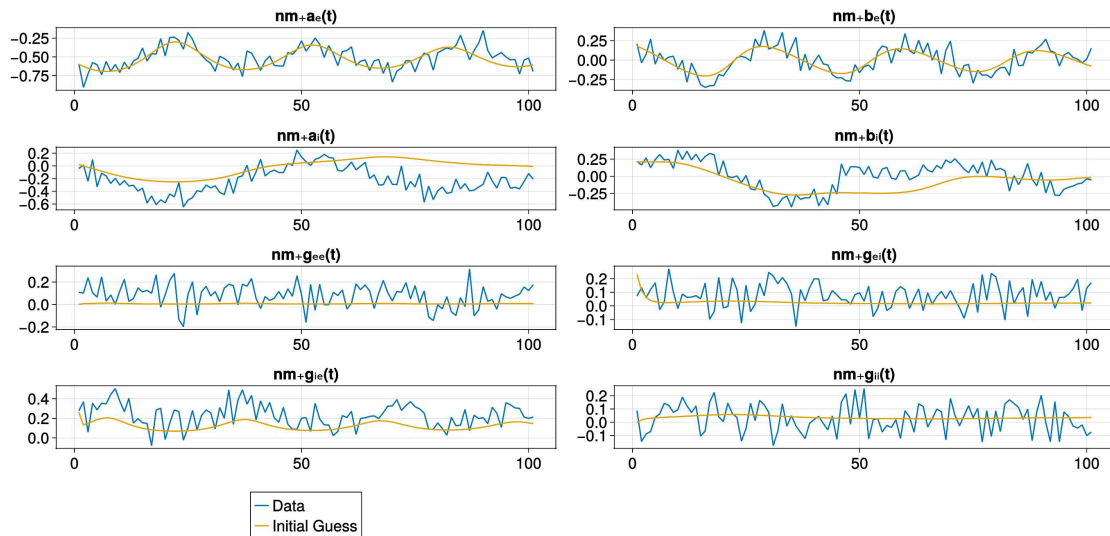


图 1: 初始猜测参数下的状态轨迹与观测数据对比, 显示未优化时偏差明显。图源: MIT Neuroblox 官方课程页 [optimization](#)。

这张图的作用不是“好看”，而是提醒我们：如果初始猜测离正确区域太远，优化器很可能要花更久时间，甚至落到不理想的局部区域。对这类 inverse problem 来说，起点本身就是实验设计的一部分。

为什么初始猜测重要

优化器不是读心术。它只能沿着损失函数的地形往下走，因此起点决定了搜索从哪里开始，也决定了我们看到的是哪一块参数空间。

3.1 本章小结

初始猜测这一步的价值，在于把“模型和数据目前差多远”直接显影出来。它既是优化的起点，也是判断问题难度的第一张诊断图。

4 用优化求解参数：损失函数、有限差分与 LBFGS

课程的核心是把参数拟合写成一个标准优化问题。最小化的目标是 least squares loss，也就是让模拟轨迹尽量贴近噪声观测：

$$\mathcal{L}(\mathbf{p}) = \sum_{t,s} (x_{t,s}^{\text{sim}}(\mathbf{p}) - x_{t,s}^{\text{data}})^2. \quad (3)$$

这里：

- $\mathbf{p}$ 是当前候选参数。
- $x_{t,s}^{\text{sim}}(\mathbf{p})$ 表示用参数 $\mathbf{p}$ 重新求解模型得到的轨迹。
- $x_{t,s}^{\text{data}}$ 是加噪后的目标数据。
- 求和覆盖所有时间点和所有状态。

这也是代码中 `sum(abs2, sol .- data)` 的含义：把每个时刻、每个状态的残差平方累加起来。损失函数前面那层 `loss(p, data, prob)` 的作用，则是把候选参数写回 `ODEProblem`，再调用 `solve(prob, Tsit5())` 生成当前轨迹。

优化管线的四个关键动作

- `setter!(prob, p)`：把候选参数写回模型。
- `solve(prob, Tsit5())`：得到当前参数下的模拟轨迹。
- `OptimizationFunction(..., AutoFiniteDiff())`：用自动有限差分近似梯度。
- `solve(prob_opt, Optimization.LBFGS())`：用 LBFGS 搜索更优参数。

这里的 `AutoFiniteDiff()` 很关键。它说明课程没有手工推导梯度，而是用数值有限差分来近似目标函数的梯度。对这个例子来说，这很合理，因为每次目标函数评估都要重新求解一个 ODE 系统，手工推导梯度既繁琐也不必要。

LBFGS 的全称是 limited-memory BFGS。课程把它称为 quasi-Newton method，是因为它不直接计算 Hessian，而是利用梯度历史去近似 Hessian 的逆。直观上讲，它比纯梯度下降更会“看地形”，但又比完整牛顿法更省内存，适合这种中等维度、光滑的参数拟合问题。

代码里还有一个小但重要的检查：

```
res.retcode = SciMLBase.ReturnCode.Success.
```

这只是说明优化过程正常结束，不代表参数一定是全球最优；它的意义是先确认数值求解没有报错。

4.1 本章小结

这一段把参数拟合正式落成一个优化问题。真正的逻辑链条是：候选参数 → 重新模拟 → 计算残差 → 数值梯度 → LBFGS 更新参数。

5 结果：参数回收与轨迹对齐

课程给出的优化结果有两层验证。第一层是直接比较参数值，第二层是把优化后的参数再放回模型里，重新看轨迹是否更接近数据。课程强调的重点是第二层，因为参数不必逐项完全相等，只要生成的动力学轨迹足够接近观测，就说明拟合在模型语义上已经有意义。

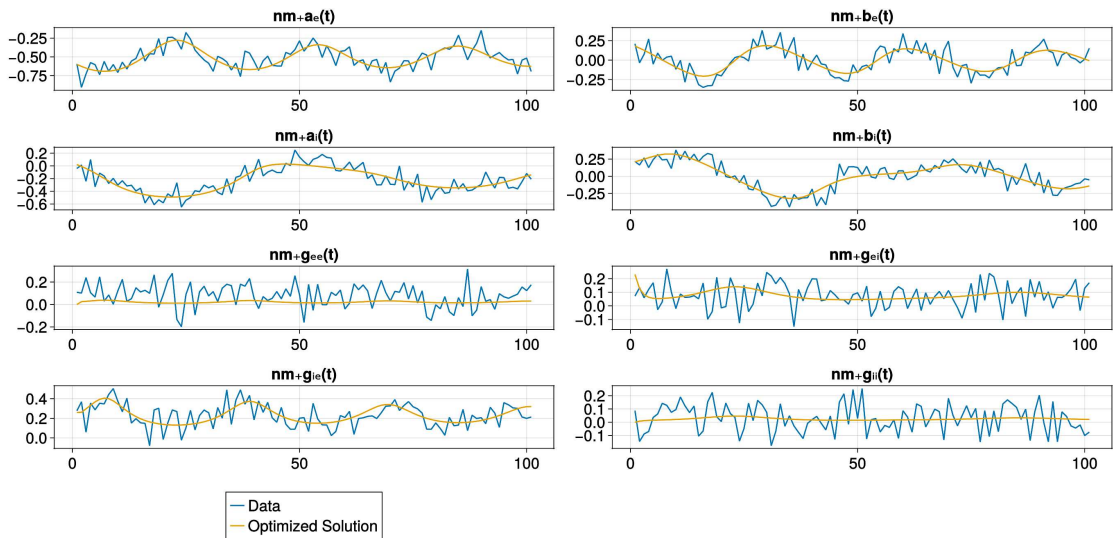


图 2: 优化后参数对应的状态轨迹与观测数据对比，展示拟合显著改善。图源：MIT Neuroblox 官方课程页 optimization。

表 1: ground truth、初始猜测与优化结果的对比

参数	Ground truth	Initial guess	Optimized
k_{ee}	2.0	0.2	0.6622
k_{ii}	1.5	3.3	1.5202
k_{ei}	5.5	2.0	4.5868
k_{ie}	7.0	3.5	6.9863

从表里可以直接看出两点。第一，优化结果整体上比初始猜测更接近 ground truth。第二，参数并没有被“精确还原”到每一位小数都一致，尤其是第一个耦合系数仍有明显偏差。这不是意外，而是噪声观测和逆问题本身都会让参数恢复变成近似问题。

但课程更看重的是轨迹质量：用优化后的参数重新模拟后，时序曲线和数据已经明显贴近，说明模型在动力学层面达到了更好的解释能力。换句话说，这一讲的成功标准不是“数字完全相等”，而是“模型用更合理的参数重建了更合理的行为”。

结果该怎么读

参数表和轨迹图要一起看。参数接近 ground truth 说明优化方向是对的；轨迹更接近数据说明这个方向在建模语义上真的有用。

5.1 本章小结

结果验证告诉我们，拟合成功至少有两层含义：参数值往正确方向收敛，模拟轨迹也同步贴近观测。对于噪声数据，这两层不必完全一致，但应该相互支持。

6 从 synthetic recovery 到 real-data workflow: 这一讲真正教会了什么

如果把这讲只读成“如何调用 LBFGS”，其实会错过它最重要的方法学价值。课程页在 Introduction 里已经明确说，这种 fictive-data fitting 是任何 parameter fitting analysis 的推荐第一步，因为它能告诉你：在当前模型、当前观测变量和当前时间采样下，哪些参数原则上是可恢复的，哪些即使优化器运行成功，也未必能被稳定辨识出来。

这一点和结果表里的现象正好呼应。虽然四个耦合参数总体朝 ground truth 靠近，但并不是每个参数都同样容易被回收到原值附近，尤其是 k_{ee} 的偏差仍然明显。这意味着拟合流程已经成功，但问题本身仍然保留了 inverse problem 的典型特征：多个参数方向对观测的敏感度并不相同。

表 2: 课程页实际上教给读者的参数拟合诊断顺序

步骤	诊断目的
先生成 fictive data	检查模型与观测设置是否具备参数回收可能性
加入 observation noise	让问题更接近真实数据场景，而非无噪声理想环境
给出 initial guess 并可 可视化偏差	判断搜索起点是否过远，误差形态是否合理
再做 least-squares + LBFGS	在标准优化流程中验证参数与轨迹能否共同改善
回头比较参数表与轨迹 图	区分“数值收敛”与“建模上真正有解释力”

这张顺序表的关键在于，它把“拟合成功”拆成了几个不同的问题：优化器有没有报错、参数是否移动到合理区域、轨迹是否贴近数据、以及这个数据本身是否真的足以约束参数。课程并没有把这些问题混成一句“loss 下降了”，而是把它们放在一个更审慎的 workflow 里。

真实数据阶段最该继承的不是哪一个优化器，而是这一套顺序

当你换成真实数据时，最应该保留的不是某个固定数值技巧，而是“先做可辨识性检查，再谈拟合结果解释”的研究习惯。否则，优化流程可能只是把不充分的信息压进一个看似精确的参数向量里。

6.1 本章小结

这一节把课程页中隐含的方法学主张显式化了：synthetic recovery 不是热身，而是逆问题分析的第一道质量控制。真正成熟的参数拟合工作，应该把这一步当成标准流程，而不是可有可无的附加实验。

7 参考文献

课程页在这一讲只给出了一条核心参考：

- Byrd, R. H., Lu, P., Nocedal, J., & Zhu, C. (1995). *A Limited Memory Algorithm for Bound Constrained Optimization*. SIAM J. Sci. Comput., 16, 1190–1208.

这条文献对应的就是课程中使用的 LBFGS 方法。它把这一讲的实现细节和经典数值优化文献连了起来。

7.1 本章小结

参考文献部分虽然短，但它把课程的实现和算法来源对应起来了。对这讲来说，LBFGS 不是一个黑盒名字，而是一条能追溯到经典优化论文的具体方法线索。

7.2 拓展阅读

如果要把这一讲继续推进到更严肃的研究工作，最自然的拓展方向有两个。第一，是回到 LBFGS 所代表的 quasi-Newton 思路，思考在更高维、更多参数和更复杂损失函数下，哪些问题仍适合这样的二阶近似方法。第二，是把课程页反复强调的 synthetic-data check 延伸成更系统的 identifiability study：不是只问“能不能拟合”，而是问“哪些参数组合在现有观测下根本不可区分”。

8 总结与延伸

这一讲的核心信息很集中：参数拟合不是“调参”，而是把模型、数据和优化方法绑成一个可检验的逆问题。课程用假数据先做一遍，是为了回答“这套模型在原则上能不能把参数找回来”；用最小二乘和 LBFGS 去做，是为了把这个问题放进一个标准、可复现的数值流程里。

如果把这一讲迁移到真实数据，结构几乎不变。唯一的变化是：不再用模型生成数据，而是从文件读取观测；不再验证“能不能回收 ground truth”，而是验证“模型能不能解释真实信号”。在这个意义上，假数据拟合是方法学训练，真实数据拟合才是应用阶段。

这一讲留下的两个判断标准

第一，看参数是否往合理方向移动；第二，看重新模拟的轨迹是否更接近数据。前者检验优化，后者检验模型。

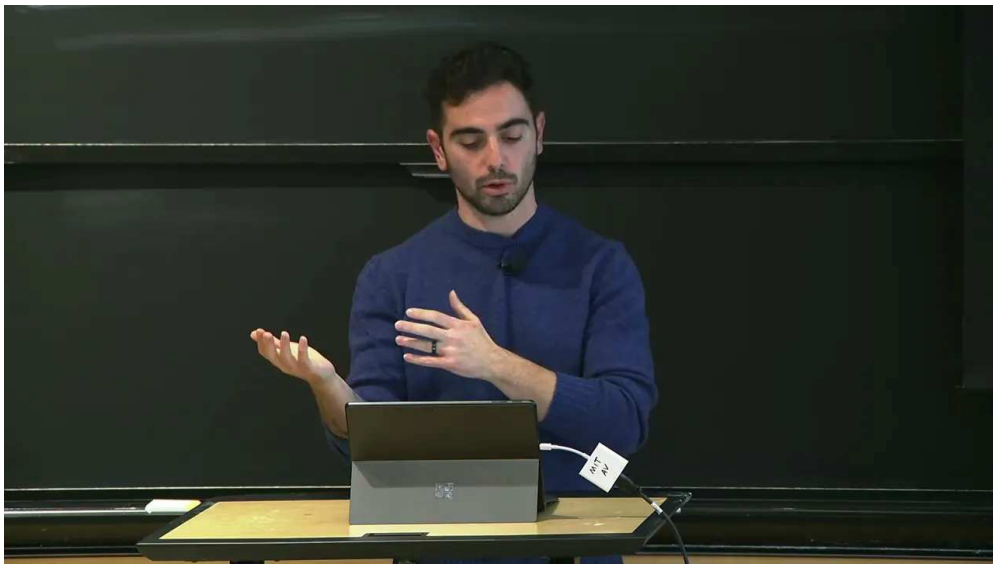
课程还给了一个更宽的提示：最小二乘不是唯一选择，优化目标可以替换成别的 objective。也就是说，这一讲给出的不是单一配方，而是一套可以迁移到不同数据和不同模型上的参数拟合模板。

课程笔记

课程笔记：MIT RES.9- 9第 13 讲谱动态 因果建模的参数拟合

五道口纳什 & Codex

2026 年 4 月 15 日



视频作者/频道：MIT Video Productions

发布日期：2025-04-10（讲课日期：2025-01-17）

视频时长：16:48

视频链接：https://www.youtube.com/watch?v=0Eeyks_HIMI

目录

1 课程定位与逆问题主线 2

1.1 本章小结 2

2 从功能连接到有效连接 3

2.1 本章小结 3

3 双线性状态空间与连接矩阵 3

3.1 本章小结 4

4 神经质量与 BOLD 流水线 4

4.1 本章小结 5

5 观测量提取与噪声处理 5

5.1 本章小结 5

6 交叉谱密度与数据压缩 5

6.1 本章小结 6

7 贝叶斯估计与模型构造 6

7.1 本章小结 7

8 自由能优化与变分推断 7

8.1 本章小结 7

9 结果与参数回收 8

9.1 本章小结 8

1 模型选择、鲁棒性与课堂延伸 8

11 总结与延伸 8

1 课程定位与逆问题主线

这一讲把课程的主轴从“向前模拟”切换到“反向拟合”。前几讲主要回答的是 forward problem：给定模型，把状态往前积分，再看轨迹会长什么样；而这一讲要解决的是 inverse problem：给定时间序列和一个候选模型，怎样回推出更合理的参数，让模型生成的数据尽量接近真实观测。

讲者强调，spDCM 不是单纯做相关性分析，而是把生物物理模型放回推断环节里。它适合神经科学里常见的时序数据，尤其是电生理和影像数据；在 fMRI 语境下，它还天然带着“有效连接”的解释目标，也就是试图恢复区域之间的模型驱动连接，而不是只算 pairwise correlation。

核心判断

功能连接回答“两个区域是否一起变化”，有效连接回答“在一个动力学模型中，哪个区域以什么方式影响另一个区域”。spDCM 的价值就在于把后者变成可估计问题。

Spectral DCM

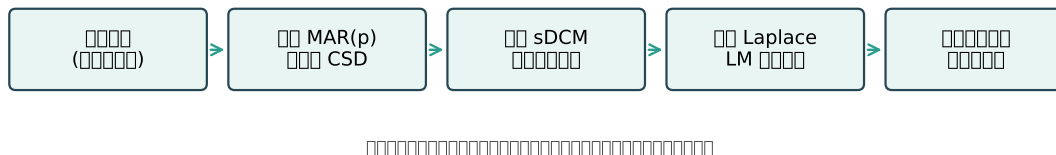


图 1: spDCM 的主流程：先压缩时间序列，再做变分推断，最后比较自由能与后验参数。

讲者还把它和状态空间模型直接连了起来。可以把状态空间理解成对微分方程的一种离散化表达：当前状态不仅依赖自身，还依赖前一时刻或输入项，因此模型自然带有“逐步演化”的味道。这个角度很重要，因为它让 DCM 不再像一个静态统计回归，而更像一个带生理结构的动态系统。

$$\dot{x}(t) = Ax(t) + \sum_{j=1}^m u_j(t)B^{(j)}x(t) + Cu(t)$$

上式对应讲者口中的三块内容：

- $x(t)$ ：隐藏状态，也就是被模型显式描述的神经动力学状态。
- A ：基线有效连接，表示没有输入调制时区域之间的固有耦合。
- $u_j(t)$ ：输入或未建模外源驱动，既可以来自外部刺激，也可以来自不显式建模的脑内群体。
- $B^{(j)}$ ：输入调制连接，表示输入如何改变连接强度。
- C ：直接输入权重，把输入送到状态方程的某些通道上。

1.1 本章小结

这一讲的总目标很明确：不是把一个神经模型继续“正向跑一遍”，而是把观测数据当作约束，反过来识别连接参数、输入参数和模型结构。spDCM 之所以重要，是因为它把动态模型、贝叶斯推断和可解释连接整合进同一个框架。

2 从功能连接到有效连接

讲者先用一个对比把问题讲清楚。功能连接通常来自相关系数或相干性，优势是简单、稳健、计算快；但它只是在统计上描述区域之间是否一起波动，并不回答“为什么一起波动”。有效连接则不同，它直接依赖一个显式模型，关心的是区域之间在动力学上如何传递影响。

因此，DCM 的第一步不是算相关矩阵，而是提出一个关于网络结构的假设：哪些区域存在固有耦合，哪些输入会调制这些耦合，哪些外源驱动需要保留。讲者在视频中也提醒，输入项不一定是外部刺激，它也可以代表脑内未显式建模的活动；昨天用 Poisson spike trains 的思路，就是这种“外源但仍在脑内”的例子。

不要把相关性当成连接

如果只看统计相关性，容易把共同驱动误判成直接耦合；而 spDCM 通过引入结构化动力学模型，试图把“共同输入”和“真实连接”拆开。

官方页面的示意图把这一点写得更具体：先定义图，再往图里放神经质量块和输入块，最后才把图编译成可求解系统。于是所谓“连接”，不再只是边的有无，而是 A、B、C 三类矩阵在系统中的角色分工。

2.1 本章小结

从功能连接走向有效连接，本质上是从纯统计问题走向模型问题。spDCM 的输出不是简单的相关图，而是一个可解释的连接结构，外加对后验不确定性的估计。

3 双线性状态空间与连接矩阵

spDCM 的状态方程具有很强的解释性： A 负责基础连接， B 负责输入调制， C 负责直接输入。讲者特别强调了 bilinear 这个词，因为它说明输入与状态是相乘的关系，而不是把输入当成一个完全独立的回归项。

这件事还有一个推论：如果没有输入，模型只剩下 A ；如果输入存在，输入就可能改变连接的有效强度，于是 B 矩阵就成了动态调制的核心。对于 fMRI 这类任务或静息态数据，这种结构化建模比简单回归更接近生理机制。

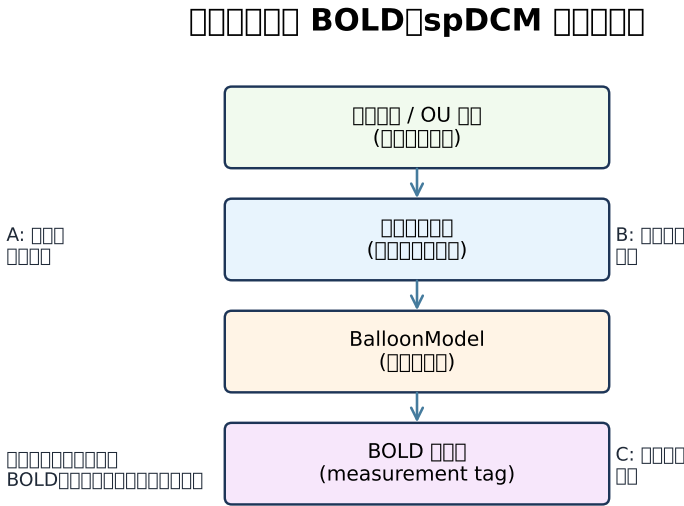


图 2: 一个区域的层级结构示意：外部输入驱动神经质量，再经血流动力学模型映射到 BOLD。

讲者在这里还给出了两条关键假设。第一，信号要近似 weakly stationary，也就是在有限时间窗内的统计特征不能剧烈漂移，这样 cross spectral density 才有意义。第二，参数分布采用 Gaussian / Laplace 近似，这样变分推断才能把后验压缩成均值与协方差。

$$p(\theta \mid y, m) = \frac{p(y \mid \theta, m) p(\theta \mid m)}{p(y \mid m)}$$

上式是 Bayes 规则在 DCM 里的基本形式：

- θ : 待估计参数，包括连接、输入和超参数。- y : 观测数据，这里通常是时间序列或其频域表示。- m : 候选模型。- $p(y \mid \theta, m)$: likelihood，表示参数给定后数据出现的概率。- $p(\theta \mid m)$: prior，表示先验假设。- $p(\theta \mid y, m)$: posterior，表示看到数据后的参数分布。- $p(y \mid m)$: model evidence，用来做模型比较。

3.1 本章小结

双线性模型是这一讲的理论骨架。A、B、C 的分工让“连接”和“输入”在数学上分离开，也让后续的参数拟合可以沿着明确的贝叶斯目标推进。

4 神经质量与 BOLD 流水线

官方页面把实现流程分成非常清楚的三层：神经质量、血流动力学和 BOLD 观测。讲者用三区域模型演示时，先放入线性神经质量块，再为每个区域加 OU 噪声输入，最后叠加 BalloonModel，把神经活动映射为 fMRI 可见的 BOLD 信号。

这个层级的意义在于：BOLD 不是直接观测到的神经活动，而是神经活动经过血流动力学折射之后的结果。于是，观测量标签 measurement 的作用就很重要，它把“要拿来拟合的数据”从模型的其他变量里区分出来。讲者还强调，神经质量块下面得到的是 population average neural activity，上面则是 hemodynamic state equations，再往上才是 BOLD。

```
1 # 1. 组装图：神经质量、输入和观测器
2 # 2. 编译成系统：system_from_graph(...)
3 # 3. 运行仿真并提取 measurement 变量
4 # 4. 加噪声、中心化、重标定
5 # 5. 计算 MAR / CSD，再进入 DCM 拟合
```

Listing 1: Neuroblox 中的实现骨架

这一段流程里，几个函数名值得记住：`get_idx_tagged_vars` 和 `get_eqidx_tagged_vars` 负责从标签中找到观测量；`DataFrame` 用来把求解结果组织成可处理的数据表；`randn` 叠加测量噪声；`mean`、`std` 和重新命名列则对应 SPM 风格的数据预处理。

讲者还解释了一个实现细节：OU 到神经质量、再到 `BalloonModel` 的连接权重在官方示例里取 $1/16$ ，这是为了让模拟更稳定。如果数值不稳，既可以降低 OU 噪声，也可以调小这条边的权重。这个细节看起来琐碎，但它直接决定了后面的频域估计是否会被数值爆炸干扰。

4.1 本章小结

这一部分展示的是 spDCM 的“生物物理管线”。你不是直接拿任意时间序列去拟合，而是先通过神经质量和 `BalloonModel` 把数据生成机制说清楚，再让拟合过程沿着这个机制回推参数。

5 观测量提取与噪声处理

在得到仿真结果后，讲者先把 BOLD 轨迹从系统中提取出来，再加上测量噪声并做中心化、重标定。这样做并不是为了“美化数据”，而是为了把数据变成符合后续频域建模假设的格式。

这里的关键点有两个。第一，官方页面明确提到初始 spike 不具代表性，因为它来自随机过程的平衡阶段，所以要删除；第二，数据要改成与 SPM 一致的尺度和列名，这样后面的 DCM 过程才有可比性。

为什么要重标定

频域拟合对幅值尺度很敏感。中心化和标准化一方面减少数值不稳定，另一方面让不同区域的信号更容易放在同一套先验尺度下比较。

讲者同时指出，这一步不是“把所有信息都抹平”。相反，保留区域间的相对变化、同时消除不必要的偏置，才是为了让 cross spectral density 更能反映真正的相互作用。

5.1 本章小结

噪声处理和重标定的目的，是让时间序列进入一个适合频域推断的规范格式。它看似简单，却是把原始仿真或观测数据变成 DCM 输入的必要门槛。

6 交叉谱密度与数据压缩

spDCM 的一个关键技巧，是不用直接在时域里硬拟合所有状态，而是先把数据压缩成 cross spectral density。讲者在页面里写得很清楚：先用一个 order 为 p 的多元自回归模型去拟合信号，再从该模型参数中解析得到 CSD。

这就是为什么讲者会强调 lower frequencies：在很多脑信号里，主要能量都集中在低频段，而 CSD 正好把这种结构性差异凸显出来。换言之，频域表示既保留了动力学信息，又减少了直接逐时点拟合的复杂度。

$$S_{yy}(\omega) = \text{CSD}(y(t))$$

这里的意思不是要把公式记死，而是要理解它在流程中的位置： $S_{yy}(\omega)$ 是后续 sDCM 真正拿来拟合的对象，而不是原始时序本身。

- ω ：频率。- $S_{yy}(\omega)$ ：观测信号的交叉谱密度矩阵。- p ：MAR 模型阶数。- dt ：采样间隔。

Spectral DCM

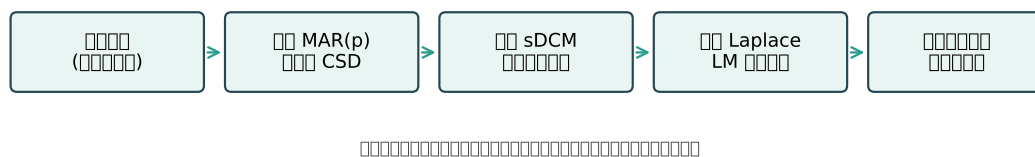


图 3: 同一条流程图再次提醒：CSD 是从时间域进入 DCM 的入口数据。

6.1 本章小结

CSD 的作用是把长时间序列压缩成更适合贝叶斯拟合的统计对象。它没有丢掉动态信息，而是把动态信息换了一种更容易推断的表达。

7 贝叶斯估计与模型构造

进入模型拟合阶段后，官方页面把“模拟模型”换成“拟合模型”。这个模型保留相似的结构，但参数不再全部固定，而是通过 @parameters、tunable=false、changetune 等机制决定哪些参数参与优化，哪些参数保持冻结。

讲者在这里特别强调了先验的重要性。比如有效连接矩阵会先给出一个小扰动的 prior expectation，然后把对角线连接单独处理，以便更容易获得对角占优的稳定结构。也就是说，DCM 不是盲目搜索参数空间，而是在生理合理的先验附近做局部推断。

```

1 # 设定先验、超先验与可调参数
2 # setup_sDCM(...)
3 # for iter in 1:max_iter
4 #     run_sDCM_iteration!(state, setup)
5 # end
6 # freeenergy(state)
7 # ebarplot(state, setup, A_true)
  
```

Listing 2: DCM 拟合阶段的关键接口

讲者还说明了一个非常重要的数值策略：Levenberg-Marquardt 通常比 Gauss-Newton 更稳。原因很直白，后者在离高概率区域较远时更容易漂离；前者会通过阻尼项把更新“拉住”，更容易找到

高概率质量集中的地方。这个性质正是 Bayes 优化里想要的：不是“随便找个极值”，而是尽量沿着 posterior 的主要质量推进。

在自由能层面，讲者把它和机器学习里的 ELBO 对应起来。这里可以把它理解为对后验和证据的一种可优化下界：自由能越高，说明当前近似分布越接近真实后验，或者说当前模型越能解释数据。

$$F(q) = \mathbb{E}_q[\log p(y, \theta | m)] - \mathbb{E}_q[\log q(\theta)]$$

- $q(\theta)$: 变分近似分布。
- $F(q)$: 自由能或 ELBO。
- 第一项：数据与先验共同决定的拟合质量。
- 第二项：近似分布自身的熵项，控制复杂度。

7.1 本章小结

这一部分把模型构造、先验设定和优化目标合在了一起。spDCM 的参数拟合不是黑箱回归，而是“在先验约束下最大化自由能”的贝叶斯优化过程。

8 自由能优化与变分推断

自由能优化的实质，是用变分近似把原本难算的后验积分替换成可迭代求解的问题。讲者在视频中提到，posterior 是参数在给定数据后的分布，而 likelihood 和 prior 的乘积是 Bayes 规则的核心；真正困难的是 evidence，但变分 Bayes 让这一步变得可操作。

官方页面的实现里，`setup_sDCM` 会把频率轴、采样间隔、MAR 阶数和先验等对象组织好，然后 `run_sDCM_iteration!` 在每一步更新状态、自由能和预测自由能变化量。停止准则也说得很清楚：如果连续若干次预测变化低于阈值，就可以认为收敛。

为什么要看 F 和 dF

F 告诉你当前拟合得有多好， dF 告诉你下一步还会不会明显改善。两者一起看，才能判断优化是已经稳定收敛，还是只是暂时停住。

Spectral DCM

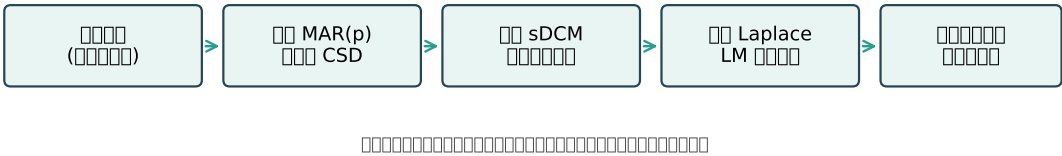


图 4: 自由能、后验和模型比较都在同一条推断链路里。

8.1 本章小结

变分推断的目标不是精确积分，而是在可计算的近似类里找到最好的后验近似。对 spDCM 来说，这个近似的质量直接决定了连接估计和模型比较是否可信。

9 结果与参数回收

拟合完成后，讲者先看自由能曲线，再看参数后验条形图。前者回答“优化有没有收敛”，后者回答“恢复出来的连接是不是接近 ground truth”。当讲者说 larger free energy better 时，本质上就是在说模型越能压低复杂度、越能解释数据，证据就越强。

这里的教学重点是：模型拟合不是一次性给出“真值”，而是给出一个后验均值和方差。方差很重要，因为它告诉你这个连接到底稳不稳、是否被噪声主导。即使均值接近真值，如果不确定性很大，也不能把结果当成高置信结论。

9.1 本章小结

结果页的价值不只在“画出了一张好看的图”，而在于它把点估计和不确定性一起展示出来。spDCM 的输出因此更像一个统计推断结论，而不是普通曲线拟合结果。

1 模型选择、鲁棒性与课堂延伸

最后一部分，讲者把视角从“估计一个模型”扩展到“比较多个模型”。课堂里的做法很清楚：从同一个 ground truth 生成数据，再尝试三种结构不同的模型，最后比较自由能。离真值越远，自由能越差，这说明模型选择确实能把不合适的结构筛掉。

不过讲者也没有把这件事说得过满。挑战题里明确要求你改变随机种子、重复多次、统计不同噪声实例下的结果，并思考自由能和参数回收究竟有多稳健。这里的潜台词是：DCM 很强，但它并不神奇；噪声、模型错设和先验偏差都会影响结论。

课程给出的三个延伸方向

第一，改变随机种子，检查结果对噪声的敏感性；第二，重复多次并做平均，观察 ensemble 统计；第三，换成别的神经动力学模型，比如 Jansen-Rit，再看先验和可调参数如何重新设定。

讲者最后还补了一句工程上的信息：Neuroblox 的 spDCM 是模块化的，神经质量块、连接结构、观测器和推断器都可以替换；而且它借助 ModelingToolkit 和自动微分，在速度上已经能和原始 SPM 路线相竞争，同时保留了符号化模型的灵活性。这一点很重要，因为它说明课程不是只讲原理，而是已经把原理落进了可复用的软件接口。

11 总结与延伸

这一讲把课程前半部分的“建模与仿真”收束到了“参数反演与模型选择”。如果要把主线再压缩一层，那就是：先用神经质量和血流动力学给出一个生理合理的生成模型，再把观测数据压缩成 CSD，最后在贝叶斯框架里用自由能去拟合和比较模型。

从方法论上看，spDCM 的意义有三点。第一，它把有效连接从纯解释性术语变成可计算对象；第二，它把先验、数据和模型复杂度放到同一个目标函数里；第三，它让不同生理层级的模型可以在同一条推断链路里替换和对比。对做计算神经科学的人来说，这种“可解释 + 可拟合 + 可比较”的组合非常有价值。

参考文献

- [1] Novelli, L., Friston, K., and Razi, A. Spectral Dynamic Causal Modeling: A Didactic Introduction and Its Relationship with Functional Connectivity. *Network Neuroscience*, 2024.
- [2] Hofmann, D., Chesebro, A. G., Rackauckas, C., Mujica-Parodi, L. R., Friston, K. J., Edelman, A., and Strey, H. H. Leveraging Julia's Automated Differentiation and Symbolic Computation to Increase Spectral DCM Flexibility and Speed. *bioRxiv*, 2023.
- [3] Friston, K. J., Kahan, J., Biswal, B., and Razi, A. A DCM for Resting State fMRI. *NeuroImage*, 2014.

课程笔记

MIT RES.9-009 第 14 讲: Experimental Design (实验设计)

五道口纳什 & Codex

2026 年 4 月 16 日



视频作者/频道: MIT Video Productions (讲者: Lilianne Mujica-Parodi)

发布日期: 课程日期 2025-01-15; YouTube 上传 2025-04-15

视频时长: 07:05

视频链接: <https://www.youtube.com/watch?v=NU9K81-gg-Q>

目录

1 为什么实验设计不能脱离建模 2

1.1 本章小结 2

2 多通道与长时间窗：不要把动态平均掉 2

2.1 本章小结 3

3 关键实验：让竞争假设在数据上分道扬镳 3

3.1 本章小结 4

4 多尺度验证：从底层到上层都要可测 4

4.1 本章小结 5

5 从课程 reading 回看这一讲：输入设计为什么是模型的一部分 5

5.1 本章小结 5

6 总结与延伸 6

6.1 本章小结 6

证据说明

本讲的官方课程页几乎只提供了标题、视频嵌入和一条参考文献；真正的教学内容主要来自字幕与课程页结构化证据。官方 slide PDF 仍未找到，因此正文没有把“缺失幻灯片”当作教学内容展开，而是将其记录在 `omission_log.jsonl` 中。

1 为什么实验设计不能脱离建模

这讲一开始就把一个常见误区摆在台面上：我们往往习惯于把“数据分析”和“实验设计”分开考虑，但在计算建模里，这两件事必须同时想。原因很简单。模型不是静态表格里的几个值，而是一个需要随时间演化的动力系统；如果实验本身不能支持这种动力学表述，那么模型再漂亮，也只是停留在纸面上的解释。

课程把这个区别说得很直接。传统统计分析更关心的是一个值是不是“更高”或“更低”，是否存在统计学意义上的差别；而建模关心的是，系统在时间上如何变化，这种变化能否被一个 ODE 系统表达出来。换句话说，统计分析经常在比较“值”，建模则是在构造“过程”。

核心区别

统计分析常常回答“有没有差异”；建模必须进一步回答“差异如何随时间展开、如何被机制解释、能否被实验检验”。

课程还提醒了一点：很多为统计目的而优化过的实验，不一定能无缝迁移到建模。也就是说，原本对一个统计检验很友好的实验设计，未必能提供足够的时间分辨率、变量覆盖和因果约束来支撑一个动态模型。这个提醒对课程后面的所有例子都成立。

最早的例子来自 glucose-insulin model。若只看单个时间点，很多不同的 disorder 可能在表面上看起来差不多，甚至近乎等价；但一旦把动态过程纳入视野，它们就会显出差异。课程借这个例子强调了一件事：单点值可以遮蔽机制，时间结构才更接近模型要解释的对象。

1.1 本章小结

本章建立了整门课的逻辑前提：实验设计不是建模完成后的附属步骤，而是建模本身的一部分。模型要能被测试，实验就必须为动态、机制和因果区分提供条件。

2 多通道与长时间窗：不要把动态平均掉

从“值”转向“过程”之后，下一个问题就变成：到底该测什么、在什么位置测、测多久。课程给出的第一个建议是，理想情况下应该尽量采集连续测量，而且要覆盖回路中的多个组成部分。因为如果你面对的是一个负反馈环路，只看一个节点，通常不足以判断环路是如何工作的；至少要同时看到兴奋性和抑制性两个方向，才有机会把动态与因果链条拼起来。

这也解释了为什么课程特别强调“strategically picking those areas”。对做 electrophysiology 的人来说，插入电极本身就是代价高昂的动作；正因如此，更要把有限的测量位置用在最能提供结构信息的地方。若条件允许，测点越多越好，因为覆盖越完整，越容易重建系统动态，也更容易追踪因果关系。

实验设计的方向感

对于动态系统，测量的目标不是“尽量拿到一个平均数”，而是“尽量保留系统中的差异、方向和时序结构”。

课程接着指出，时间序列平均化本身可能是问题。原因不是平均本身错误，而是平均会把动态信号洗平。既然你关心的是系统如何变化，那么过度压缩时间结构就会损失最关键的信息。所以在可能的情况下，应该优先提高 SNR，并尽量保持较长的数据窗口；课程甚至明确说，这一点有时会与神经影像里的某些大规模数据策略相冲突。

这里最值得注意的对比，是课程拿 UK Biobank 一类神经影像设计来作参照。那类研究常见的做法是单个被试的 resting-state 数据很短，但被试人数极大；它的逻辑是“人多补质量”。而本讲的建模语境不同：如果目标是刻画动态机制，就往往更需要长窗口、高质量、可追踪的信号，而不是只靠大量短片段来平均掉个体内部的变化。

表 1: 课程对“统计分析型实验”和“建模型实验”的侧重点区分

视角	常见偏好	对实验设计的含义
统计分析	关注值的高低、显著性、群体差异	单点观测有时足够，但不一定支持机制解释
建模	关注时序、动力学与 ODE 结构	需要连续测量、多个节点、尽量少把动态平均掉
大规模影像	以人数补偿扫描短、单体质量低	适合群体统计，不一定适合精细回路建模

2.1 本章小结

如果实验的目标是给模型提供辨识能力，那么测量位置、测量数量、测量时长和信噪比，都是和理论同等重要的设计变量。课程在这一段的态度很明确：要保留动力学，不要把动力学平均没了。

3 关键实验：让竞争假设在数据上分道扬镳

课程后半段给出的是更具建模味道的思路：利用模型去设计关键实验 (critical experiment)。这不是先把所有参数都钉死，再被动等待数据来“验证”；相反，是先把可能的竞争假设都放进一个 provisional circuit，再看这些假设的输出在什么地方最大程度分歧，然后把实验专门设计在这些分歧点上。

这个方法有两个直接收益。第一，你不再只是问“哪个模型更像数据”，而是问“哪个观测最能区分模型”。第二，模型不再只是一个解释器，而成为实验生成器：它先告诉你应该在哪些点上测，才能最大化辨识力。

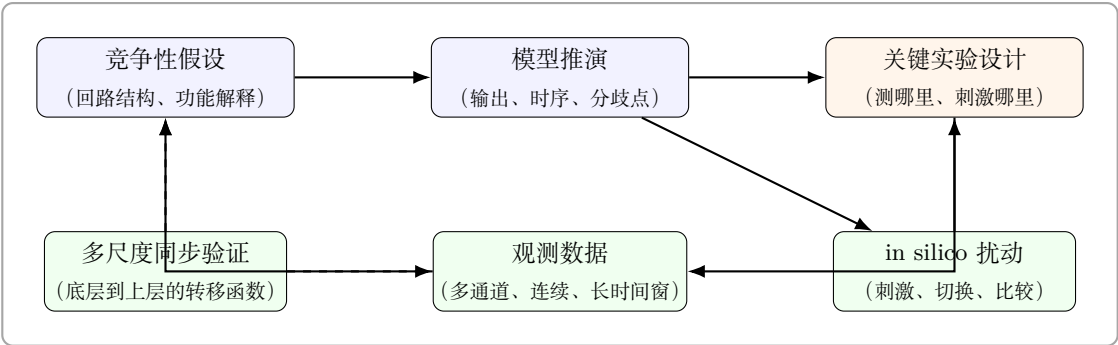


图 1: 关键实验的设计循环: 从竞争假设出发, 经由模型推演找到最大分歧点, 再把实验和扰动放到最有区分度的位置。

课程还把这个思路进一步推进到 *in silico* perturbation。也就是说, 既然已经有了模型, 就可以先在模型里问: 如果我刺激这个位置, 而不是那个位置, 动态会怎样变化? 这种虚拟刺激不会替代真实实验, 但它能帮助你提前筛选出最有信息量的操作条件, 从而把真实实验的资源花在最值得做的地方。

为什么叫“关键实验”

关键实验不是最昂贵的实验, 而是最能把竞争假设区分开的实验。它的判据是区分力, 不是规模感。

课程强调的一个细节也很重要: 有时即使你没有足够信息去完全确定系统结构, 仍然可以先搭一个 provisional circuit, 并穷举其组合方式, 再让数据告诉你哪些配置与观测相容、哪些不相容。也就是说, 模型可以先粗后细, 关键是每一步都在朝“可区分、可检验”推进。

3.1 本章小结

建模在这里的角色不只是解释数据, 而是主动生成实验方案。先比较模型输出的分歧, 再去设计观测和扰动位置, 这是课程对“模型如何服务实验”的最核心回答。

4 多尺度验证: 从底层到上层都要可测

课程最后把视角从单一回路扩展到多尺度系统。若要真正利用 Neuroblox 的 multiscale capabilities, 就不能只盯着某一层的观测; 更稳妥的做法, 是在多个尺度上同时采集数据, 然后据此建立从底层到上一层、再到更高层的 transfer functions。

这一步的意义在于, 模型不再只是某一尺度上的拟合, 而是可以追问“怎样从底层走到上层”。课程用的表达很朴素: 你要知道如何从 bottom layer 到 next layer, 再到 next layer。这样的表达背后, 其实是在要求一个跨尺度的可追踪映射, 而不是只在单尺度上看起来好看。

最后的建模标准

如果你的最终目标是 rigorous model, 那么多尺度验证不是可选项, 而是模型可信度的一部分。单尺度看起来正确, 并不等于跨尺度也成立。

⁰ 依据字幕 00:04:28-00:06:18 以及 00:06:23-00:07:04 的内容综合绘制。

课程也没有回避难度：这样的实验并不容易做，尤其当同时采集多个尺度意味着额外的设备、同步、噪声控制和分析复杂度时。不过，课程的态度并不是“因为难就不做”，而是“如果目标真的是 rigorous model，就必须把这个难题算进去”。

4.1 本章小结

多尺度验证把实验设计从“测到一点信息”提升为“建立尺度之间的可追踪关系”。这也是课程最后把实验设计和模型严谨性重新绑回一起的地方。

5 从课程 reading 回看这一讲：输入设计为什么是模型的一部分

这一讲虽然只有一条官方 reading，但这条 reading 的题目本身已经把课程想说的话压缩得很清楚：*Input Signal Design for System Identification*。课程视频一直在强调，不要把实验刺激或输入条件当作数据采集前的背景设定；相反，输入如何被设计，往往直接决定了系统是否可辨识、竞争假设是否能被分开，以及多尺度 transfer function 是否能被稳定估计出来。

需要注意的是，这里我们不能把那篇 IFAC 论文的详细技术内容强行写进正文，因为课程页只显式给出了书目信息，没有附上论文全文。但即便只根据标题和课程本身的论述，也已经足够得出一个安全且重要的结论：**输入设计不是实验开始前的行政安排，而是系统辨识的一部分。**

表 2: 本讲视频内容与官方 reading 标题之间的对应关系

课程中的表述	reading 标题所强化的方向
不要只看单点值，要看动态过程	输入设计必须服务于动态辨识，而不是只服务于静态比较
要找 critical experiment	输入或扰动应该被放在最能区分竞争假设的位置
要做多尺度验证	输入方案必须让不同尺度上的响应都留下可追踪结构

换句话说，这一讲真正教的不是某一种“最好实验范式”，而是一种更严格的提问方式：如果我改变输入、刺激、采样位置和采样时长，哪些设计会让模型更容易被区分，哪些设计则只会制造一堆统计上可比较但机制上不可判别的数据。

把 reading 用在什么地方最合适

这条 reading 最适合被当作方法论提醒来读，而不是当作本讲的唯一知识来源。视频负责给出建模语境与设计原则，reading 的标题则把这些原则和 system identification 的语言重新接起来。

5.1 本章小结

这一节把课程页上看似孤零零的一条参考文献重新放回了整讲的语境里。它提醒我们，实验输入本身就是建模对象的一部分，因此“怎样刺激”“刺激多久”“在哪些点上刺激”都不能和模型辨识分开谈。

6 总结与延伸

这讲的主线其实很统一：对计算神经科学来说，实验设计不是建模之外的外部条件，而是模型是否站得住脚的前提。先要有动态观测，而不是只看单点值；再要有多通道覆盖，而不是只盯一个节点；然后要用模型构造关键实验，找出竞争假设的最大分歧；最后还要做多尺度验证，让模型不只在一個层面上成立。

如果把整讲压缩成一句话，那就是：**好的实验设计，是为了让模型“有机会被证明错”，从而更可靠地被证明对。**

官方课程页参考

官方课程页在这一讲只列出了一条参考文献：A. Królikowski 与 P. Eykhoff, *Input Signal Design for System Identification: A Comparative Analysis*, IFAC Proceedings Volumes, 18(5), 1985, 915–920, doi 链接。这与本讲的主题很一致：实验输入怎么设计，往往决定了系统能否被辨识出来。

6.1 本章小结

实验设计的价值，不是把数据变得更像预期，而是把数据变得更能回答模型问题。对于 Neuroblox 这类分层、动态、可扰动的建模框架，这一点尤其重要。

1 术语表

本附录是占位骨架，用于后续按章汇总统一译名与关键术语定义。

术语		说明
Neuroblox		后续章节会给出统一译名、项目背景和使用语境。
computational science	neuro-	后续章节会统一给出中文译法与边界说明。
spiking neuron		后续章节会按具体模型给出正式定义。
synaptic plasticity		后续章节会按课程覆盖范围整理。

2 符号约定

本书会尽量保持各章符号一致；若某章沿用官方 slide 的记号但与全书记号不同，merge 阶段应在正文中显式说明。

符号	说明
x_t	时间步 t 的输入。具体含义由章节上下文决定。
y_t	时间步 t 的输出或观测。
V_m	膜电位 (membrane potential)；仅在相关章节中启用。
w_{ij}	神经元或状态之间的连接权重。